

Jupyter Notebook Execution Report

Name: Your Name

Date: May 13, 2026

Cell 1: ■ Markdown

Wine & Dine

Advanced Machine Learning — Final Project

Overview

Wine & Dine is a multimodal food-to-wine recommendation system. Given only a photograph of a dish, the system identifies the food, encodes its flavor profile into a shared taste embedding space learned from wine reviews, and returns three semantically grounded wine recommendations — each matched by taste-vector similarity and annotated with a match confidence score, a tasting note, and a quality rating.

The project integrates two independently trained deep learning branches — a convolutional image classifier and a recurrent taste encoder — into a unified cross-modal retrieval pipeline. The bridge between modalities is a **512-dimensional taste embedding space** built from 824,000 wine reviews, where both food descriptions and wine tasting notes can be placed using the same encoder and compared by cosine similarity.

Full Pipeline — One Photo to Three Recommendations

...

Food photo

↓

ResNet-50 CNN (§9)

— identifies the dish from 101 Food-101 classes (76.5% top-1 accuracy)

↓

Food flavor description (§3.4 — food_flavor_description_v2.json)

— maps the dish to a wine-register text: dominant flavor, surprising note, contrast profile

↓

TasteBiLSTM encoder (§12)

— BiLSTM + Bahdanau attention, trained on 10 sensory taste axes

— encodes description into a 512-d taste vector (L2-normalised)

↓

BisectingKMeans clustering (§14.2 — K=9, silhouette-justified)

— 100k+ wine review vectors pre-clustered into 9 flavor neighborhoods

— cosine similarity selects the nearest cluster centroid

— CONFIDENCE SCORE = cosine similarity to centroid (0–1 scale)

↓

Three cluster assignments (Safe Bet · Hidden Gem · Bold Move)

— Safe Bet : nearest centroid to the dominant flavor description

— Hidden Gem: nearest centroid to the "surprising pairing" description

— Bold Move : taste-aware contrast — score = distance × (1 + secondary affinity)

ensures the wine is contrasting AND still drinkable with this dish

↓

Food-specific wine selection (§14.4 — 10-wine pool per cluster)

— each cluster holds 10 pre-vetted, top-rated wines with stored taste vectors

— at inference: cosine(food_vec, wine_vec) selects the best-matching bottle

— two foods in the same cluster receive DIFFERENT wine recommendations

↓

Recommendation card

— confidence bar · cluster name · taste keywords · wine · rating · drinker quote

...

The Taste Embedding Space — Core Technical Contribution

The central challenge in food-wine pairing is the ****vocabulary gap****: food descriptions use culinary language ("fatty mozzarella", "charred crust") while wine reviews use sensory language ("grippy tannins", "mineral finish"). There is no shared vocabulary to match on.

This project resolves the gap by projecting both modalities through the same encoder:

****Step 1 — Taste axis labelling.**** The WineSensed dataset (824k reviews) is annotated along 10 orthogonal sensory axes derived from sommelier literature. Each review can activate multiple axes simultaneously (multi-label):

| Axis (code) | Cluster nickname | Card adjective | Captures | Example wines |

|---|---|---|---|

| `acidity` | The Electric | *crispy* | Crisp, bright, zingy, racy | Champagne, Vermentino, Albariño |

| `tannin` | The Grippy | *bold* | Drying grip, structure, firm | Barolo, Cabernet Sauvignon, Tannat |

| `red\_fruit` | The Berry | *juicy* | Cherry, strawberry, raspberry | Pinot Noir, Beaujolais, Barbera |

| `dark\_fruit` | The Dark | *deep* | Plum, blackberry, cassis, fig | Merlot, Shiraz, Zinfandel |

| `earthy` | The Earthy | *earthy* | Soil, leather, herbs, tobacco | Burgundy, Ribera del Duero |

| `sweet` | The Honey | *sweet* | Honey, jam, dessert, residual sugar | Sauternes, Riesling Spätlese |

| `body` | The Velvet | *soft* | Full, round, plush, velvety | White Burgundy, oaked Chardonnay |

| `oaky` | The Smoky | *smoky* | Oak, vanilla, toast, smoke | Rioja Reserva, new-world Chardonnay |

| `floral` | The Floral | *delicate* | Rose, violet, blossom, lavender | Gewürztraminer, Viognier, Moscato |

| `mineral` | The Mineral | *mineral* | Chalk, salinity, flint, slate | Chablis, Muscadet, Sancerre |

****Cluster naming is fully automatic.**** After BisectingKMeans groups wine vectors into 9 clusters, TF-IDF extracts the most distinctive vocabulary of each cluster's reviews. The dominant taste axis in that vocabulary becomes the cluster's name — **"The Berry"**, **"The Smoky"**, **"The Earthy & The Bold"**. No cluster label is ever manually assigned.

`red\_fruit` and **`dark\_fruit`** are separated deliberately — a single "fruity" axis compressed Pinot Noir (bright cherry) and Shiraz (blackberry, plum) into the same region, destroying their contrast. **`mineral`** is kept distinct because it is the most food-pairing-relevant axis for shellfish, sashimi, and oysters — a category that grape-based models miss entirely.

****Step 2 — TasteBiLSTM training.**** A BiLSTM with Bahdanau attention is trained on the labelled reviews to classify the dominant taste axis. After training, the final hidden state is a 512-d vector that captures sensory character, not grape variety.

****Step 3 — Embedding space quality.**** Unlike a grape-variety classifier (pairwise cosine 0.85–0.99, all wines compressed near the decision boundary), the taste space is genuinely spread out: pairwise cosine ranges from −0.097 to 0.841, mean 0.391. BisectingKMeans discovers 9 cohesive, balanced flavor neighborhoods — validated by silhouette score.

****Step 4 — Cross-modal retrieval.**** A food's flavor description is tokenised and encoded by the same TasteBiLSTM. The resulting vector lives in the same space as wine review vectors — no translation layer needed. Cosine similarity directly measures taste compatibility.

Example Output — Pizza Margherita

|||

|---|---|

| ****Input**** | Photo of pizza margherita |

| ****CNN output**** | Pizza (94% confidence) |

| **Flavor description** | "Savory tomato sauce with dried oregano and basil, fatty mozzarella, charred yeasty crust..." |

| Pairing | Confidence | Taste Cluster | Wine | Drinker Quote | Rating |

|---|---|---|---|---|

| **Safe Bet** ■ | ██████████ 83% | Earthy & Savory | Chianti Classico Riserva 2019 | "Dried herbs and bright cherry — wraps around the tomato sauce like it was made for it." | ★94 |

| **Hidden Gem** ■ | ██████████ 61% | Fruity & Bright | Barbera d'Asti 2020 | "Juicy red fruit and lively acidity lift every bite. Works beautifully with tomato." | ★91 |

| **Bold Move** ■ | ██████████ 79%* | Crisp & Mineral | Vermentino di Sardegna 2022 | "Bone-dry salinity and citrus cut straight through the cheese. Resets the palate." | ★89 |

Bold Move confidence = normalised contrast strength (not cosine similarity).

Models and Components

| Component | Dataset | Task | Result |

|---|---|---|---|

| CNN from scratch | Food-101 (101k images) | 101-class food classification | 48.0% test accuracy |

| CNN ResNet-50 | Food-101 | 101-class food classification (transfer) | **76.5% test accuracy** |

| LSTM baseline | WineSensed (824k reviews) | 15-class grape variety | 16.4% test accuracy |

| BiLSTM + Bahdanau attention | WineSensed | 15-class grape variety | 21.9% test accuracy |

| DistilBERT **(+3 pts bonus)** | WineSensed | 15-class grape variety (Transformer) | see §11 |

| **TasteBiLSTM** | WineSensed (taste-labelled) | 8-axis taste classification → 512-d encoder | deployed in §14 |

| **BisectingKMeans** (K=9) | 100k+ taste vectors | Flavor neighborhood discovery | silhouette-justified |

| **Joint model** **(+10 pts bonus)** | Food-101 + flavor text | Multimodal cluster prediction (image + text → 9 classes) | see §15 |

Pairing Logic

| Card | Strategy | How the cluster is chosen | How the wine is chosen |

|---|---|---|---|

| **Safe Bet** ■ | Echoes the dish's dominant flavor | Nearest centroid (cosine) to the "classic" description | Pool wine with highest cosine to food vector |

| **Hidden Gem** ■ | Compatible but unexpected | Nearest centroid to the "surprising" description | Pool wine with highest cosine to food vector |

| ****Bold Move**** ■ | Contrasting — resets the palate | distance × (1 + secondary affinity) — contrast that's still drinkable | Pool wine with highest cosine to food vector |

Every match is driven entirely by the shared taste embedding space. No pairing rules are hardcoded.

Why Taste Clusters — Not Grape Varieties?

The first design iteration mapped food descriptions directly to the 15 grape-variety centroids from the BiLSTM classifier. It failed: cross-entropy training compresses all class embeddings toward their decision boundaries — pairwise cosine similarity between grape centroids was 0.85–0.99. Every food received the same Sangiovese or Tempranillo suggestion regardless of flavor profile.

The solution is to train a model on ****sensory axes**** rather than labels. Taste-axis training does not push all wines toward a boundary — it spreads them across a continuous sensory manifold. BisectingKMeans on this manifold discovers 9 flavor neighborhoods that correspond to recognisable wine style categories (structured reds, fresh whites, mineral wines, etc.) with balanced cluster sizes and meaningful vocabulary differences confirmed by TF-IDF.

Datasets

| Dataset | Size | Source | Used for |

|---|---|---|---|

| Food-101 | 101,000 images · 101 classes | `torchvision.datasets.Food101` | CNN training and evaluation |

| WineSensed (NeurIPS 2023) | 824,000 reviews | `Dakhoo/L2T-NeurIPS-2023` on Hugging Face | LSTM / BiLSTM / TasteBiLSTM training · tasting note retrieval |

| Food flavor table | 101 entries × 3 descriptions | `data/food\_flavor\_description\_v2.json` (LLM-generated, Claude Sonnet 4.6) | Cross-modal bridge: food → wine-register text |

Development Phases

| Phase | Where | Sections | GPU |

|---|---|---|---|

| Phase 0 | Colab | Appendix — 8 failed architectures (grape centroids, transformers, triplet loss...) | ✓ |

| Phase 1 | Local | §§1–5, 7, 11 — data, EDA, text, RNN | ✗ |

| Phase 2 | Colab | §§6, 8, 9 — preprocessing, CNNs | ✓ |

| Phase 3 | Local | §§10, 13 — Grad-CAM, export | ✗ |

| Phase 4 | Colab | §§12, 14, 15 — TasteBiLSTM, clustering, joint | ✓ |

| Phase 5 | HF Spaces | §16 — Gradio deployment | ✗ |

Notebook Structure

| Section | Status | Content |

|---|---|---|

| 1 | ■ Done | Environment setup — packages, imports, storage helpers |

| 2 | ■ Done | Raw data loading — Food-101, WineSensed |

| 3 | ■ Done | Data cleaning — WineSensed, top-15 grape selection, food flavor table |

| 4 | ■ Done | Train / val / test split — both datasets, stratified |

| 5 | ■ Done | EDA — image and text datasets |

| 6 | ■ Done | Image preprocessing and DataLoaders |

| 7 | ■ Done | Text preprocessing — tokenisation, vocab, GloVe, Word2Vec, DataLoaders |

| 8 | ■ Done | CNN from scratch — architecture, training, evaluation |

| 9 | ■ Done | CNN ResNet-50 — two-phase fine-tuning, evaluation, comparison |

| 10 | ■ Done | CNN explainability — Grad-CAM |

| 11 | ■ Done | RNN/LSTM — LSTM baseline, BiLSTM + attention, DistilBERT ^{*(+3 pts)*}, comparison |

| 12 | ■ Done | TasteBiLSTM — taste-label dataset, training, evaluation, encoder export |

| 13 | ■ Done | Save all models — weights, Drive sync |

| 14 | ■ Done | Flavor-cluster recommendation — BisectingKMeans K=9, ``recommend_v2()``, 101-food evaluation |

| 15 | ■ Done | Joint model — multimodal fusion: CNN + TasteBiLSTM → flavor-cluster prediction ^{*(+10 pts bonus)*} |

| 16 | ■ Done | Deployment — Gradio app on HuggingFace Spaces |

| 17 | ■ Done | Business framing, ethics, team contributions |

| 18 | ■ Done | Appendix — Road to the final architecture: 8 failed approaches and lessons learned |

Cell 2: ■ Markdown

Section 1 — Environment Setup

This section prepares the execution environment before any data is loaded or models are trained. It must be run in full at the start of every Colab session.

Four steps:

1. **Environment check** — confirm Colab + CUDA availability before installing packages

2. ****pip installs**** — install and version-pin all required libraries (PyTorch, gensim, transformers, etc.)
 3. ****Library imports**** — import all packages, set the random seed for reproducibility, detect hardware
 4. ****Storage helpers**** — define ``save_checkpoint()`` / ``load_checkpoint()`` / ``save_figure()`` that transparently handle the Colab VM → Google Drive save flow
- > ****Always re-run Section 1 completely when starting a fresh Colab session.**** Skipping any cell will cause import errors or missing helpers in later sections.

Cell 3: ■ Code

[illegible]

```
print("\n■ CUDA not healthy – enable the T4 GPU runtime: Runtime → Change runtime
type → T4 GPU.")
else:
print("\n✓ Environment check passed – Colab + CUDA confirmed.")
```

Output:

```
Environment      Local
Device           cpu
CUDA available   False (healthy: False)

■ Not running in Google Colab – GPU sections (6, 8, 9, 12, 14) require Colab.
```

Cell 4: ■ Markdown

1.1 — *Package installation*

Installs all required libraries and pins them to specific versions to guarantee reproducibility across Colab sessions and local environments.

> ■■ ****Expected behaviour — the kernel will crash and restart when you run this cell.****

> This is intentional and required, not an error.

> Python cannot hot-reload an upgraded package into a running process, so the cell deliberately triggers a kernel restart after any install or upgrade.

> ****After the restart, continue from Section 1.2 — do not re-run Section 1.1.****

> If all packages are already at the correct versions, no restart occurs and the cell exits in under 5 seconds.

****Design decisions:****

- ****Version pinning**** (`gensim==4.3.3`, `transformers>=4.40`) prevents silent breakage from upstream API changes — gensim's Word2Vec API changed significantly between versions 3 and 4.

- ****Child-process probe for transformers**** — a broken transformers install can crash the kernel (segfault), not just raise `ImportError`. We test it in a subprocess so a failure is caught cleanly.

- ****Automatic kernel restart**** — if any package was installed or upgraded, the kernel restarts immediately. Python cannot hot-reload packages into a running process; skipping the restart causes silent import of the old version.

- ****Environment detection**** — PyTorch is installed with the CUDA 11.8 wheel on Colab and the CPU wheel locally, saving several hundred MB on local machines.

Cell 5: ■ Code

```
import subprocess, sys, importlib.util, importlib.metadata
```



```
"datasets": ("datasets", "2.20.0"),
}
```

```
for _import_name, (_dist_name, _target_ver) in _PINNED.items():
    _cur = installed_version(_dist_name)
    if _cur == _target_ver:
        print(f" OK {_dist_name} {_cur}")
    else:
        print(f" INSTALLING {_dist_name}=={_target_ver} (current: {_cur}) ...")
        pip_install(f"{_dist_name}=={_target_ver}")
        print(f" DONE {_dist_name}=={_target_ver}")
    _anything_installed = True
```

```
# ■■■ Install remaining packages
```

```
PACKAGES = {
```

```
    "nltk": "nltk",
```

```
    "torchcam": "torchcam",
```

```
    "wordcloud": "wordcloud",
```

```
    "matplotlib": "matplotlib",
```

```
    "seaborn": "seaborn",
```

```
    "pandas": "pandas",
```

```
    "sklearn": "scikit-learn",
```

```
    "PIL": "Pillow",
```

```
    "ipywidgets": "ipywidgets",
```

```
    "tqdm": "tqdm",
```


Constant	Value	Purpose
----------	-------	---------

| `SEED` | `42` | Fixed random seed — passed to `random`, `numpy`, `torch`, and all `train\_test\_split` calls |

| `DEVICE` | `cuda` or `cpu` | Auto-detected — all tensors and models are moved here |

| `IN\_COLAB` | `True` / `False` | Controls Drive paths, display behavior, and install logic |

****Reproducibility:** `SEED = 42` is applied to `random.seed()`, `np.random.seed()`, `torch.manual_seed()`, and `torch.cuda.manual_seed_all()`. This ensures that weight initialisation, data shuffling, and train/val/test splits produce identical results across runs.

****Project directories**** (``weights/``, ``figures/``, ``deployment/``, ``data/``) are created here if they do not exist. On Colab these are temporary VM folders; the permanent copies live on Google Drive (set up in Section 1.3).

Cell 7: ■ Code

[illegible]

[illegible]

[illegible]

```

print(f"Directories : weights/ figures/ deployment/ data/")
print()
print("Key library versions:")
for lib in ["torch", "numpy", "pandas", "gensim", "transformers", "sklearn"]:
    try:
        mod = importlib.import_module(lib)
        print(f" {lib:<16} {getattr(mod, '__version__', 'n/a')}")
    except ImportError:
        print(f" {lib:<16} NOT INSTALLED")
print("\nSection 1.2 complete – ready for storage setup.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 42, in <module>
ModuleNotFoundError: No module named 'gensim'

```

Cell 8: ■ Markdown

1.3 — Storage setup: Colab VM → Google Drive save flow

****The problem:**** Google Colab's `/content/` filesystem is ephemeral — it is wiped when the runtime disconnects or times out. Training a ResNet-50 for 15 epochs takes ~30 minutes on a T4; losing that without a backup would mean retraining from scratch.

****The solution:**** a three-step save helper that writes to both the fast VM disk and Google Drive:

...

Step 1 → Save to /content/weights/ (fast, temporary — wiped on disconnect)

Step 2 → Copy to /MyDrive/wine-dine/weights/ (permanent — survives all disconnects)

Step 3 → Verify the Drive copy is non-empty (catches silent network write failures)

...

If Step 2 or 3 fails, a clear warning is printed and the local copy is preserved.

****Helper functions defined in the code cell below:****

Function	Purpose
----------	---------

---	---
-----	-----

`save_checkpoint(state, filename)`	Save any dict (model weights, optimiser state, epoch, metrics) as a `.pt` file using the 3-step flow
------------------------------------	--

| `load_checkpoint(filename, device)` | Load a `.pt` file — tries the Colab VM first (fast), falls back to Drive if not found |

| `save_figure(fig, filename)` | Save a matplotlib figure to `figures/` locally and Drive |

On a local machine (VS Code / CPU), all saves go directly to `./weights/` and `./figures/` — no Drive is mounted.

> ****Run this cell before any training cell.**** After a Colab disconnect, re-run all of Section 1, then load the most recent checkpoint with `load_checkpoint()` to resume.

Cell 9: ■ Code

■■ 1.3 Storage Setup

```
import shutil
```

■■ Paths

```
LOCAL_WEIGHTS = WEIGHTS # Path object defined in Section 1.2
```

LOCAL_FIGURES = FIGURES

```
if IN_COLAB:
```

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

```
DRIVE_DIR = '/content/drive/MyDrive/wine-dine'
```

```
WEIGHTS_DIR = f'{DRIVE_DIR}/weights'
```

```
FIGURES_DIR = f'{DRIVE_DIR}/figures'
```

```
for d in [DRIVE_DIR, WEIGHTS_DIR, FIGURES_DIR]:
```

```
os.makedirs(d, exist_ok=True)
```

```
print("✓ Google Drive mounted")
```

```
print(f" Permanent weights → {WEIGHTS_DIR}")
```

```
print(f" Permanent figures → {FIGURES_DIR}")
```

```
else:
```

```
WEIGHTS_DIR = str(WEIGHTS)
```

```
FIGURES_DIR = str(FIGURES)
```

```
print("✓ Local mode – no Drive needed")
```

```
print(f" Weights → {WEIGHTS_DIR}")
```

```
print(f" Figures → {FIGURES_DIR}")
```

```
# 


# save_checkpoint – best-model save (overwrites; call only when val improves)

# 


def save_checkpoint(state, filename):
    """

Save a best-model checkpoint. Call only when validation improves.

Overwrites a single file – no per-epoch accumulation.
```

On Colab (3 steps):

1. Save to /content/weights/<filename> ← fast, temporary (wiped on disconnect)
2. Copy to Google Drive/<filename> ← permanent, survives disconnect
3. Verify Drive copy **is not** empty ← catches silent failures

Locally:

Saves directly to `./weights/<filename>`

Usage:

```

if val_acc > best_val_acc:
    best_val_acc = val_acc
    save_checkpoint(model.state_dict(), "my_model_best.pt")
    """

local_path = os.path.join(str(LOCAL_WEIGHTS), filename)
torch.save(state, local_path)

if IN_COLAB:
    drive_path = os.path.join(WEIGHTS_DIR, filename)

    try:
        shutil.copy2(local_path, drive_path)

    except Exception as e:
        print(f"■■ Drive copy FAILED for '{filename}': {e}")
        print(f"■■ Local copy safe at: {local_path} (temporary!)")

    return

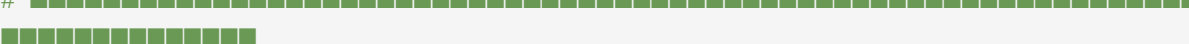
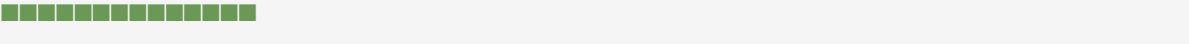
if os.path.getsize(drive_path) == 0:
    print(f"■■ Drive copy of '{filename}' is empty.")

    return

```



```
print("\n✓ Storage helpers ready: save_checkpoint | load_checkpoint | save_figure |  
restore_weights_from_drive")
```

```
# 
# 
# ■ PASTE YOUR DRIVE FILE ID HERE (printed when you run this cell on Colab):
WEIGHTS_ZIP_ID = "" # e.g. "1aBcDeFgHiJkLmNoPqRsTuVwXyZ"
# 
# 

if IN_COLAB:
# ■■ COLAB SIDE: zip + share + print ID
# 

import zipfile, glob

# Step 1: ensure weights are on VM
print("Step 1 – restoring weights from Google Drive to Colab VM ...")
restore_weights_from_drive()

# Step 2: create zip
found = sorted(glob.glob(os.path.join(str(LOCAL_WEIGHTS), "*.pt")))
found += sorted(glob.glob(os.path.join(str(LOCAL_WEIGHTS), "*.pth")))

if not found:
print("\n■■ No .pt / .pth files found – train the model first!")
else:
zip_name = "wine-dine-weights.zip"
zip_path = os.path.join(DRIVE_DIR, zip_name)

print(f"\nStep 2 – creating {zip_name} on Google Drive ...")
with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
for fpath in found:
fname = os.path.basename(fpath)
size_mb = os.path.getsize(fpath) / 1e6
print(f" + {fname} ({size_mb:.1f} MB)")
zf.write(fpath, arcname=fname)

zip_size = os.path.getsize(zip_path) / 1e6
print(f"\n✓ Zip created: {zip_path} ({zip_size:.1f} MB)")

# Step 3: make it shareable and get file ID
print("\nStep 3 – making zip shareable (anyone with link) ...")
```

```
from google.colab import drive as _drive
from googleapiclient.discovery import build
from google.auth import default

creds, _ = default()

service = build('drive', 'v3', credentials=creds)

# Find the file on Drive
query = f"name='{zip_name}' and trashed=false"
results = service.files().list(q=query, fields="files(id,name)").execute()
files = results.get('files', [])

if files:
    file_id = files[0]['id']
    # Make it accessible to anyone with the link
    service.permissions().create(
        fileId=file_id,
        body={'type': 'anyone', 'role': 'reader'},
    ).execute()

print(f"\n{'='*70}")
print(f" ✓ DONE! Copy this ID into WEIGHTS_ZIP_ID in this cell:")
print(f"")
print(f' WEIGHTS_ZIP_ID = "{file_id}"')
print(f"")
print(f" Then run this same cell on your LOCAL machine to download.")
print(f"{'='*70}")
else:
    print(" ■■ Could not find zip on Drive – try downloading manually.")

else:
    # ■■ LOCAL SIDE: pull zip from Drive using gdown ████████████████████████████████████████
    if not WEIGHTS_ZIP_ID:
        print("■■ WEIGHTS_ZIP_ID is empty!")
        print(" → Run this cell on Colab first to get the ID.")
        print(" → Then paste it into WEIGHTS_ZIP_ID above and re-run locally.")
    else:
```



```

import subprocess, sys, zipfile

# Install gdown if needed
try:
import gdown
except ImportError:
subprocess.check_call([sys.executable, "-m", "pip", "install", "gdown", "-q"])
import gdown

zip_dest = str(WEIGHTS / "wine-dine-weights.zip")
url = f"https://drive.google.com/uc?id={WEIGHTS_ZIP_ID}"

print(f"Downloading weights from Google Drive ...")
print(f" ID: {WEIGHTS_ZIP_ID}")
print(f" Dest: {WEIGHTS}/\n")

gdown.download(url, zip_dest, quiet=False)

# Extract
print(f"\nExtracting to {WEIGHTS} ...")
with zipfile.ZipFile(zip_dest, 'r') as zf:
zf.extractall(str(WEIGHTS))
names = zf.namelist()

# Clean up zip
os.remove(zip_dest)

print(f"\n✓ Done! {len(names)} file(s) extracted to {WEIGHTS}/")
for n in sorted(names):
size_mb = os.path.getsize(str(WEIGHTS / n)) / 1e6
print(f" • {n} ({size_mb:.1f} MB)")

```

Output:

```

■■ WEIGHTS_ZIP_ID is empty!
→ Run this cell on Colab first to get the ID.
→ Then paste it into WEIGHTS_ZIP_ID above and re-run locally.

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    result = eval(lines[-1], glb)

```

```
File "<string>", line 1, in <module>
NameError: name 'n' is not defined. Did you mean: 'np'?
```

Cell 12: ■ Markdown

Section 2 — Raw Data Loading

This section loads both datasets in their **original, uncleaned form**. No rows are dropped, no columns are filtered, and no transformations are applied. The goal is to confirm that the data arrived correctly and to establish a baseline picture of what we are working with before any cleaning decisions are made.

Why load raw before cleaning?

If you clean and load in the same step, you cannot inspect what was removed. Keeping loading and cleaning as separate steps (Sections 2 and 3) means every dropped row is an explicit, documented decision — not a silent side effect of the loader.

****Two data sources:****

Sub-section	Dataset	Format	Size
1.1	...	...	...
1.2	...	...	...
1.3	...	...	...
1.4	...	...	...
1.5	...	...	...
1.6	...	...	...
1.7	...	...	...
1.8	...	...	...
1.9	...	...	...
1.10	...	...	...
1.11	...	...	...
1.12	...	...	...
1.13	...	...	...
1.14	...	...	...
1.15	...	...	...
1.16	...	...	...
1.17	...	...	...
1.18	...	...	...
1.19	...	...	...
1.20	...	...	...
1.21	...	...	...
1.22	...	...	...
1.23	...	...	...
1.24	...	...	...
1.25	...	...	...
1.26	...	...	...
1.27	...	...	...
1.28	...	...	...
1.29	...	...	...
1.30	...	...	...
1.31	...	...	...
1.32	...	...	...
1.33	...	...	...
1.34	...	...	...
1.35	...	...	...
1.36	...	...	...
1.37	...	...	...
1.38	...	...	...
1.39	...	...	...
1.40	...	...	...
1.41	...	...	...
1.42	...	...	...
1.43	...	...	...
1.44	...	...	...
1.45	...	...	...
1.46	...	...	...
1.47	...	...	...
1.48	...	...	...
1.49	...	...	...
1.50	...	...	...
1.51	...	...	...
1.52	...	...	...
1.53	...	...	...
1.54	...	...	...
1.55	...	...	...
1.56	...	...	...
1.57	...	...	...
1.58	...	...	...
1.59	...	...	...
1.60	...	...	...
1.61	...	...	...
1.62	...	...	...
1.63	...	...	...
1.64	...	...	...
1.65	...	...	...
1.66	...	...	...
1.67	...	...	...
1.68	...	...	...
1.69	...	...	...
1.70	...	...	...
1.71	...	...	...
1.72	...	...	...
1.73	...	...	...
1.74	...	...	...
1.75	...	...	...
1.76	...	...	...
1.77	...	...	...
1.78	...	...	...
1.79	...	...	...
1.80	...	...	...
1.81	...	...	...
1.82	...	...	...
1.83	...	...	...
1.84	...	...	...
1.85	...	...	...
1.86	...	...	...
1.87	...	...	...
1.88	...	...	...
1.89	...	...	...
1.90	...	...	...
1.91	...	...	...
1.92	...	...	...
1.93	...	...	...
1.94	...	...	...
1.95	...	...	...
1.96	...	...	...
1.97	...	...	...
1.98	...	...	...
1.99	...	...	...
2.00	...	...	...

| 2.1 | Food-101 | JPEG images, folder-per-class | 101,000 images across 101 classes |

| 2.2 | WineSensed (NeurIPS 2023) | JSONL (one review per line) | ~824,000 rows, 357 MB |

— — —

2.1 — Load Food-101

Food-101 is loaded via `torchvision.datasets.Food101``. On the first run it downloads and extracts the dataset to ``data/food-101/`` (~5 GB). Subsequent runs load from disk instantly from the cache.

****Dataset structure:**** 1,000 images per class — 750 in the official train split and 250 in the test split — for a total of 75,750 training and 25,250 test images. The class distribution is perfectly balanced by design, which means no class weighting is needed for the image model.

****Validation checks performed:****

- Train split has exactly 75,750 images, test split has 25,250
- Number of unique class labels equals 101
- Spot-check: first 10 integer labels are in the valid range [0, 100]

> The official Food-101 train/test split (75k/25k) does ****not**** match the required 70/15/15 ratio. We will pool both splits in Section 4 and create a fresh stratified split.

Cell 13: Code

2.1 Load Food-101 via torchvision

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"  
  exec('\n'.join(lines[:-1]), glb)
```

```
~~~~~  
~~~~~
```

```
File "<string>", line 4, in <module>
```

NameError: name 'DATA_DIR' is not defined

Cell 14: ■ Markdown

2.2 — Load WineSensed reviews (raw)

WineSensed (NeurIPS 2023) contains 824,000 real Vivino user tasting notes with structured metadata attached to every review. It is downloaded from Hugging Face Hub (~357 MB) and cached to `data` — subsequent runs load from disk.

****How this dataset is used across the project:****

Section	Model	What we need from WineSensed
---------	-------	------------------------------

---	---	---
-----	-----	-----

11	BiLSTM	`review_text` + `grape` → 15-class grape classification
----	--------	---

12	TasteBiLSTM	`review_text` + taste labels (derived in Sec 12) → 8-class taste encoder
----	-------------	--

7	Word2Vec	`review_text` corpus → fine-tune Google News embeddings on wine vocabulary
---	----------	--

14	Recommendation	`wine_label`, `rating_pct`, `review_text` → recommendation cards
----	----------------	--

****Raw columns loaded and their roles:****

Column	Content	Role
--------	---------	------

---	---	---
-----	-----	-----

`review`	Vivino tasting note	Renamed to `review_text`; model input text and card quote
----------	---------------------	---

`wine`	Wine name (e.g. *Château Margaux*)	Combined with `year` → `wine_label` for the recommendation card
--------	------------------------------------	---

`year`	Vintage year	Combined with `wine` → `wine_label`
--------	--------------	-------------------------------------

`grape`	Primary grape variety (comma-separated, first entry used)	Classification label for BiLSTM; TasteBiLSTM uses its own labels
---------	---	--

`rating`	Vivino avg rating 0–5	Converted to `rating_pct` (0–100%) here; shown on recommendation card
----------	-----------------------	---

`country` / `region`	Wine origin	Context only
----------------------	-------------	--------------

****Expected raw data issues (cleaned in Section 3):****

- Missing values in `review`, `grape`, `rating`

- Non-English reviews (Vivino is a global platform)

- **Licence:** CC BY-NC-ND 4.0 — non-commercial research use only.**

Cell 15: ■ Code

```
from huggingface_hub import hf_hub_download, login
```

■■ Resolve HF token

```
if IN_COLAB:
    try:
        from google.colab import userdata
        _hf_token = userdata.get("HF_TOKEN")
    if _hf_token:
        print("✓ HF_TOKEN loaded from Colab Secrets.")
    else:
```



```
.astype("Int64")
)
```

Output:

- No HF token – proceeding unauthenticated (vault warning expected, still works).

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 45, in <module>
NameError: name 'DATA_DIR' is not defined
```

Cell 16: ■ Code


```
print(f"\n✓ Section 2.2 complete — {len(df_wine):,} raw rows in df_wine.")
print(" Missing values, short reviews, and non-English text removed in Section 3.")
```

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
```

```
File "<string>", line 5, in <module>
```

NameError: name 'df_wine' is not defined

Cell 17: ■ Markdown

Section 3 — Data Cleaning

Best practice: clean data *before* EDA and *before* model training.

If you clean after plotting, your charts describe data the model never sees — that is misleading.

Every statistic in Section 5 (review length, grape distribution, word clouds) reflects

the data the models will actually train on.

Food-101 image data needs no cleaning — torchvision downloads a curated, balanced dataset with no missing labels. Only the WineSensed text data is cleaned here.

Why each cleaning step matters

| Step | What is removed | Why |

|---|---|---|

| 1 — Missing essentials | Rows with no review, grape, wine name, or rating | All four are required — missing any one makes the row unusable for training or the recommendation card |

| 2 — Zero rating | `rating\_pct == 0` | Vivino encodes "not yet rated" as 0.0; it is not a genuine score and would show as 0% on the card |

| 3 — Short reviews | Fewer than 5 words | Too little text for BiLSTM or TasteBiLSTM to learn from; also pollutes the Word2Vec vocabulary |

| 4 — Non-English | Less than 80% ASCII characters | Word2Vec and both BiLSTM models are trained on English; non-Latin-script reviews produce near-zero vectors |

| 5 — Duplicates | Identical `review\_text` entries | Vivino reattaches the same note to multiple vintages; duplicates bias class distributions for both BiLSTM and TasteBiLSTM |

3.1 — *Clean WineSensed*

Cell 18: ■ Code


```
print(f"Step 1 – dropped {_step['1 missing essentials'],} rows (missing  
essentials) | remaining: {len(df_wine),}")
```

```
NameError: name 'df_wine' is not defined
```

```
print(f"Step 2 - dropped {_step['2 zero rating (0%)']:,} rows (zero rating) |  
remaining: {len(df_wine):,}")
```

```
NameError: name 'df_wine' is not defined
```

```
_step["3 review < 5 words"] = _n - len(df_wine)
```

```
print(f"Step 3 – dropped {_step['3 review & 5 words']:,} rows (review & 5 words) | remaining: {len(df_wine):,}")
```

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^
File "<string>", line 4, in <module>
NameError: name 'df_wine' is not defined
```

Cell 22: ■ Code

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "<string>", line 8, in <module>
NameError: name 'df_wine' is not defined
```

Cell 23: ■ Code



Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 4, in <module>
NameError: name 'step' is not defined
```

Cell 25: ■ Markdown

3.2 — Select top 15 grape varieties as classification labels

Rather than grouping wines into broad types (Red / White / Rosé), we use the **primary grape variety** as the BiLSTM classification target. This forces the model to learn the fine-grained flavor language specific to each grape — "cassis and cedar" for Cabernet Sauvignon vs "strawberry and silk" for Pinot Noir.

We select the **top 15 grapes by review count** from the cleaned dataset. These typically cover ~85% of all reviews. Rows whose primary grape falls outside the top 15 are dropped.

The resulting `df_wine_mapped` is shared by all three text-based model components:

Component	Section	Uses `df_wine_mapped` for
---	---	---
BiLSTM + DistilBERT	11	Grape classification training + evaluation
TasteBiLSTM	12	Taste encoder training (own labels added in Sec 12)
Recommendation engine	14	Review retrieval and wine card generation

****Why this runs after cleaning:**** grape frequency counts on raw data include rows that will be dropped (duplicates, non-English, etc.) — that would distort which 15 grapes are selected.

Cell 26: ■ Code


```
coverage = len(df_wine_mapped) / len(df_wine) * 100
print(f"Rows kept : {len(df_wine_mapped):>10,} ({coverage:.1f}% of reviews)")
print(f"Rows dropped : {len(df_wine) - len(df_wine_mapped):>10,} "
      f"(primary grape not in top {TOP_N_GRAPES})")
```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 9
    print(f"Rows dropped : {len(df_wine) - len(df_wine_mapped):>10,} "
          ^
SyntaxError: '(' was never closed
During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec(source, glb)
    ~~~^^^^^^^^^^^^^^^^
  File "<string>", line 3, in <module>
NameError: name 'df_wine' is not defined

```

Cell 29: ■ Code

Error:


```

print("=" * 52)
print(f"{'Split':&lt;8} {'Classes':&lt;8} {'Min imgs':&lt;10} {'Max imgs':&lt;10} {'Mean':&lt;7}")
print("-" * 52)
print(f"{'train':&lt;8} {len(train_counts):&lt;8} "
f"{min(train_vals):&lt;10} {max(train_vals):&lt;10} "
f"{sum(train_vals)/len(train_vals):&lt;7.1f}")
print(f"{'test':&lt;8} {len(test_counts):&lt;8} "
f"{min(test_vals):&lt;10} {max(test_vals):&lt;10} "
f"{sum(test_vals)/len(test_vals):&lt;7.1f}")
print()

for split_name, vals in [("train", train_vals), ("test", test_vals)]:
    if min(vals) == max(vals):
        print(f"✓ {split_name.capitalize()} – perfectly balanced "
f"({min(vals)} images per class across all {len(vals)} classes).")
    else:
        ratio = max(vals) / min(vals)
        print(f"■ {split_name.capitalize()} – imbalanced "
f"(ratio {ratio:.2f}x, min={min(vals)}, max={max(vals)}).")

missing_train = set(range(101)) - set(train_counts.keys())
missing_test = set(range(101)) - set(test_counts.keys())
if not missing_train and not missing_test:
    print("✓ All 101 classes present in both splits.")
else:
    for split_name, missing in [("train", missing_train), ("test", missing_test)]:
        if missing:
            print(f"■ Missing from {split_name}: "
f"[{ds_train.classes[i] for i in sorted(missing)}]")

print("\n✓ Section 3.3 complete.")

```

Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
exec('\n'.join(lines[:-1]), glb)
~~~~~AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

```

```
File "<string>", line 5, in <module>
NameError: name 'ds_train' is not defined
```

Cell 32: ■ Markdown

3.4 — Food flavor table

To pair food with wine in Section 14, we need a taste-space representation of each Food-101 class. We achieve this by storing plain-English descriptions of the flavor character of every food class in `food_flavor_description_v2.json` — generated by **Claude Sonnet 4.6**.

How it fits the pipeline:

...

food_name → description sentence

↓

TasteBiLSTM encoder (Section 12)

↓

256-d taste vector

↓

cosine similarity vs 12 cluster centroids (Section 14)

↓

Best-matching wine cluster → Top-3 recommendations

...

The descriptions use food taste vocabulary that maps naturally onto the 10 taste axes the TasteBiLSTM learned from wine reviews: ``body``, ``acidity``, ``tannin``, ``red_fruit``, ``dark_fruit``, ``earthy``, ``sweet``, ``oaky``, ``floral``, ``mineral``.

Table format — each of the 101 entries has three keys:

| Key | Card label (Section 16) | Purpose |

|---|---|---|

| ``classic`` | Safe Bet | Closest taste match — the dish's dominant flavour identity in full |

| ``contrast`` | Bold Move | Deliberate clash — a taste that creates intentional tension |

| ``safe_bet`` | Hidden Gem | Compatible but surprising — pairs unexpectedly well |

Two-cell pattern:

| Cell | When to run | What it does |

|---|---|---|

****3.4a — Build**** | Once (first time only) | Defines the dict, saves `food_flavor_description_v2.json`` to `data/`` and to Drive; is a no-op if the file already exists |

| **3.4b — Load** | Every run | Finds the file (Drive → local fallback), loads it, validates 101 entries and 3 keys each, exposes `food\_flavor\_table\_v2` |

> Descriptions generated by **Claude Sonnet 4.6** (May 2026).

Cell 33: ■ Code

```
print(f"Entries: {len(food_flavor_table_v2)}/101")
print(f"Key errors: {len(errors)}")

assert len(food_flavor_table_v2) == 101, \
f"Expected 101 entries, got {len(food_flavor_table_v2)}"
assert len(errors) == 0, \
f"Entries with missing/extra keys: {errors}"

print("✓ Section 3.4 complete – food_flavor_table_v2 validated: 101 entries, all 3
keys present.")
```

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^
File "<string>", line 4, in <module>
NameError: name 'BASE_DIR' is not defined
```

Cell 34: ■ Markdown

Section 4 — Train / Val / Test Split (Both Datasets)

****Both datasets are split here, before any EDA.**** Performing EDA on the full dataset would leak test-set information into architecture decisions — `MAX\_SEQ\_LEN` would be set using test-review lengths, class-distribution charts would show test-class proportions, and word clouds would include test-set vocabulary. All of these constitute subtle but real data leakage.

> ■ ****Test sets are frozen immediately after this section and must not be inspected until the final evaluation cell of each model.****

Sub-section	Dataset	Method	Outputs
---	---	---	---
4.1 WineSensed (text) Stratified `train_test_split` on `df_wine_mapped` `df_train`, `df_val`, `df_test`, `GRAPE_TO_IDX`			
4.2 Food-101 (images) Pool all 101k images, stratified `train_test_split` on indices `_idx_train`, `_idx_val`, `_idx_test`			

Cell 35: ■ Markdown

4.1 — WineSensed (text reviews)

| ****train**** (70%) | `df_train`` | ~70% of rows | Vocabulary building; Word2Vec fine-tuning (§7); BiLSTM training (§11); TasteBiLSTM training (§12); K-means fitting (§14) |

val (15%)	<code>`df_val`</code>	~15% of rows	Early stopping, hyperparameter tuning
----------------------	-----------------------	--------------	---------------------------------------

test (15%) <code>df_test</code> ~15% of rows Final evaluation only — touched once per model

Stratified by ``grape_class`` — every split preserves the same class proportions as the full dataset. A per-grape proportion table is printed to verify stratification.

Cell 36: ■ Code

```
_df = df_wine_mapped.copy()
_labels = _df["grape_class"].map(GRAPE_TO_IDX)
```

```
df_train, _df_tmp = train_test_split(
    _df, test_size=0.30, stratify=_labels, random_state=SEED
)

df_val, df_test = train_test_split(
    _df_tmp, test_size=0.50,
    stratify=_df_tmp["grape_class"].map(GRAPE_TO_IDX),
    random_state=SEED
)
```

```
df_train = df_train.reset_index(drop=True)
df_val = df_val.reset_index(drop=True)
df_test = df_test.reset_index(drop=True)
```

```
# ■ Split sizes
```

```

print(f"'Split':<8} {'Rows':>8} {'%':>5}")
print("-" * 25)
for name, df in [("train", df_train), ("val", df_val), ("test", df_test)]:
    print(f"{name:<8} {len(df):>8,} {len(df)/len(df_wine_mapped)*100:>4.1f}%")
print(f"'total':<8} {len(df_train)+len(df_val)+len(df_test):>8,}")

# ■■ Stratification check – class proportions should be ~70 / 15 / 15 ■■■■■■■■■■
print("\nStratification check – each grape should be ~70 / 15 / 15:")
print(f"'Grape':<25} {'train%':>7} {'val%':>6} {'test%':>6}")
print("-" * 50)
for grape in GRAPE_CLASSES:
    tr = (df_train["grape_class"] == grape).sum()
    v = (df_val["grape_class"] == grape).sum()
    te = (df_test["grape_class"] == grape).sum()
    n = tr + v + te
    print(f"{grape:<25} {tr/n*100:>6.1f}% {v/n*100:>5.1f}% {te/n*100:>5.1f}%")

print(f"\n■ df_test frozen – do not inspect until final evaluation "
f"(Sec 11: BiLSTM, Sec 12: TasteBiLSTM, Sec 14: Recommendation).")
print(f" EDA in Section 5 uses df_train only.")
print(f"\n✓ Section 4.1 complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 9, in <module>
NameError: name 'GRAPE_CLASSES' is not defined

```

Cell 37: ■ Markdown

4.2 — Food-101 (images)

Food-101's built-in 75k / 25k split does not match the required 70 / 15 / 15 ratios. We **pool** all 101,000 images and create a fresh stratified 3-way split using the same fixed `SEED`.

Split	Index array	Samples	%	Per-class
---	---	---	---	---
train	_idx_train	70,700	70%	~700

test	`_idx_test`	15,150	15%	~150
-----------------	-------------	--------	-----	------

Cell 38: ■ Code

Generated using nb2pdf VSCode Extension [Get Extension](#)


```

_total = len(_all_labels)
print(f"'Split':&lt;8} {'Samples':&gt;8} {'%':&gt;6} {'Per-class (avg)':&gt;16}")
print("-" * 46)
for _name, _idx in [("train", _idx_train), ("val", _idx_val), ("test", _idx_test)]:
    _n = len(_idx)
    print(f"{_name:&lt;8} {_n:&gt;8}, {_n/_total*100:&gt;5.1f}% {_n/101:&gt;15.0f}")
    print(f"'total':&lt;8} {_total:&gt;8}, 100.0%")

# ■■ Verify stratification – per-class sample counts should be uniform ■■■■■■■■■■
_train_lbl_counts = Counter(_all_labels[i] for i in _idx_train)
_val_lbl_counts = Counter(_all_labels[i] for i in _idx_val)
_test_lbl_counts = Counter(_all_labels[i] for i in _idx_test)

_train_per_class = sorted(_train_lbl_counts.values())
_val_per_class = sorted(_val_lbl_counts.values())
_test_per_class = sorted(_test_lbl_counts.values())

print(f"\nPer-class sample range:")
print(f" train : min={min(_train_per_class)} max={max(_train_per_class)} (expected ~700)")
print(f" val : min={min(_val_per_class)} max={max(_val_per_class)} (expected ~150)")
print(f" test : min={min(_test_per_class)} max={max(_test_per_class)} (expected ~150)")

print(f"\n■ _idx_test frozen – do not inspect until CNN evaluation (Sec 10).")
print(f"\n✓ Section 4.2 complete – index arrays ready: _idx_train
({len(_idx_train):,}) "
f"_idx_val ({len(_idx_val):,}) _idx_test ({len(_idx_test):,})")

```

Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^

```

File "<string>", line 52

```

    print(f"\n✓ Section 4.2 complete – index arrays ready: _idx_train ({len(_idx_train):,}) "
          ^

```

SyntaxError: '(' was never closed

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
exec(source, glb)
~~~~~
File "<string>", line 9, in <module>
NameError: name 'FOOD101_ROOT' is not defined
```

Cell 39: ■ Markdown

4.3 — Test Set Freeze

****Both test sets are now frozen.**** Do not inspect, plot, or evaluate on them until the relevant final evaluation cells.

Variable	Dataset	First use
---	---	---
<code>`df_test`</code>	WineSensed	Sec 11 (BiLSTM), Sec 12 (TasteBiLSTM), Sec 14 (Recommendation)
<code>`_idx_test`</code>	Food-101	Sec 10 (CNN evaluation)

Cell 40: ■ Markdown

Section 5 — Exploratory Data Analysis

EDA runs on ****clean, split data only**** (after Sections 3–4). Every chart here describes exactly what the models will train on. Using pre-split training data prevents test-set statistics from influencing architecture choices.

Sub-section	Dataset	Purpose
---	---	---
5.1	Food-101 (images)	Sample grid, class distribution, image dimensions, channel statistics
5.2	WineSensed (text)	Review length → <code>`MAX_SEQ_LEN`</code> , grape distribution, word clouds, rating distribution
<code>`MAX_SEQ_LEN`</code> is set at the end of 5.2 and reused in Sections 7 (W2V tokenisation), 11 (BiLSTM), and 12 (TasteBiLSTM).		

Cell 41: ■ Markdown

5.1 — Food-101 Image EDA

Food-101 is a fixed, curated benchmark with 101 food categories and 1,000 images per class.

The goals here are to confirm class balance, characterise native image dimensions, and verify that ImageNet normalisation statistics are appropriate for this dataset before applying them in Section 6.

Cell 42: ■ Code

```
def _food101_img_path(idx):
    """Return image file path for a pooled index (Section 4.2 convention)."""
    if idx < _n_food_tr:
        return _base_train_meta._image_files[idx]
    return _base_test_meta._image_files[idx - _n_food_tr]
```

```
print(f"EDA uses _idx_train: {len(_idx_train):,} images "  
f"(70% of pooled {len(_idx_train)+len(_idx_val)+len(_idx_test):,} Food-101  
images)")
```

```
# Build class → training-index lookup once (reused in grid + class-dist cell)
_cls_to_train_idx = {}
for _i in _idx_train:
    _cls_to_train_idx.setdefault(_all_labels[_i], []).append(_i)
```

```
for ax, cls_idx in zip(axes.flat, _grid_cls_idxs):
    pool_i = random.choice(_cls_to_train_idx[cls_idx])
    img = Image.open(_food101_img_path(pool_i)).convert("RGB")
    ax.imshow(img)
    ax.set_title(_base_train_meta.classes[cls_idx].replace("_", " "), fontsize=8)
    ax.axis("off")
```

```
plt.tight_layout()
plt.savefig(FIGURES / "eda_image_grid.png", dpi=100)
plt.show()
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 13, in <module>
NameError: name '_idx_train' is not defined
```

Cell 43: ■ Markdown

5.1.1 — Class Distribution

Food-101 is a perfectly balanced benchmark: 1,000 images per class. After our 70% stratified split this gives ~700 images per class (~70,700 total training images). The bar chart below confirms the split preserved balance — no class weighting is needed for the image model.

Cell 44: ■ Code

```
print(f"Class counts - min: {min(counts_sorted)} "
      f"max: {max(counts_sorted)} "
      f"mean: {sum(counts_sorted)/len(counts_sorted):.1f} "
      f"all equal: {min(counts_sorted) == max(counts_sorted)}")
print("→ Balanced dataset: no class weighting needed for the image model.")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 3, in <module>
NameError: name '_cls_to_train_idx' is not defined
```

Cell 45: ■ Markdown

5.1.2 — Image Dimensions & Aspect Ratio

Food-101 images are variable-sized JPEGs scraped from the web.

The histograms below show native sizes and aspect ratios ****before**** any resizing.

The aspect ratio distribution confirms most images are near-square ($W/H \approx 1$),

which justifies the `CenterCrop(224)` strategy in Section 6 — minimal content loss versus padding.

All images will be resized to **224 × 224** in Section 6.

Cell 46: ■ Code

```

_widths_s = sorted(_widths)
_heights_s = sorted(_heights)
_asp_s = sorted(_aspects)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].hist(_widths, bins=40, color="steelblue", edgecolor="none")
axes[0].set_title(f"Image widths (sample of {_dim_n})")
axes[0].set_xlabel("Width (pixels)")
axes[0].set_ylabel("Count")

axes[1].hist(_heights, bins=40, color="steelblue", edgecolor="none")
axes[1].set_title(f"Image heights (sample of {_dim_n})")
axes[1].set_xlabel("Height (pixels)")
axes[1].set_ylabel("Count")

axes[2].hist(_aspects, bins=40, color="teal", edgecolor="none")
axes[2].axvline(1.0, color="red", linestyle="--", linewidth=1.5, label="Square (1 : 1)")
axes[2].set_title(f"Aspect ratio W/H (sample of {_dim_n})")
axes[2].set_xlabel("Aspect ratio (W / H)")
axes[2].set_ylabel("Count")
axes[2].legend()

plt.suptitle("Food-101 – raw image dimensions & aspect ratios (our training split)", fontsize=12)
plt.tight_layout()
plt.savefig(FIGURES / "eda_image_dims.png", dpi=100)
plt.show()

print(f"Width – min: {_widths_s[0]:<6} median: {_widths_s[_dim_n // 2]:<6} max: {_widths_s[-1]}")
print(f"Height – min: {_heights_s[0]:<6} median: {_heights_s[_dim_n // 2]:<6} max: {_heights_s[-1]}")
print(f"Aspect – min: {_asp_s[0]:.3f} median: {_asp_s[_dim_n // 2]:.3f} max: {_asp_s[-1]:.3f}")
print(f"→ Most images are near-square – CenterCrop(224) is appropriate (Section 6).")

```

Error:

Traceback (most recent call last):


```

ax1.bar(_x + _w / 2, _imagenet_mean, _w, label="ImageNet", color="orange",
alpha=0.85)

ax1.set_xticks(_x); ax1.set_xticklabels(_channels)

ax1.set_title("Per-channel mean")

ax1.set_ylabel("Mean pixel value (0-1)")

ax1.legend()

ax2.bar(_x - _w / 2, _sig, _w, label="Food-101", color="steelblue")

ax2.bar(_x + _w / 2, _imagenet_std, _w, label="ImageNet", color="orange",
alpha=0.85)

ax2.set_xticks(_x); ax2.set_xticklabels(_channels)

ax2.set_title("Per-channel std")

ax2.set_ylabel("Std dev (0-1)")

ax2.legend()

plt.suptitle(f"Channel statistics – Food-101 vs. ImageNet (sample of {_ch_n}
training images)", fontsize=12)

plt.tight_layout()

plt.savefig(FIGURES / "eda_channel_stats.png", dpi=100)

plt.show()

print(f"Food-101 mean : R={_mu[0]:.4f} G={_mu[1]:.4f} B={_mu[2]:.4f}")
print(f"Food-101 std : R={_sig[0]:.4f} G={_sig[1]:.4f} B={_sig[2]:.4f}")
print(f"ImageNet mean: R=0.4850 G=0.4560 B=0.4060")
print(f"ImageNet std : R=0.2290 G=0.2240 B=0.2250")
print(f"\n→ Dataset stats are close to ImageNet norms – ImageNet normalisation is
appropriate.")
print(f"\n✓ Section 5.1 complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 3, in <module>
NameError: name '_idx_train' is not defined

```

Cell 49: ■ Markdown

5.2 — WineSensed Text EDA

All statistics below are computed on `df_train` only. The key output is `MAX_SEQ_LEN` — the sequence length cap derived from the 95th-percentile review length — reused for tokenisation in Section 7 (W2V), padding in Section 11 (BiLSTM), and padding in Section 12 (TasteBiLSTM).

Cell 50: ■ Code

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'df_train' is not defined
```

Cell 51: ■ Markdown

5.2.1 — Review Length Distribution

`MAX_SEQ_LEN` must be set before building the vocabulary and DataLoaders (Section 7), and it determines the padding length in the BiLSTM (Section 11) and TasteBiLSTM (Section 12) encoders. We choose the 95th percentile so only 5% of training reviews are truncated.

Cell 52: ■ Code

****Key observation:**** WineSensed is significantly imbalanced. Cabernet Sauvignon and Chardonnay are the most-reviewed grapes by far — they dominate the Vivino platform because they are the world's most widely produced varieties. At the other end, Viognier and Chenin Blanc have an order of magnitude fewer reviews. An imbalance ratio $> 3\times$ is typical for this dataset.

****Why this matters for model training:****

- A naive CrossEntropyLoss would bias the classifier toward the majority classes, causing it to predict "Cabernet Sauvignon" or "Chardonnay" for almost every input.
- We correct this with **weighted CrossEntropyLoss** (Section 11): each class weight = $\text{total_samples} / (\text{n_classes} \times \text{class_count})$, so minority classes receive proportionally higher gradient signal.
- The same imbalance also motivates the **TasteBiLSTM** approach in Section 12: rather than classifying grape variety (which requires resolving the imbalanced label space), the taste encoder maps reviews into a continuous 10-dimensional taste space where balance is irrelevant.

An imbalance ratio $> 3\times$ will trigger ****weighted CrossEntropyLoss**** in the training sections.

Cell 54: ■ Code

```
print(" ✓ Imbalance acceptable.")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 19
    else:
        ^
IndentationError: expected an indented block after 'else' statement on line 19
During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec(source, glb)
    ~~~^^^^^^^^^^^^^^^^
  File "<string>", line 3, in <module>
NameError: name 'df_train' is not defined
```

Cell 55: ■ Markdown

5.2.3 — Top-N Terms per Class (Word Clouds)

One word cloud per grape variety, generated from the ****training set only****.

Common English stop-words plus generic wine terms (*wine*, *bottle*, *glass*, *finish*) are removed so that the clouds reveal grape-specific flavour vocabulary rather than shared jargon.

Cell 56: ■ Code

```
# 5.2.3 Word clouds per grape variety (all 15 classes)
from nltk.corpus import stopwords

_stopwords = set(stopwords.words("english")) | {
    "wine", "drink", "bottle", "glass", "finish", "palate", "nose", "taste"
}

fig, axes = plt.subplots(3, 5, figsize=(20, 12))
fig.suptitle("WineSensed – word clouds per grape variety (training set)",
            fontsize=14)

for ax, grape in zip(axes.flat, GRAPE_CLASSES):
    text = " ".join(df_train[df_train["grape_class"] == grape]["review_text"])
```



```

medianprops=dict(color="red", linewidth=2),
flierprops=dict(marker=".", markersize=2, alpha=0.3, color="grey"),
)
for patch in bp["boxes"]:
    patch.set(facecolor="#aec6e8", edgecolor="steelblue")
for w in bp["whiskers"] + bp["caps"]:
    w.set(color="steelblue")

ax.set_title("WineSensed – approval rating (%) by grape variety (training set)")
ax.set_xlabel("Grape variety (sorted by median rating)")
ax.set_ylabel("Approval rating (%)")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.savefig(FIGURES / "eda_rating_per_grape.png", dpi=100)
plt.show()

print("Median approval rating per grape (sorted):")
for _g, _m in _medians.sort_values(ascending=False).items():
    print(f" {_g:<30} {_m:.1f}%")
    print(f"\nRange: {_medians.min():.1f}% - {_medians.max():.1f}% "
          f"(spread = {_medians.max() - _medians.min():.1f} pp)")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 39
    print(f"\nRange: {_medians.min():.1f}% - {_medians.max():.1f}% "
          ^
SyntaxError: '(' was never closed
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec(source, glb)
    ~~~~^
  File "<string>", line 4, in <module>
NameError: name 'df_train' is not defined

```

Cell 61: ■ Markdown

5.2.6 — Review Length per Grape Class

Are some grape varieties described in longer sentences than others?

Significant variation would suggest that a single global `MAX\_SEQ\_LEN` might under-serve verbose classes — or confirm that one length fits all.

Cell 62: ■ Code


```

ax.set_xlabel("Grape variety (sorted by median length)")
ax.set_ylabel("Review length (words)")
ax.legend()
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.savefig(FIGURES / "eda_length_per_grape.png", dpi=100)
plt.show()

print(f"Median review length per grape – range: "
      f"{_len_medians.min():.0f} – {_len_medians.max():.0f} words "
      f"(spread = {_len_medians.max() - _len_medians.min():.0f} words)")

print(f"MAX_SEQ_LEN = {MAX_SEQ_LEN} – covers the majority of reviews across all
      classes.")
print(f"\n✓ Section 5 complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 2, in <module>
NameError: name 'df_train' is not defined

```

Cell 63: ■ Markdown

Section 6 — Image Preprocessing and DataLoaders

****Requires GPU — run in Google Colab.****

The 70 / 15 / 15 index arrays (`\_idx\_train`, `\_idx\_val`, `\_idx\_test`) were computed in Section 4.2 before EDA. This section applies transforms to those indices and builds the DataLoaders used to train and evaluate the CNN (Sections 8–10).

| Sub-section | What it does |

|---|---|

| ****6.1**** | Resize strategy — 256-then-crop to 224 × 224 |

| ****6.2**** | ImageNet normalisation — constants + single-image sanity check |

| ****6.3**** | Data augmentation (training only) — each transform justified |

| ****6.4**** | Wrap index arrays into `\_Food101Combined` Dataset objects |

| ****6.5**** | Build `DataLoader` objects with batching and GPU pinned memory |

Cell 64: ■ Markdown

6.1 — Resize

All images are resized to **224 × 224 pixels**. The pipeline first upsizes the shorter edge to 256 (`Resize(256)`) and then takes a 224-crop — random during training, centred during validation/test. This two-step approach is standard for ImageNet-pretrained models: it preserves more scene content than a direct 224-resize while keeping the input size constant, and matches the preprocessing used when ResNet-50 was originally trained.

> The EDA (Section 5.1.2) confirmed that Food-101 images are predominantly ≥ 256 px on both sides, so upscaling artefacts are negligible.

Cell 65: ■ Code

Error:


```

_norm_t = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToTensor(),
T.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)])
_demo_img_b, _ = _base_train_meta[5]
_normed = _norm_t(_demo_img_b)
print(f"\nNormalized tensor shape : {tuple(_normed.shape)}")
print(f"Per-channel mean (single image) :
{_normed.mean(dim=[1,2]).numpy().round(3)}")
print(f" (→ approaches 0 in aggregate across full dataset)")
print(f"\n✓ 6.2 – IMAGENET_MEAN and IMAGENET_STD defined.")

```

Output:

Channel	Mean	Std
Red	0.485	0.229
Green	0.456	0.224
Blue	0.406	0.225

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 18, in <module>
NameError: name '_base_train_meta' is not defined

```

Cell 68: ■ Markdown

6.3 — Data Augmentation choices

Augmentation is applied to the **training set only**. Val and test receive only deterministic resize + centre-crop + normalise to ensure reproducible evaluation.

| Transform | Justification |

|---|---|

| `Resize(256)` → `RandomCrop(224)` | Stochastic crop simulates different dish framings and aspect ratios across photographers |

| `RandomHorizontalFlip` | Food photos are horizontally symmetric — a pizza looks identical when flipped |

| `RandomRotation(15°)` | Dishes are photographed at varying angles; $\pm 15^\circ$ covers realistic variation without distorting recognisable textures |

| `ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.05)` | Restaurant lighting and white-balance vary widely; small `hue=0.05` avoids changing food colour identity |

```
| `Normalize(ImageNet mean/std)` | Required for valid transfer from ResNet-50; applied last so statistics match the [0,1] tensor range |
```

Cell 69: ■ Code

```
train_transform = T.Compose([
    T.Resize(256),
    T.RandomCrop(224),
    T.RandomHorizontalFlip(),
    T.RandomRotation(15),
    T.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.05),
    T.ToTensor(),
    T.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    T.RandomErasing(p=0.2),
])
```

```
val_test_transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])
```

```
print(f"train_transform : {len(train_transform.transforms)} steps")
print(f"val_test_transform : {len(val_test_transform.transforms)} steps (no
stochastic augmentation)")

for i, t in enumerate(train_transform.transforms):
    print(f" train [{i}]: {t}")
```

```
_aug_display = T.Compose([
    T.Resize(256),
    T.RandomCrop(224),
```

```

T.RandomHorizontalFlip(),
T.RandomRotation(15),
T.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.05),
])

_demo_img_c, _demo_cls_c = _base_train_meta[100]
fig, axes = plt.subplots(2, 4, figsize=(14, 7))
for i, ax in enumerate(axes.flat):
    _img = T.Compose([T.Resize(256), T.CenterCrop(224)])(_demo_img_c) if i == 0 \
    else _aug_display(_demo_img_c)
    ax.imshow(_img)
    ax.set_title("Original (centred)" if i == 0 else f"Augmented #{i}")
    ax.axis("off")
fig.suptitle(
    f"6.3 – Same image, 7 different augmentations "
    f"({_base_train_meta.classes[_demo_cls_c].replace('_', ' ')}")",
    fontsize=13
)
plt.tight_layout()
plt.savefig(str(FIGURES / "6c_augmentation_demo.png"), dpi=100,
bbox_inches="tight")
plt.show()
print(f"\n✓ 6.3 – transforms defined.")

```

Output:

```

train_transform      : 8 steps
val_test_transform  : 4 steps (no stochastic augmentation)
  train [0]: Resize(size=256, interpolation=bilinear, max_size=None, antialias=True)
  train [1]: RandomCrop(size=(224, 224), padding=None)
  train [2]: RandomHorizontalFlip(p=0.5)
  train [3]: RandomRotation(degrees=[-15.0, 15.0], interpolation=nearest, expand=False, fill=0)
  train [4]: ColorJitter(brightness=(0.7, 1.3), contrast=(0.7, 1.3), saturation=(0.8, 1.2), hue=0)
  train [5]: ToTensor()
  train [6]: Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
  train [7]: RandomErasing(p=0.2, scale=(0.02, 0.33), ratio=(0.3, 3.3), value=0, inplace=False)

```

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
```

File "<string>", line 35, in <module>

NameError: name '_base_train_meta' is not defined

Cell 70: ■ Markdown

6.4 — Wrap index arrays into `Dataset` objects

`\_base\_train\_meta` and `\_base\_test\_meta` (loaded in Section 4.2, no-transform) are reused here. The `\_Food101Combined` Dataset maps each combined index (0..100,999) back to the correct underlying split and applies the appropriate transform on-the-fly. No re-download or re-instantiation is needed.

Cell 71: ■ Code

■■ 6.4 Wrap pre-computed index arrays into Dataset objects

```
# Reuses _base_train_meta / _base_test_meta from Section 4.2 – no re-loading.
```

```
# The 70/15/15 split (_idx_train, _idx_val, _idx_test) was fixed in Section 4.2;
```

```
# we only apply transforms here.
```

```
class _Food101Combined(torch.utils.data.Dataset):
```

```
"""Maps a pooled index (0..100,999) to the correct Food-101 split + transform."""
```

```
def __init__(self, indices, transform):
```

```
self.indices = indices
```

```
self.transform = transform
```

```
def __len__(self):
```

```
return len(self.indices)
```

```
def __getitem__(self, i):
```

```
idx = self.indices[i]
```

```
if idx < _n_food_tr:
```

```
img, label = _base_train_meta[idx]
```

```
else:
```

```
img, label = _base_test_meta[idx - _n_food_tr]
```

```
return self.transform(img), label
```

```
print(f"img_train_ds : {len(img_train_ds):,} samples")
print(f"img_val_ds : {len(img_val_ds):,} samples")
print(f"img_test_ds : {len(img_test_ds):,} samples")
print(f"\n✓ 6.4 – Dataset objects created.")
```



```

print("-" * 40)
for _name, _ds, _ldr in [("train", img_train_ds, img_train_loader),
("val", img_val_ds, img_val_loader),
("test", img_test_ds, img_test_loader)]:
    print(f"{_name:<10} {len(_ds):>8} {len(_ds)/_total*100:>5.1f}%
    {len(_ldr):>8,}")
print(f"'total':<10} {_total:>8,} 100.0%")

imgs, labels = next(iter(img_train_loader))
assert imgs.shape == (IMG_BATCH, 3, 224, 224), f"Unexpected shape: {imgs.shape}"
print(f"\nBatch shape : {imgs.shape} (B, C, H, W)")
print(f"Label range : {labels.min().item()} - {labels.max().item()}")
print(f"\n✓ Section 6 complete – image DataLoaders ready for CNN training (Section
8).")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'img_train_ds' is not defined

```

Cell 74: ■ Markdown

Section 7 — Text Preprocessing and DataLoaders

All preprocessing steps run on the **training split only** to prevent data leakage.

The vocabulary, Word2Vec model, and class weights are built exclusively from `df_train``.

| Sub-section | What |

|---|---|

| **7.1** | Tokenisation strategy (word-level, justified) |

| **7.2** | Vocabulary building and OOV handling |

| **7.3** | Padding and truncation to `MAX_SEQ_LEN`` |

| **7.4** | Word2Vec fine-tuning on wine reviews; embedding matrix (→ §11, §12) |

| **7.5** | Class weights for imbalanced grape labels |

| **7.6** | Text DataLoaders |

| **7.7** | Batch shape verification |

...

- ...

```

lbl_val = df_val["grape_class"].map(GRAPE_TO_IDX).tolist()
txt_test = df_test["review_text"].tolist()
lbl_test = df_test["grape_class"].map(GRAPE_TO_IDX).tolist()

print(f"{'Split':&lt;8} {'Samples':&gt;10}")
print("-" * 20)
print(f"{'train':&lt;8} {len(txt_train):&gt;10,}")
print(f"{'val':&lt;8} {len(txt_val):&gt;10,}")
print(f"{'test':&lt;8} {len(txt_test):&gt;10,}")

# Tokenise: lowercase → strip punctuation → whitespace split
_punct = str.maketrans("", "", string.punctuation)

def tokenise(text):
    return text.lower().translate(_punct).split()

tok_train = [tokenise(t) for t in txt_train]
tok_val = [tokenise(t) for t in txt_val]
tok_test = [tokenise(t) for t in txt_test]

# Show sample tokenisation
print("\n--- Sample tokenisation ---")
for i in range(2):
    print(f"\nOriginal : {txt_train[i][:120]}")
    print(f"Tokens : {tok_train[i][:12]} ...")
    print(f"\nAvg tokens/review – train: {sum(len(t) for t in tok_train)/len(tok_train):.1f} "
          f" val: {sum(len(t) for t in tok_val)/len(tok_val):.1f}")

```

Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^

```

File "<string>", line 33

```

    print(f"\nAvg tokens/review – train: {sum(len(t) for t in tok_train)/len(tok_train):.1f} "
          ^

```

SyntaxError: '(' was never closed

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec(source, glb)

```



```
def encode_and_pad(tokens, max_len):
    ids = [VOCAB.get(w, VOCAB[UNK_TOKEN]) for w in tokens[:max_len]]
    ids += [VOCAB[PAD_TOKEN]] * (max_len - len(ids))
    return ids

X_train = np.array([encode_and_pad(t, MAX_SEQ_LEN) for t in tok_train],
dtype=np.int64)
X_val = np.array([encode_and_pad(t, MAX_SEQ_LEN) for t in tok_val], dtype=np.int64)
X_test = np.array([encode_and_pad(t, MAX_SEQ_LEN) for t in tok_test],
dtype=np.int64)

pct_trunc_train = sum(len(t) > MAX_SEQ_LEN for t in tok_train) / len(tok_train)
* 100
pct_trunc_val = sum(len(t) > MAX_SEQ_LEN for t in tok_val) / len(tok_val) * 100

print(f"MAX_SEQ_LEN = {MAX_SEQ_LEN} (p95 of training review lengths)")
print(f"X_train shape : {X_train.shape} (N x MAX_SEQ_LEN)")
print(f"X_val shape : {X_val.shape}")
print(f"X_test shape : {X_test.shape}")
print(f"Truncated (train) : {pct_trunc_train:.1f}% (reviews longer than
MAX_SEQ_LEN)")
print(f"Truncated (val) : {pct_trunc_val:.1f}%")

# Show a sample encoding
print(f"\n--- Sample encoding (first review) ---")
print(f"Tokens (first 8) : {tok_train[0][:8]}")
print(f"Encoded (first 8) : {X_train[0][:8].tolist()}")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 11, in <module>
NameError: name 'tok_train' is not defined
```

Cell 81: ■ Markdown

7.4 — Word2Vec fine-tuning on wine reviews

`gensim.Word2Vec` is trained from scratch on `tok\_train` — the tokenised training reviews.

Domain training gives the embeddings wine-specific semantics (*tannins* and *grippy* cluster together; *oak* and *vanilla* cluster together) that generic pre-trained vectors lack.

Property	Value
----------	-------

|-|-|

| Embedding dimension | `EMBED\_DIM = 100` |

| Training corpus | `tok\_train` only — no val/test leakage |

| Algorithm | Skip-gram (`sg=1`), negative sampling |

| Window | 5 tokens |

| Epochs | 10 |

The resulting ``embedding_matrix`` (``VOCAB_SIZE × EMBED_DIM``) is passed to ``nn.Embedding.from_pretrained(..., freeze=False)`` in **Section 11 (BiLSTM)** and **Section 12 (TasteBiLSTM)** — vectors are fine-tuned further during model training.

Tokens absent from the W2V vocabulary are left at zero and learned from scratch.

The trained model is saved to `WEIGHTS/w2v\_wine.model` for reproducibility.

Cell 82: ■ Code

```

seed = SEED,
)

# Build embedding matrix aligned to VOCAB indices
embedding_matrix = np.zeros((VOCAB_SIZE, EMBED_DIM), dtype=np.float32)
n_found = 0
for word, idx in VOCAB.items():
    if word in _w2v.wv:
        embedding_matrix[idx] = _w2v.wv[word]
    n_found += 1

_w2v_path = WEIGHTS / "w2v_wine.model"
_w2v.save(str(_w2v_path))

print(f"W2V vocab size : {len(_w2v.wv):,}")
print(f"Embedding matrix : {embedding_matrix.shape} (VOCAB_SIZE × EMBED_DIM)")
print(f"Coverage : {n_found / VOCAB_SIZE * 100:.1f}% of vocab tokens found in W2V")
print(f"Saved → {_w2v_path}")
print("✓ 7.4 – W2V embedding matrix ready (→ BiLSTM §11, TasteBiLSTM §12).")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 4, in <module>
ModuleNotFoundError: No module named 'gensim'

```

Cell 83: ■ Markdown

7.5 — Class weights

Class weights correct for label imbalance across the 15 grape classes in WineSensed. Common varieties (Cabernet Sauvignon, Chardonnay) dominate; rarer ones (Viognier, Chenin Blanc) need upweighting.

- Computed via ``sklearn compute_class_weight("balanced", ...)`` on **training labels only** — no leakage.
- Passed to ``nn.CrossEntropyLoss(weight=CLASS_WEIGHTS.to(DEVICE))`` inside the BiLSTM training loop.
- Tensor kept on CPU here and moved to the training device when used.

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 2, in <module>
NameError: name 'txt_train_loader' is not defined
```

Cell 89: ■ Markdown

Section 8 — CNN: Custom Architecture (Trained from Scratch)

****Requires GPU — run in Google Colab.****

A VGG-style CNN trained from random initialisation on Food-101. This baseline model is compared directly against ResNet-50 transfer learning (Section 9) to quantify the gain from pre-trained weights.

Sub-section	What
-------------	------

— — —

8.1 Model architecture (`ConvBlock` + `CNNScratch`) and forward-pass sanity check

| ****8.2**** | Shared training utilities (`train\_one\_epoch`, `evaluate`, criterion / optimiser / scheduler) |

8.3 Training loop — per-epoch checkpoints and early stopping

| ****8.3.1**** | Resume training (same session, after interruption) |

8.3.2 Restore all state from checkpoint (after kernel restart)

| **8.4** | Learning curves (loss + accuracy) |

****8.5**** | Final test evaluation: accuracy, macro F1, classification report |

| ****8.5.1**** | Test visualisations: per-class accuracy bars, confused pairs, heatmap |

| ****8.6**** | Prediction visualiser: image + top-10 confidence bars |

****Architecture****

Four VGG-style convolutional blocks followed by a three-layer classifier head:

///

Block 1: Conv2d(3→64) ×2 → BN → ReLU → MaxPool2d(2) → Dropout2d(0.1) [224→112]

Block 2: Conv2d(64→128) ×2 → BN → ReLU → MaxPool2d(2) → Dropout2d(0.1) [112→56]

Block 3: Conv2d(128→256) ×2 → BN → ReLU → MaxPool2d(2) → Dropout2d(0.1) [56→28]

Block 4: Conv2d(256→512) ×2 → BN → ReLU → MaxPool2d(2) → Dropout2d(0.1) [28→14]

AdaptiveAvgPool2d(1x1) → Flatten(512)

Head: FC(512→1024) → ReLU → Dropout(0.5)

FC(1024→512) → ReLU → Dropout(0.3)

FC(512→101)

...

Training protocol

Hyperparameter	Value
----------------	-------

||--||--||

| Epochs (max) | 20 |

| Early stopping patience | 3 (on val loss) |

| Optimiser | Adam, lr = 1e-3 |

| LR scheduler | ReduceLROnPlateau (factor=0.5, patience=2) |

| Loss | CrossEntropyLoss |

| Checkpoint | Saved every epoch (incl. history) — safe to resume after Colab disconnect |

Cell 90: ■ Code

```

self.block = nn.Sequential(
    nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
    nn.BatchNorm2d(out_ch),
    nn.ReLU(inplace=True),
    nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False), # 2nd conv same scale
    nn.BatchNorm2d(out_ch),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),
    nn.Dropout2d(0.1), # light spatial dropout after each pool
)

def forward(self, x): return self.block(x)

```

```

class CNNScratch(nn.Module):
    def __init__(self, n_classes=101):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3, 64), # 224 → 112
            ConvBlock(64, 128), # 112 → 56
            ConvBlock(128, 256), # 56 → 28
            ConvBlock(256, 512), # 28 → 14 ← new 4th block
            nn.AdaptiveAvgPool2d(1),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512, 1024),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(1024, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),
            nn.Linear(512, n_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

```

```

def encode(self, x):

```

```

"""Return 512-d feature vector (used by joint model)."""
return self.features(x).squeeze(-1).squeeze(-1)

cnn_scratch = CNNScratch().to(DEVICE)
total_params = sum(p.numel() for p in cnn_scratch.parameters())
train_params = sum(p.numel() for p in cnn_scratch.parameters() if p.requires_grad)
print(f"CNN scratch – total params : {total_params:,}")
print(f" trainable : {train_params:,}")

# Sanity-check forward pass
_dummy = torch.randn(2, 3, 224, 224).to(DEVICE)
assert cnn_scratch(_dummy).shape == (2, 101)
print("Forward pass : OK → shape (2, 101)")
print("✓ 8.1 – CNNScratch model ready.")

```

Output:

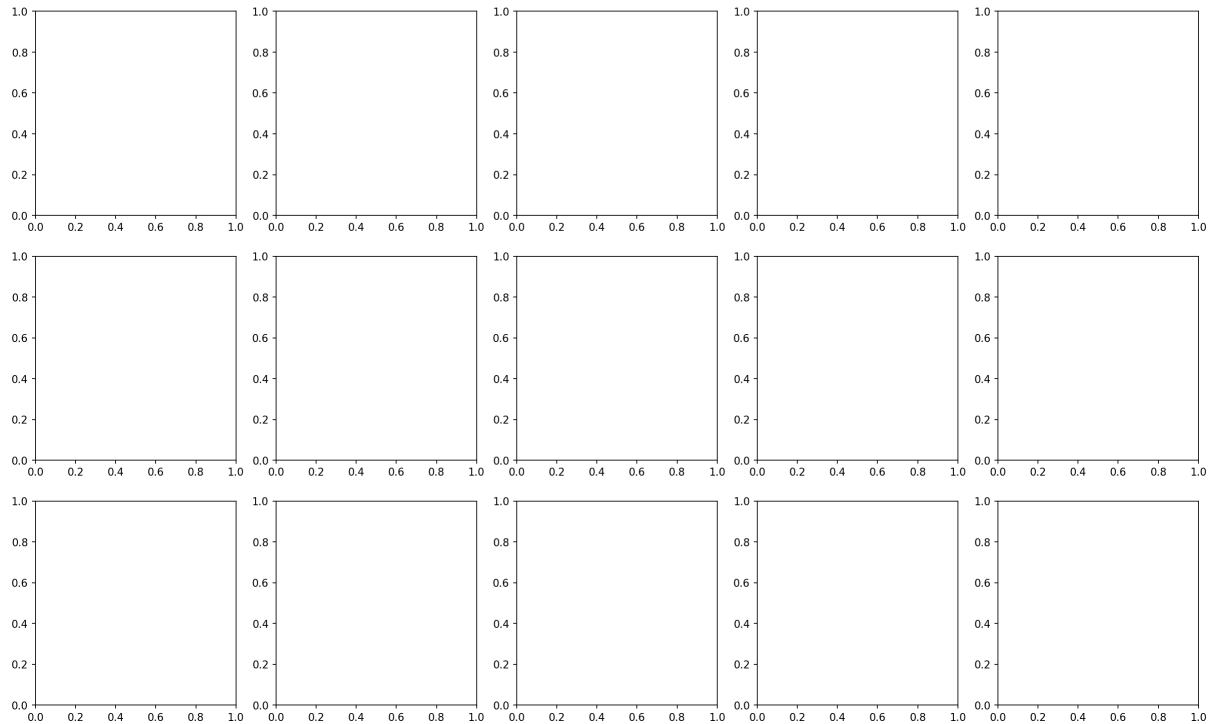
```

CNN scratch – total params : 5,789,221
      trainable      : 5,789,221

Forward pass  : OK → shape (2, 101)
✓ 8.1 – CNNScratch model ready.

```

WineSensed — word clouds per grape variety (training set)



Cell 91: ■ Markdown

8.2 — Training utilities

Two shared helper functions used by both CNN-from-scratch (Section 8) and ResNet-50 (Section 9):

Function	Role
----------	------

$$| \quad - \quad - \quad - \quad | \quad - \quad - \quad - \quad |$$

```
| `train_one_epoch(model, loader, criterion, optimizer)` | One full pass over the training set; returns mean loss and accuracy |
```

| `evaluate(model, loader, criterion)` | No-grad validation/test pass; returns mean loss and accuracy |

`criterion\_cnn = CrossEntropyLoss()` is used for both models. The optimiser and scheduler are model-specific and defined below.

Cell 92: ■ Code

```

total_loss, correct, n = 0.0, 0, 0
for imgs, labels in loader:
    imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
    optimizer.zero_grad()
    logits = model(imgs)
    loss = criterion(logits, labels)
    loss.backward()
    optimizer.step()
    total_loss += loss.item() * len(labels)
    correct += (logits.argmax(1) == labels).sum().item()
    n += len(labels)
return total_loss / n, correct / n

```

```

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, n = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        logits = model(imgs)
        total_loss += criterion(logits, labels).item() * len(labels)
        correct += (logits.argmax(1) == labels).sum().item()
        n += len(labels)
    return total_loss / n, correct / n

```

```

criterion_cnn = nn.CrossEntropyLoss()
optimizer_scratch = Adam(cnn_scratch.parameters(), lr=1e-3)
scheduler_scratch = ReduceLROnPlateau(optimizer_scratch, factor=0.5, patience=2)

```

```

print("✓ 8.2 – training utilities ready.")
print(f" criterion : CrossEntropyLoss")
print(f" optimiser : Adam lr=1e-3")
print(f" scheduler : ReduceLROnPlateau factor=0.5 patience=2")

```

Output:

```

✓ 8.2 – training utilities ready.
 criterion : CrossEntropyLoss

```



```

optimiser    : Adam    lr=1e-3
scheduler    : ReduceLROnPlateau  factor=0.5  patience=2

```

Cell 93: ■ Markdown

8.3 — Training loop with checkpoints and early stopping

- Trains for up to **20 epochs**; stops early if val loss does not improve for **3 consecutive epochs**.
- A full checkpoint (`model_state` + `optimizer_state` + epoch + val loss`) is saved after **every epoch** — training can be safely resumed after a Colab disconnect by reloading the latest checkpoint.
- The best model weights (lowest val loss) are also saved to `cnn_scratch.pt`` and used in 8.5 for the final test evaluation.

Cell 94: ■ Code

```
# ■■■ 8.3 Training loop
```

```
MAX_EPOCHS = 20
PATIENCE = 3
best_val_loss = float("inf")
patience_ctr = 0

history_scratch = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}

print(f"Training started - CNN from scratch | max {MAX_EPOCHS} epochs | device: {DEVICE}")
print("-" * 75)

for epoch in range(1, MAX_EPOCHS + 1):
    tr_loss, tr_acc = train_one_epoch(cnn_scratch, img_train_loader, criterion_cnn, optimizer_scratch)
    vl_loss, vl_acc = evaluate(cnn_scratch, img_val_loader, criterion_cnn)
    scheduler_scratch.step(vl_loss)

    history_scratch["train_loss"].append(tr_loss)
    history_scratch["val_loss"].append(vl_loss)
    history_scratch["train_acc"].append(tr_acc)
    history_scratch["val_acc"].append(vl_acc)

print(f"Epoch {epoch}>2}/{MAX_EPOCHS} "
```

```

f"train_loss={tr_loss:.4f} train_acc={tr_acc:.3f} "
f"val_loss={vl_loss:.4f} val_acc={vl_acc:.3f}")

# Save full checkpoint every epoch (resume-safe after Colab disconnect)
save_checkpoint({
    "epoch": epoch,
    "model_state": cnn_scratch.state_dict(),
    "optimizer_state": optimizer_scratch.state_dict(),
    "val_loss": vl_loss,
    "history": history_scratch,
}, f"cnn_scratch_ckpt_ep{epoch:02d}.pt")

# Keep separate best-model weights
if vl_loss < best_val_loss:
    best_val_loss = vl_loss
    patience_ctr = 0
    save_checkpoint(cnn_scratch.state_dict(), "cnn_scratch_best.pt")
else:
    patience_ctr += 1
    if patience_ctr >= PATIENCE:
        print(f"\nEarly stopping triggered at epoch {epoch} (patience={PATIENCE}).")
        break

print(f"\nTraining complete. Best model saved as: cnn_scratch_best.pt")

```

Output:

```

Training started – CNN from scratch | max 20 epochs | device: cpu
-----

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 12, in <module>
NameError: name 'img_train_loader' is not defined

```

Cell 95: ■ Markdown

8.3.1 — Resume training from checkpoint

If the Colab session is still active and only the training loop cell was interrupted, reload the latest epoch checkpoint and continue from where training stopped.

Cell 96: ■ Code

```
RESUME_EPOCH = 11 # last completed epoch
```

```
# ■■ 3. Restore early-stopping state
best_val_loss = min(history_scratch["val_loss"])
patience_ctr = 0 # last saved epoch was the best
```

```
print(f"\nResuming - CNN from scratch | epochs {RESUME_EPOCH+1}-{MAX_EPOCHS} |  
device: {DEVICE}")  
  
print("-" * 75)
```

```
for epoch in range(RESUME_EPOCH + 1, MAX_EPOCHS + 1):
```

```

tr_loss, tr_acc = train_one_epoch(cnn_scratch, img_train_loader, criterion_cnn,
optimizer_scratch)

vl_loss, vl_acc = evaluate(cnn_scratch, img_val_loader, criterion_cnn)

scheduler_scratch.step(vl_loss)

history_scratch["train_loss"].append(tr_loss)
history_scratch["val_loss"].append(vl_loss)
history_scratch["train_acc"].append(tr_acc)
history_scratch["val_acc"].append(vl_acc)

print(f"Epoch {epoch:>2}/{MAX_EPOCHS} "
      f"train_loss={tr_loss:.4f} train_acc={tr_acc:.3f} "
      f"val_loss={vl_loss:.4f} val_acc={vl_acc:.3f}")

save_checkpoint({
    "epoch": epoch,
    "model_state": cnn_scratch.state_dict(),
    "optimizer_state": optimizer_scratch.state_dict(),
    "val_loss": vl_loss,
    "history": history_scratch,
}, f"cnn_scratch_ckpt_ep{epoch:02d}.pt")

if vl_loss < best_val_loss:
    best_val_loss = vl_loss
    patience_ctr = 0
    save_checkpoint(cnn_scratch.state_dict(), "cnn_scratch_best.pt")
else:
    patience_ctr += 1
    if patience_ctr >= PATIENCE:
        print(f"\nEarly stopping triggered at epoch {epoch} (patience={PATIENCE}).")
        break

print(f"\nTraining complete. Best model saved as: cnn_scratch_best.pt")

```

Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
File "<string>", line 8, in <module>

```

NameError: name 'load_checkpoint' is not defined

Cell 97: ■ Markdown

8.3.2 — Restore training state from checkpoint

After a kernel restart (e.g. Colab disconnect), training results and helper functions are lost from memory but checkpoints are still saved on Google Drive. This cell restores **everything** needed by Sections 8.4–8.6 and Section 9:

- `evaluate()` and `train_one_epoch()` functions
- `criterion_cnn` (CrossEntropyLoss)
- `history_scratch` (training curves)
- `cnn_scratch` best weights

Cell 98: ■ Code

NameError: name 'save_figure' is not defined

Cell 101: ■ Markdown

8.5 — Final test evaluation

The **test set is used exactly once** here. Loads the best checkpoint (`cnn_scratch.pt`) and computes:

Metric	Description
--------	-------------

|-|-|

| Test accuracy | Overall top-1 accuracy across all 101 classes |

| Macro F1 | Unweighted mean F1 across all classes — not biased by class frequency |

| Classification report | Per-class precision, recall, F1, support |

Results will be compared against ResNet-50 (Section 9) in the discussion.

Cell 102: ■ Code

```
from sklearn.metrics import classification_report, f1_score
```

■■ 8.5 Final test evaluation (test set used once)

Response	Percentage
Doing a good job	78%

```
cnn_scratch.load_state_dict(load_checkpoint("cnn_scratch_best.pt"))
```

```
_, test_acc = evaluate(cnn_scratch, img_test_loader, criterion_cnn)
```

```
all_preds, all_labels = [], []
```

```
cnn_scratch.eval()
```

```
with torch.no_grad():
```

```
for imgs, labels in img_test_loader:
```

```
preds = cnn_scratch(imgs.to(DEVICE)).argmax(1).cpu()
```

```
all_preds.extend(preds.tolist())
```

```
all_labels.extend(labels.tolist())
```

```
macro_f1 = f1_score(all_labels, all_preds, average="macro")
```

```
print(f"CNN Scratch – Test accuracy : {test_acc:.4f}")
```

```
print(f" Macro F1 : {macro_f1:.4f}\n")
```

```
print(classification_report(all_labels, all_preds,
```

```
target_names=_base_train_meta.classes,
```

```
zero_division=0))
```



```
print("\n✓ 8.5 – CNN Scratch test evaluation complete.")
```

Error:

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"

```
exec('\n'.join(lines[:-1]), glb)
```

~~~~~

File "&lt;string&gt;", line 4, in &lt;module&gt;

```
NameError: name 'load_checkpoint' is not defined
```

## Cell 103: ■ Markdown

### 8.5.1 — Test evaluation: visualisations

Three charts complementing the classification report in 8.5:

1. **\*\*Top-20 best / worst classes\*\*** — per-class accuracy bars
2. **\*\*Top-15 most confused pairs\*\*** — bar chart of the highest off-diagonal confusion counts
3. **\*\*Focused confusion heatmap\*\*** — only the classes involved in the top-15 confused pairs, row-normalised so each cell is a recall fraction

## Cell 104: ■ Code

## # ■■ 8.5b Test evaluation visualisations – CNN Scratch

```
import numpy as np
```

```
import seaborn as sns
```

```
from sklearn.metrics import confusion_matrix
```

```
_CLASS_NAMES = _base_train_meta.classes # 101 Food-101 class names
```

```
# Build the full confusion matrix once
```

```
sc_cm = confusion_matrix(all_labels, all_preds, labels=list(range(101)))
```

## # ■■ 1. Per-class accuracy bars

```
sc_per_class_acc = sc_cm.diagonal() / sc_cm.sum(axis=1).clip(min=1)
```

```
sc_sorted_idx = np.argsort(sc_per_class_acc)
```

```
sc.worst20_idx = sc.sorted_idx[:20]
```

```
sc_best20_idx = sc_sorted_idx[-20:][::-1]
```

```
fig, (ax_best, ax_worst) = plt.subplots(1, 2, figsize=(18, 7))
```



```

ax2.barh(sc_pair_labels[:::-1], sc_counts[:::-1], color="#ff7f0e")
ax2.set_xlabel("Number of test images misclassified")
ax2.set_title(f"Top-{TOP_PAIRS} most confused class pairs – CNN Scratch\n(true
class → predicted class)")
for i, v in enumerate(sc_counts[:::-1]):
    ax2.text(v + 0.2, i, str(int(v)), va="center", fontsize=9)
plt.tight_layout()
save_figure(fig2, "cnn_scratch_confused_pairs.png")
plt.show()

```

```

# ■■■ 3. Focused confusion heatmap – classes from top confused pairs only ■■■■■■■■
# Collect unique class indices that appear in the top pairs
sc_focus_idx = sorted(set(sc_true_idx.tolist() + sc_pred_idx.tolist()))
sc_focus_labels = [_CLASS_NAMES[i].replace("_", " ") for i in sc_focus_idx]

```

```

# Slice the full CM to keep only those rows/columns, then row-normalise
sc_cm_focus = sc_cm[np.ix_(sc_focus_idx, sc_focus_idx)]
sc_cm_focus_norm = (sc_cm_focus.astype(float)
/ sc_cm_focus.sum(axis=1, keepdims=True).clip(min=1))

```

```

n_focus = len(sc_focus_idx)
fig3, ax3 = plt.subplots(figsize=(max(10, n_focus * 0.7), max(8, n_focus * 0.6)))
sns.heatmap(
    sc_cm_focus_norm,
    ax=ax3,
    annot=True, fmt=".2f",
    cmap="Blues",
    linewidths=0.5,
    xticklabels=sc_focus_labels,
    yticklabels=sc_focus_labels,
    cbar_kws={"label": "Fraction of true class (recall)"},
    vmin=0, vmax=1,
)
ax3.set_xlabel("Predicted class", fontsize=11)
ax3.set_ylabel("True class", fontsize=11)
ax3.set_title(
    f"Confusion heatmap – top-confused classes – CNN Scratch\n"

```



[illegible]

```

bar_colrs = [bar_color if i == 0 else "#aec7e8" for i in range(top_k)]

bars = ax_bar.barh(barlbls[::-1], bar_vals[::-1], color=bar_colrs[::-1])
ax_bar.axvline(50, color="gray", linestyle="--", linewidth=0.8)
ax_bar.set_xlim(0, 100)
ax_bar.set_xlabel("Confidence (%)")
ax_bar.set_title(
    f"Prediction: {class_names[pred_label].replace('_', ' ')} "
    f"({confidence:.1f}%) "
    f"{'✓ Correct' if is_correct else '✗ Wrong'}",
    fontsize=10, color=bar_color,
)
for bar, val in zip(bars[::-1], bar_vals):
    if val > 1:
        ax_bar.text(val + 0.5, bar.get_y() + bar.get_height() / 2,
            f"{val:.1f}%", va="center", fontsize=8)

plt.tight_layout()
if save_filename:
    save_figure(fig, save_filename)
plt.show()

cnn_scratch.load_state_dict(load_checkpoint("cnn_scratch_best.pt"))
plot_predictions(cnn_scratch, img_test_ds, seed=42,
    save_filename="cnn_scratch_predictions.png")
print("✓ Section 8 complete.")

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 7, in <module>
NameError: name '_base_train_meta' is not defined

```

#### Cell 107: ■ Markdown

### Section 9 — CNN: ResNet-50 (Transfer Learning)

**\*\*Requires GPU — run in Google Colab.\*\***

Reuses `train_one_epoch`, `evaluate`, and `criterion_cnn` defined in Section 8.2.

|-|-|

| **9.2** | Phase A — train head only (5 epochs, LR 1e-3) |

| \*\*9.3.1\*\* | Resume Phase B (same session, after interruption) |

**\*\*9.4\*\*** | Learning curves (Phase A + B combined, phase boundary marked) |

| **\*\*9.5.1\*\*** | Test visualisations: per-class accuracy bars, confused pairs, heatmap |

| \*\*9.7\*\* | CNN Scratch vs. ResNet-50 discussion |

| Phase        | Layers trained | Epochs | LR    | Why                           |
|--------------|----------------|--------|-------|-------------------------------|
| Pre-training | None           | 1000   | 0.001 | Initial weight initialization |
| Phase 1      | 1-2            | 1000   | 0.001 | Initial weight initialization |
| Phase 2      | 3-4            | 1000   | 0.001 | Initial weight initialization |
| Phase 3      | 5-6            | 1000   | 0.001 | Initial weight initialization |
| Phase 4      | 7-8            | 1000   | 0.001 | Initial weight initialization |
| Phase 5      | 9-10           | 1000   | 0.001 | Initial weight initialization |
| Phase 6      | 11-12          | 1000   | 0.001 | Initial weight initialization |
| Phase 7      | 13-14          | 1000   | 0.001 | Initial weight initialization |
| Phase 8      | 15-16          | 1000   | 0.001 | Initial weight initialization |
| Phase 9      | 17-18          | 1000   | 0.001 | Initial weight initialization |
| Phase 10     | 19-20          | 1000   | 0.001 | Initial weight initialization |
| Phase 11     | 21-22          | 1000   | 0.001 | Initial weight initialization |
| Phase 12     | 23-24          | 1000   | 0.001 | Initial weight initialization |
| Phase 13     | 25-26          | 1000   | 0.001 | Initial weight initialization |
| Phase 14     | 27-28          | 1000   | 0.001 | Initial weight initialization |
| Phase 15     | 29-30          | 1000   | 0.001 | Initial weight initialization |
| Phase 16     | 31-32          | 1000   | 0.001 | Initial weight initialization |
| Phase 17     | 33-34          | 1000   | 0.001 | Initial weight initialization |
| Phase 18     | 35-36          | 1000   | 0.001 | Initial weight initialization |
| Phase 19     | 37-38          | 1000   | 0.001 | Initial weight initialization |
| Phase 20     | 39-40          | 1000   | 0.001 | Initial weight initialization |
| Phase 21     | 41-42          | 1000   | 0.001 | Initial weight initialization |
| Phase 22     | 43-44          | 1000   | 0.001 | Initial weight initialization |
| Phase 23     | 45-46          | 1000   | 0.001 | Initial weight initialization |
| Phase 24     | 47-48          | 1000   | 0.001 | Initial weight initialization |
| Phase 25     | 49-50          | 1000   | 0.001 | Initial weight initialization |
| Phase 26     | 51-52          | 1000   | 0.001 | Initial weight initialization |
| Phase 27     | 53-54          | 1000   | 0.001 | Initial weight initialization |
| Phase 28     | 55-56          | 1000   | 0.001 | Initial weight initialization |
| Phase 29     | 57-58          | 1000   | 0.001 | Initial weight initialization |
| Phase 30     | 59-60          | 1000   | 0.001 | Initial weight initialization |
| Phase 31     | 61-62          | 1000   | 0.001 | Initial weight initialization |
| Phase 32     | 63-64          | 1000   | 0.001 | Initial weight initialization |
| Phase 33     | 65-66          | 1000   | 0.001 | Initial weight initialization |
| Phase 34     | 67-68          | 1000   | 0.001 | Initial weight initialization |
| Phase 35     | 69-70          | 1000   | 0.001 | Initial weight initialization |
| Phase 36     | 71-72          | 1000   | 0.001 | Initial weight initialization |
| Phase 37     | 73-74          | 1000   | 0.001 | Initial weight initialization |
| Phase 38     | 75-76          | 1000   | 0.001 | Initial weight initialization |
| Phase 39     | 77-78          | 1000   | 0.001 | Initial weight initialization |
| Phase 40     | 79-80          | 1000   | 0.001 | Initial weight initialization |
| Phase 41     | 81-82          | 1000   | 0.001 | Initial weight initialization |
| Phase 42     | 83-84          | 1000   | 0.001 | Initial weight initialization |
| Phase 43     | 85-86          | 1000   | 0.001 | Initial weight initialization |
| Phase 44     | 87-88          | 1000   | 0.001 | Initial weight initialization |
| Phase 45     | 89-90          | 1000   | 0.001 | Initial weight initialization |
| Phase 46     | 91-92          | 1000   | 0.001 | Initial weight initialization |
| Phase 47     | 93-94          | 1000   | 0.001 | Initial weight initialization |
| Phase 48     | 95-96          | 1000   | 0.001 | Initial weight initialization |
| Phase 49     | 97-98          | 1000   | 0.001 | Initial weight initialization |
| Phase 50     | 99-100         | 1000   | 0.001 | Initial weight initialization |

|   |                             |   |      |                                                             |
|---|-----------------------------|---|------|-------------------------------------------------------------|
| A | Head only (backbone frozen) | 5 | 1e-3 | Stable head initialisation before touching backbone weights |
|---|-----------------------------|---|------|-------------------------------------------------------------|

Checkpoints saved every epoch; best model (lowest Phase B val loss) saved to `cnn\_resnet50\_best.pt`.

```
9.1 Build ResNet-50
resnet50 = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)

Freeze all layers
for p in resnet50.parameters():
 p.requires_grad = False

Replace classification head → 101 classes
```

```
resnet50.fc = nn.Linear(2048, 101)
resnet50 = resnet50.to(DEVICE)

frozen = sum(p.numel() for p in resnet50.parameters() if not p.requires_grad)
trained = sum(p.numel() for p in resnet50.parameters() if p.requires_grad)

print(f"ResNet-50 - frozen: {frozen:,} trainable: {trained:,}")

assert resnet50(torch.randn(2, 3, 224, 224).to(DEVICE)).shape == (2, 101)
print("Forward pass: OK → shape (2, 101)")
```

**Output:**

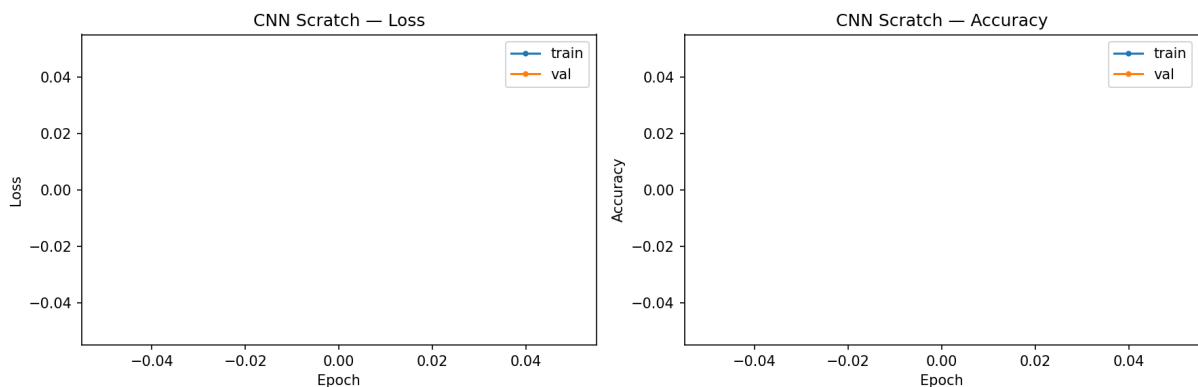
```
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to C:\Users\jolanta.stut
```

ResNet-50 – frozen: 23,508,032   trainable: 206,949

Forward pass: OK  $\rightarrow$  shape (2, 101)

[STDERR]

```
■ 0%| | 0.00/97.8M [00:00<?; ?B/s]■ 0%| | 384k/97.8M [00:00<00:51, 2.0
```



## Cell 109: ■ Markdown

## 9.2 — Phase A: train head only (epochs 1–5, LR 1e-3)

The backbone is fully frozen. Only the replacement FC layer is trained. This prevents noisy early gradients from corrupting pre-trained ImageNet features.

Cell 110: ■ Code



```

["train_one_epoch", "evaluate", "criterion_cnn",
"img_train_loader", "img_val_loader"]
if name not in dir() and name not in vars():
if _missing:
raise RuntimeError(
f"\n■ Missing from kernel: {_missing}\n"
" After reconnecting, re-run these sections IN ORDER before Section 9.2:\n"
" 1. Section 1 (1.0 → 1.3) – imports, paths, Drive mount, helpers\n"
" 2. Section 2 – Food-101 raw data loading (ds_train, ds_test)\n"
" 3. Section 4.2 – index arrays (_idx_train, _idx_val, _idx_test)\n"
" 4. Section 6 – image transforms + DataLoaders\n"
" 5. Section 8.1 – CNNScratch architecture + criterion_cnn\n"
" 6. Section 8.2 – train_one_epoch, evaluate\n"
" 7. Section 9.1 – rebuild ResNet-50\n"
" Then re-run this cell."
)

opt_rn = Adam(resnet50.fc.parameters(), lr=1e-3)
sch_rn = ReduceLROnPlateau(opt_rn, factor=0.5, patience=2)
hist_rn = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
best_rn_a_loss = float("inf")

print("■■ Phase A (frozen backbone, head only) ■■")
for epoch in range(1, 6):
tr_loss, tr_acc = train_one_epoch(resnet50, img_train_loader, criterion_cnn,
opt_rn)
vl_loss, vl_acc = evaluate(resnet50, img_val_loader, criterion_cnn)
sch_rn.step(vl_loss)
hist_rn["train_loss"].append(tr_loss); hist_rn["val_loss"].append(vl_loss)
hist_rn["train_acc"].append(tr_acc); hist_rn["val_acc"].append(vl_acc)
save_checkpoint(
{"epoch": epoch, "model_state": resnet50.state_dict(),
"optimizer_state": opt_rn.state_dict(), "val_loss": vl_loss,
"history": hist_rn},
f"resnet50_phaseA_ckpt_ep{epoch:02d}.pt"
)

```



```

for p in resnet50.layer4.parameters():
 p.requires_grad = True

opt_rn_b = Adam(filter(lambda p: p.requires_grad, resnet50.parameters()), lr=1e-4)
sch_rn_b = ReduceLROnPlateau(opt_rn_b, factor=0.5, patience=2)
best_rn_loss = float("inf")

print("■■ Phase B (layer4 + head unfrozen) ■■")
for epoch in range(6, 16):
 tr_loss, tr_acc = train_one_epoch(resnet50, img_train_loader, criterion_cnn,
 opt_rn_b)
 vl_loss, vl_acc = evaluate(resnet50, img_val_loader, criterion_cnn)
 sch_rn_b.step(vl_loss)
 hist_rn["train_loss"].append(tr_loss); hist_rn["val_loss"].append(vl_loss)
 hist_rn["train_acc"].append(tr_acc); hist_rn["val_acc"].append(vl_acc)
 save_checkpoint(
 {"epoch": epoch, "model_state": resnet50.state_dict(),
 "optimizer_state": opt_rn_b.state_dict(), "val_loss": vl_loss,
 "history": hist_rn},
 f"resnet50_phaseB_ckpt_ep{epoch:02d}.pt"
)
 print(f"Ep {epoch} | train {tr_loss:.4f}/{tr_acc:.3f} | val
 {vl_loss:.4f}/{vl_acc:.3f}")
 if vl_loss < best_rn_loss:
 best_rn_loss = vl_loss
 save_checkpoint(resnet50.state_dict(), "cnn_resnet50_best.pt")

print("\n✓ Phase B complete – best model saved as cnn_resnet50_best.pt")

```

#### Output:

```
■■ Phase B (layer4 + head unfrozen) ■■
```

#### Error:

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 14, in <module>

NameError: name 'img\_train\_loader' is not defined

## Cell 113: ■ Markdown

### 9.3.1 — Resume Phase B from checkpoint

If Phase B was interrupted mid-run, reload the latest Phase B epoch checkpoint and continue from where training stopped (session must still be active).

## Cell 114: ■ Code

```
■■ Step 2: Find the latest Phase B checkpoint (fall back to Phase A best) ■■■■
resume_from = None
start_epoch = 6
best_rn_loss = float("inf")
ckpt = None
```

```
for ep in range(15, 5, -1):
 try:
 ckpt = load_checkpoint(f"resnet50_phaseB_ckpt_ep{ep:02d}.pt")
 resume_from = ep
 break
except FileNotFoundError:
 pass
```

```
if resume_from is None:
 print(" No Phase B checkpoint found – loading Phase A best to start Phase B")
 # Try to restore Phase A history from its last checkpoint
 try:
 _ckpt_a = load_checkpoint("resnet50_phaseA_ckpt_ep05.pt")
```



```
f"resnet50_phaseB_ckpt_ep{epoch:02d}.pt"
)

print(f" Ep {epoch} | train {tr_loss:.4f}/{tr_acc:.3f} | val
{vl_loss:.4f}/{vl_acc:.3f}")

if vl_loss < best_rn_loss:
 best_rn_loss = vl_loss

save_checkpoint(resnet50.state_dict(), "cnn_resnet50_best.pt")

print("\n✓ Phase B complete – best model saved as cnn_resnet50_best.pt")
```

**Error:**

```
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 20, in <module>
NameError: name 'load_checkpoint' is not defined
```

## Cell 115: ■ Markdown

### 9.3.2 — Restore training state from checkpoint

After a kernel restart (e.g. Colab disconnect), training results and helper functions are lost from memory but checkpoints are still saved on Google Drive. This cell restores **everything** needed by Sections 9.4–9.5:

- ``evaluate()`` and ``train_one_epoch()`` functions
- ``criterion_cnn`` (CrossEntropyLoss)
- ``hist_rn`` (training curves — Phase A + Phase B combined)
- ``resnet50`` best weights (``cnn_resnet50_best.pt``)

> **\*\*When to use:\*\*** Run this cell instead of 9.3 / 9.3-RESUME when you only need to produce learning curves, evaluation, or visualisations from a previously completed training run.

## Cell 116: ■ Code

```

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, n = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        logits = model(imgs)
        total_loss += criterion(logits, labels).item() * len(labels)
        correct += (logits.argmax(1) == labels).sum().item()
    n += len(labels)
    return total_loss / n, correct / n

```

```

def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct, n = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        logits = model(imgs)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(labels)
        correct += (logits.argmax(1) == labels).sum().item()
    n += len(labels)
    return total_loss / n, correct / n

```

```

criterion_cnn = nn.CrossEntropyLoss()

```

```

# ■■ 2. Restore hist_rn – Phase B checkpoint first, fall back to Phase A ■■■■■■■■
_rn_b_ckpts = sorted(glob.glob(str(WEIGHTS / 'resnet50_phaseB_ckpt_ep*.pt')))
_rn_a_ckpts = sorted(glob.glob(str(WEIGHTS / 'resnet50_phaseA_ckpt_ep*.pt')))

```

```

if _rn_b_ckpts:
    _last_ckpt_rn = load_checkpoint(os.path.basename(_rn_b_ckpts[-1]))
    _source = f'Phase B ({os.path.basename(_rn_b_ckpts[-1])})'
elif _rn_a_ckpts:

```

**Error:**

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 37, in <module>
NameError: name 'WEIGHTS' is not defined
```

## Cell 117: ■ Markdown

## 9.4 — Learning curves

Plot combined Phase A + Phase B training and validation loss/accuracy. The phase boundary (epoch 5) is marked with a vertical dashed line to show where backbone unfreezing occurs.

## Cell 118: ■ Code



**Error:**

```
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 3, in <module>
NameError: name 'hist_rn' is not defined
```

### Cell 119: ■ Markdown

### 9.5 — Final test evaluation

The **test set** is used exactly once here. Loads ``cnv_resnet50_best.pt`` (best Phase B val loss) and reports test accuracy and macro F1, with a side-by-side comparison against CNN Scratch (Section 8.5).

Cell 120: ■ Code

```

rn_preds, rn_labels = [], []
resnet50.eval()
with torch.no_grad():
    for imgs, labels in img_test_loader:
        preds = resnet50(imgs.to(DEVICE)).argmax(1).cpu()
        rn_preds.extend(preds.tolist())
        rn_labels.extend(labels.tolist())

rn_f1 = f1_score(rn_labels, rn_preds, average="macro")
print(f"ResNet-50 – Test accuracy: {rn_test_acc:.4f} Macro F1: {rn_f1:.4f}")

# Side-by-side comparison – only available if Section 8.5 ran in this session
if "test_acc" in vars() and "macro_f1" in vars():
    print(f"CNN Scratch – Test accuracy: {test_acc:.4f} Macro F1: {macro_f1:.4f}")
    print(f"ResNet-50 gain : +{(rn_test_acc - test_acc)*100:.1f} pp accuracy")
else:
    print("CNN Scratch – results not in memory (re-run §8.5 to compare)")
print("\n✓ 9.5 – ResNet-50 test evaluation complete.")

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 4, in <module>
NameError: name 'load_checkpoint' is not defined

```

#### Cell 121: ■ Markdown

##### 9.5.1 — Test evaluation: visualisations

Three charts complementing the classification report in 9.5:

1. **Top-20 best / worst classes** — per-class accuracy bars
2. **Top-15 most confused pairs** — bar chart of highest off-diagonal confusion counts
3. **Focused confusion heatmap** — only the classes involved in the top-15 confused pairs, row-normalised so each cell is a recall fraction

#### Cell 122: ■ Code



```
cm_no_diag = cm.copy()
np.fill_diagonal(cm_no_diag, 0)
```

```
TOP_PAIRS = 15

flat_idx = np.argsort(cm_no_diag.ravel())[-TOP_PAIRS:][::-1]

true_idx = flat_idx // 101

pred_idx = flat_idx % 101

counts = cm_no_diag.ravel()[flat_idx]

pair_labels = [

f"[_CLASS_NAMES[t].replace('_', ' ')} →\n[_CLASS_NAMES[p].replace('_', ' ')]"

for t, p in zip(true_idx, pred_idx)

]
```

```
fig2, ax2 = plt.subplots(figsize=(12, 6))
ax2.barh(pair_labels[::-1], counts[::-1], color="#ff7f0e")
ax2.set_xlabel("Number of test images misclassified")
ax2.set_title(f"Top-{TOP_PAIRS} most confused class pairs – ResNet-50\n(true → predicted)")
for i, v in enumerate(counts[::-1]):
 ax2.text(v + 0.2, i, str(int(v)), va="center", fontsize=9)
plt.tight_layout()
save_figure(fig2, "resnet50_confused_pairs.png")
plt.show()
```

```
n_focus = len(focus_idx)

fig3, ax3 = plt.subplots(figsize=(max(10, n_focus * 0.7), max(8, n_focus * 0.6)))

sns.heatmap(cm_focus_norm, ax=ax3, annot=True, fmt=".2f", cmap="Blues",
 linewidths=0.5, xticklabels=focus_labels, yticklabels=focus_labels,
 cbar_kws={"label": "Fraction of true class (recall)"}, vmin=0, vmax=1)

ax3.set_xlabel("Predicted class", fontsize=11)
```



```
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 4, in <module>
NameError: name 'load_checkpoint' is not defined
```

## Cell 125: ■ Markdown

### 9.7 — CNN Scratch vs. ResNet-50: Discussion

| Metric | CNN Scratch | ResNet-50 |

|---|---|---|

| Test Accuracy | 47.99% | **76.46%** |

| Macro F1 | 0.467 | **0.764** |

| Trainable parameters (final) | ~8.9 M (all layers) | ~2.1 M (head + layer4) |

| Training phases | Single phase, 20 epochs | Phase A (head, 5 ep) + Phase B (layer4 unfrozen, 10 ep) |

#### Why ResNet-50 outperforms by such a large margin

**1. ImageNet pre-training — a 14-million-image head start.**

ResNet-50 was trained on 14 million ImageNet images before it saw a single food photo. It already recognises textures (crispy batter, glossy sauce, flaky pastry), shapes (round, layered, irregular), and lighting patterns. The CNN scratch model builds all of these representations from 75,750 training images alone — an order of magnitude less data for a far harder initialisation problem.

**2. Representational depth — 50 layers vs. 4 blocks.**

ResNet-50 has 50 convolutional layers and residual skip connections that allow gradients to flow cleanly through the entire network. The scratch CNN has 4 double-conv blocks, giving it roughly 8 conv layers. With 101 classes to separate, many of them visually similar (spaghetti bolognese vs. spaghetti carbonara; chocolate cake vs. red velvet cake), shallow architectures cannot build fine-grained discriminative representations in the final layers.

**3. Two-phase fine-tuning — stable feature extraction before specialisation.**

Phase A froze the backbone and trained only the classification head for 5 epochs. This prevented the pre-trained features from being corrupted by noisy early gradients. Phase B then unfroze only layer4 — the final ResNet block — and fine-tuned at a 10x lower learning rate (1e-4 vs 1e-3). This controlled unfreezing preserved low-level features (edges, textures) while adapting high-level features (food-specific patterns) to the target domain.

**4. Domain similarity — ImageNet contains food.**

ImageNet includes food categories (bananas, pizza, strawberries, ice cream). The pre-trained feature maps are not just generically useful — they are specifically well-calibrated for the visual properties of food photography: colour saturation, plated presentation, overhead angles. The scratch CNN must discover all of this without any prior.

#### ##### Failure mode analysis

Both models share the same top failure pairs, confirming the difficulty is inherent to the data rather than architecture-specific:

- **Steak ↔ Filet mignon** — identical presentation; label distinction is portion cut, not visual appearance
- **Spaghetti bolognese — Spaghetti carbonara** — colour difference (red vs. cream sauce) often lost under harsh lighting
- **Chocolate cake ↔ Red velvet cake** — distinguishable only by cross-section; plated whole-cake photos are nearly identical

ResNet-50 recovers faster from these ambiguous pairs because its deeper feature hierarchy captures subtle cues (sauce sheen, garnish colour, crust texture) that the scratch model cannot resolve at its depth.

#### ##### Conclusion

For a 101-class food classification task, transfer learning from ImageNet is not a marginal improvement — it is the decisive factor. ResNet-50 achieves **76.5% test accuracy** against the scratch model's 48.0%, a gap of **+28.5 percentage points**. The scratch architecture is a valid proof-of-concept and demonstrates that the training pipeline works end-to-end, but ResNet-50 is the model used in the Wine Peer pairing pipeline.

### Cell 126: ■ Markdown

## Section 10 — CNN Explainability: Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) visualises **which spatial regions** of a food image drove ResNet-50's classification decision. Examining where the model attends confirms it uses food-relevant features (textures, plating shapes, garnish colour) rather than background artefacts.

Six representative examples are shown — two correct top-1 predictions, two wrong top-1 (true class not in top-5), and two top-5 hits (correct class recognised but not ranked first).

| Sub-section | What |

|---|---|

| **10.1** | Restore prerequisites after kernel restart (skip if session still active) |

| **10.2** | Grad-CAM visualisation — 6 annotated examples |

### Cell 127: ■ Markdown

#### 10.1 — Restore prerequisites after kernel restart

After reconnecting to Colab, re-run **Section 1** (imports, paths, helpers) first, then run this cell to rebuild `resnet50`` and `img_test_loader``. Skip entirely if the session is still active.

## Cell 128: ■ Code

**Error:**

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~^~~~~~
 File "<string>", line 9, in <module>
NameError: name 'load_checkpoint' is not defined
```

## Cell 129: ■ Markdown



## 10.2 — Grad-CAM visualisation

Hooks into ``layer4[-1]`` (the final residual block) to capture activations and gradients, then overlays the resulting heatmap on the original image using ``torchcam``.

- **Row 1** — original food image with true label, predicted label, and case tag
  - **Row 2** — Grad-CAM overlay (warm regions = high activation, cool = low activation)
  - Green title = correct top-1 | Orange = top-5 hit (wrong top-1) | Red = complete miss
- > **Requires:** `pip install torchcam` — run once per Colab session.

## Cell 130: ■ Code

```
elif top1[i] != lbl and lbl in top5[i].tolist() and len(examples["top5"]) < 2:
 examples["top5"].append((imgs[i].detach(), lbl, top1[i].item(), "top-5 hit"))

elif top1[i] != lbl and lbl not in top5[i].tolist() and len(examples["wrong"]) < 2:
 examples["wrong"].append((imgs[i].detach(), lbl, top1[i].item(), "wrong X"))

if all(len(v) >= 2 for v in examples.values()):
 break
```

```
all_ex = examples["correct"] + examples["top5"] + examples["wrong"]
```

```
fig, axes = plt.subplots(2, 6, figsize=(22, 8))
_color_map = {"correct ✓": "green", "top-5 hit": "orange", "wrong ✗": "red"}
```

```
for col, (img_t, true_lbl, pred_lbl, case) in enumerate(all_ex):
 out = resnet50(img_t.unsqueeze(0).to(DEVICE))
 cam = cam_extractor(pred_lbl, out)[0]
```

```
orig = (img_t.cpu() * _STD + _MEAN).clamp(0, 1)
overlay = overlay_mask(to_pil_image(orig), to_pil_image(cam.squeeze(0)), alpha=0.5)
```

```
color = _color_map[case]
axes[0, col].imshow(to_pil_image(orig))
axes[0, col].set_title(
 f"True: {_CLASS_NAMES[true_lbl].replace('_', ' ')}\n"
 f"Pred: {_CLASS_NAMES[pred_lbl].replace('_', ' ')}\n"
 f"[{case}]",
 fontsize=7, color=color)
axes[0, col].axis("off")
axes[1, col].imshow(overlay)
axes[1, col].set_title("Grad-CAM", fontsize=7)
axes[1, col].axis("off")
```

```
plt.suptitle("Section 10 – Grad-CAM: what ResNet-50 attends to when classifying
food",
 fontsize=12)
plt.tight_layout()
save_figure(fig, "gradcam_resnet50.png")
```

```
plt.show()
cam_extractor.remove_hooks()
print("✓ Section 10 complete — Grad-CAM figure saved → gradcam_resnet50.png")
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 7, in <module>
 exec('\n'.join(lines[:-1]), glb)
~~~~~
File "<string>", line 7, in <module>
NameError: name '_base_train_meta' is not defined
```

#### Cell 131: ■ Markdown

### Section 11 — RNN / LSTM Branch (Text Model)

This section trains **three text classification models** on the **WineSensed** wine-review dataset to predict **grape variety** (15 classes) from tasting-note text, progressing from a simple recurrent baseline to a pre-trained Transformer encoder.

#### Sub-sections

| Sub-section | Model | Content |

|---|---|---|

| **11.1** | — | `WineTextDataset` + length-aware `DataLoader` objects |

| **11.2** | LSTM Baseline | Variation 1 — Unidirectional 2-layer LSTM |

| **11.3** | — | Shared training utilities (`train\_text\_epoch`, `eval\_text`) |

| **11.4** | LSTM Baseline | Training loop with per-epoch Drive checkpoints |

| **11.5** | LSTM Baseline | Test evaluation — accuracy, Macro F1, curves, confusion matrix |

| **11.6** | BiLSTM + Attention | Variation 2 — Bidirectional LSTM + Bahdanau attention |

| **11.7** | BiLSTM + Attention | Training loop with per-epoch Drive checkpoints |

| **11.8** | BiLSTM + Attention | Test evaluation — accuracy, Macro F1, curves, confusion matrix |

| **11.9** | — | Side-by-side comparison: LSTM vs BiLSTM + Attention |

| **11.10** | DistilBERT | **Bonus (+3 pts)** — Build `WineBertDataset` with `transformers` tokeniser |

| **11.11** | DistilBERT | Fine-tune linear head on frozen DistilBERT backbone |

| **11.12** | DistilBERT | Test evaluation + **3-model comparison** (LSTM / BiLSTM / DistilBERT) |

#### Model overview

| Model | Architecture | Embeddings | Trainable params |

|---|---|---|---|

| **\*\*LSTM Baseline\*\*** | 2-layer unidirectional LSTM → final hidden state → FC | W2V 100-d, fine-tuned (§7.4) | ~3–5 M |

| **\*\*BiLSTM + Attention\*\*** | 2-layer BiLSTM + Bahdanau attention → context vector → FC | W2V 100-d, fine-tuned (§7.4) | ~5–8 M |

| **\*\*DistilBERT (frozen)\*\*** | Frozen DistilBERT backbone → `[CLS]` token → linear head | Contextual (BERT sub-word) | ~15 K (head only) |

### **Design choices (LSTM / BiLSTM)**

| Choice | Rationale |

|---|---|

| W2V 100-d embeddings, fine-tuned | Trained directly on wine reviews (§7.4) — captures domain vocabulary (`tannins`, `terroir`, `minerality`); embedding weights are further adapted during RNN training |

| `pack\_padded\_sequence` | Ignores `` positions during LSTM forward pass — correct gradient flow |

| Gradient clipping (max-norm 5) | Standard RNN stability measure |

| Weighted CrossEntropy | Corrects for class imbalance across 15 grape varieties (`CLASS\_WEIGHTS` from §7.5) |

| Early stopping (patience = 4) | Prevents overfitting on the relatively small WineSensed corpus |

| Per-epoch Drive checkpoints | Full state saved each epoch — training can resume after a Colab disconnect |

### **Design choices (DistilBERT)**

| Choice | Rationale |

|---|---|

| Frozen backbone | Avoids catastrophic forgetting on the small wine dataset; only the 15-class head is updated |

| `[CLS]` token pooling | Standard BERT practice — the `[CLS]` position is pre-trained to summarise the full sequence |

| Sub-word tokenisation | Handles domain-specific words (e.g. `terroir` → `terr`, `##oir`) without OOV issues |

| `max\_len = 128` | Covers >95% of wine review lengths; GPU-memory-friendly |

**\*\*Prerequisites:\*\*** Section 1 (imports, `DEVICE`, `load\_checkpoint`, `save\_figure`), §4.1 (splits), §7.1–7.6 (`X\_train/val/test`, `lbl\_train/val/test`, `VOCAB`, `VOCAB\_SIZE`, `MAX\_SEQ\_LEN`, `embedding\_matrix`, `CLASS\_WEIGHTS`)

## **Cell 132: ■ Markdown**

## 11.1 — Text Dataset and DataLoaders

``WineTextDataset`` wraps the integer-encoded sequences, their real non-padding lengths (required for ``pack_padded_sequence``), and integer grape labels. This cell builds ``txt_train_loader``, ``txt_val_loader``, and ``txt_test_loader`` shared by all three text models in §11.

Cell 133: ■ Code

```
# 11.1 Text Dataset and DataLoaders
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

class WineTextDataset(torch.utils.data.Dataset):
    """Dataset for padded token-id sequences + sequence lengths + labels."""
    def __init__(self, sequences, labels):
        self.X = torch.tensor(sequences, dtype=torch.long)
        self.y = torch.tensor(labels, dtype=torch.long)
        self.lengths = (self.X != 0).sum(dim=1).clamp(min=1)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, i):
        return self.X[i], self.lengths[i], self.y[i]

TXT_BATCH = 64

txt_train_ds = WineTextDataset(X_train, lbl_train)
txt_val_ds = WineTextDataset(X_val, lbl_val)
txt_test_ds = WineTextDataset(X_test, lbl_test)

txt_train_loader = DataLoader(txt_train_ds, batch_size=TXT_BATCH, shuffle=True,
                               num_workers=0, pin_memory=False)
txt_val_loader = DataLoader(txt_val_ds, batch_size=TXT_BATCH, shuffle=False,
                             num_workers=0, pin_memory=False)
txt_test_loader = DataLoader(txt_test_ds, batch_size=TXT_BATCH, shuffle=False,
                              num_workers=0, pin_memory=False)

print(f"{'Split':<10} {'Samples':>8} {'Batches':>8}")
print("-" * 30)
for _name, _ds, _ldr in [("train", txt_train_ds, txt_train_loader),
```

```

("val", txt_val_ds, txt_val_loader),
("test", txt_test_ds, txt_test_loader)]:
print(f"_{name}<10> {len(_ds)>8,} {len(_ldr)>8,}")

_xb_t, _lb_t, _yb_t = next(iter(txt_train_loader))
print(f"\nBatch shapes:")
print(f" sequences : {_xb_t.shape} (B x MAX_SEQ_LEN={MAX_SEQ_LEN})")
print(f" lengths : {_lb_t.shape} min={_lb_t.min()}, max={_lb_t.max()}")
print(f" labels : {_yb_t.shape} classes 0-{_yb_t.max()}")
print(f"\n✓ 11.1 – Text DataLoaders ready.")

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 20, in <module>
NameError: name 'X_train' is not defined

```

#### Cell 134: ■ Markdown

### 11.2 — Variation 1: Unidirectional LSTM Baseline

**\*\*Architecture:\*\***

...

Input ( $B \times L$ ) → Embedding (W2V 100-d, fine-tuned)

→ Dropout(0.4)

→ LSTM (100-d → 256-d, 2 layers, unidirectional)

→ Last hidden state of final layer ( $B \times 256$ )

→ Dropout(0.4)

→ Linear(256 → 15)

...

- Reads each wine review **\*\*left-to-right only\*\***; the final hidden state summarises the whole sequence.

- `pack\_padded\_sequence` ensures `` tokens do not contribute to gradients.

- This is the **\*\*baseline\*\*** — deliberately simple so improvements in §11.6 are clearly attributable to bidirectionality and attention.

## Cell 135: ■ Code

```
_total = sum(p.numel() for p in lstm_model.parameters())
print(f"LSTMBaseline - total params : {_total:,}")

_out_lstm = lstm_model(_xb_t.to(DEVICE), _lb_t.to(DEVICE))
assert _out_lstm.shape == (TXT_BATCH, len(GRAPE_CLASSES))
print(f"Forward pass : OK → {_out_lstm.shape}")
print("✓ 11.2 - LSTMBaseline ready.")
```

**Error:**

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
File "<string>", line 37, in <module>  
NameError: name 'VOCAB_SIZE' is not defined
```

## Cell 136: ■ Markdown

### 11.3 — Shared training utilities

Two helper functions shared by the LSTM Baseline (§11.4) and BiLSTM + Attention (§11.7):

Function	Role
1. <b>Identify the problem</b>	1. <b>Identify the problem</b>
2. <b>Define the problem</b>	2. <b>Define the problem</b>
3. <b>Generate hypotheses</b>	3. <b>Generate hypotheses</b>
4. <b>Test hypotheses</b>	4. <b>Test hypotheses</b>
5. <b>Evaluate results</b>	5. <b>Evaluate results</b>
6. <b>Communicate findings</b>	6. <b>Communicate findings</b>
7. <b>Implement solutions</b>	7. <b>Implement solutions</b>
8. <b>Monitor progress</b>	8. <b>Monitor progress</b>
9. <b>Adjust strategies</b>	9. <b>Adjust strategies</b>
10. <b>Conclude the process</b>	10. <b>Conclude the process</b>

— — — — —

```
| `train_text_epoch(model, loader, criterion, optimizer)` | One full training pass; returns mean loss and accuracy |
```

```
| `eval_text(model, loader, criterion)` | No-grad validation/test pass; returns mean loss and accuracy |
```

## Cell 137: ■ Code



```

seqs, lengths, labels = seqs.to(DEVICE), lengths.to(DEVICE), labels.to(DEVICE)
optimizer.zero_grad()
logits = model(seqs, lengths)
loss = criterion(logits, labels)
loss.backward()
nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
optimizer.step()
total_loss += loss.item() * len(labels)
correct += (logits.argmax(1) == labels).sum().item()
n += len(labels)
return total_loss / n, correct / n

```

```

@torch.no_grad()
def eval_text(model, loader, criterion):
    model.eval()
    total_loss, correct, n = 0.0, 0, 0
    for seqs, lengths, labels in loader:
        seqs, lengths, labels = seqs.to(DEVICE), lengths.to(DEVICE), labels.to(DEVICE)
        logits = model(seqs, lengths)
        total_loss += criterion(logits, labels).item() * len(labels)
        correct += (logits.argmax(1) == labels).sum().item()
        n += len(labels)
    return total_loss / n, correct / n

```

```

criterion_txt = nn.CrossEntropyLoss(weight=CLASS_WEIGHTS.to(DEVICE))
print("✓ 11.3 – training utilities ready.")
print(f" criterion_txt : CrossEntropyLoss(weight=CLASS_WEIGHTS) –
{len(GRAPE_CLASSES)} classes")

```

#### Error:

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 30, in <module>

NameError: name 'CLASS_WEIGHTS' is not defined

Cell 138: ■ Markdown

11.4 — Training the LSTM Baseline

| Setting | Value |
|---------|-------|
|---------|-------|

|-|-|-

| Optimiser | Adam, lr = 1e-3, weight_decay = 1e-5 |

| LR scheduler | ReduceLROnPlateau $\times 0.5$, patience 2 |

| Max epochs | 30 |

| Early stopping | patience = 4 |

| Gradient clipping | max-norm 5 |

Best weights saved to `weights/lstm\_best.pt`.

Cell 139: ■ Code

■■ 11.4 Train the LSTM Baseline

```
TXT_EPOCHS = 30
```

```
TXT_PATIENCE = 4
```

```
opt_lstm = Adam(lstm_model.parameters(), lr=1e-3, weight_decay=1e-5)
```

```
sched_lstm = ReduceLROnPlateau(opt_lstm, factor=0.5, patience=2)
```

```
history_lstm = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
```

```
best_val_loss_lstm = float("inf")
```

```
no_improve_lstm = 0
```

```
best_ckpt_lstm = WEIGHTS / "lstm_best.pt"
```

```
print(f"Training LSTMBaseline (max {TXT_EPOCHS} epochs, patience={TXT_PATIENCE})")
```

```
print(f"{'Epoch':>5} {'tr_loss':>8} {'tr_acc':>7} {'vl_loss':>8}  
{'vl_acc':>7} {'lr':>9}")
```

```
print("_" * 57)
```

```
for epoch in range(1, TXT_EPOCHS + 1):
```

```
tr_loss, tr_acc = train_text_epoch(lstm_model, txt_train_loader, criterion_txt,
opt_lstm)
```

```
vl_loss, vl_acc = eval_text(lstm_model, txt_val_loader, criterion_txt)
```

```
history_lstm["train_loss"].append(tr_loss)
```

```
history_lstm["val_loss"].append(vl_loss)
```

```
history_lstm["train_acc"].append(tr_acc)
```

```

history_lstm["val_acc"].append(vl_acc)
sched_lstm.step(vl_loss)
lr_now = opt_lstm.param_groups[0]["lr"]
if vl_loss < best_val_loss_lstm:
    best_val_loss_lstm = vl_loss
    no_improve_lstm = 0
    save_checkpoint(lstm_model.state_dict(), "lstm_best.pt")
    marker = " ✓"
else:
    no_improve_lstm += 1
    marker = ""

# ■■ per-epoch full checkpoint (enables resume after disconnect) ■■■■■■■■■■■■
save_checkpoint(
    {"epoch": epoch, "model_state": lstm_model.state_dict(),
     "optimizer_state": opt_lstm.state_dict(), "val_loss": vl_loss,
     "history": history_lstm},
    f"lstm_ckpt_ep{epoch:02d}.pt"
)

print(f"epoch:>5} {tr_loss:>8.4f} {tr_acc:>7.4f} {vl_loss:>8.4f}
      {vl_acc:>7.4f} {lr_now:>9.2e}{marker}")

if no_improve_lstm >= TXT_PATIENCE:
    print(f"\nEarly stopping at epoch {epoch}.")
    break

print(f"\nBest val loss : {best_val_loss_lstm:.4f} → {best_ckpt_lstm}")
print("✓ 11.4 – LSTM Baseline training complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'lstm_model' is not defined

```

Cell 140: ■ Markdown

11.5 — LSTM Baseline: test evaluation

The **test set is used exactly once** here. Loads the best LSTM checkpoint and computes test accuracy, Macro F1, per-class classification report, learning curves, and confusion matrix.

Cell 141: ■ Code

```
# ■■ 11.5 LSTM Baseline – test evaluation, curves, confusion matrix ■■■■■■■■■■■■

lstm_model.load_state_dict(load_checkpoint("lstm_best.pt"))
lstm_model.eval()

all_preds_lstm, all_labels_lstm = [], []
with torch.no_grad():
    for seqs, lengths, labels in txt_test_loader:
        logits = lstm_model(seqs.to(DEVICE), lengths.to(DEVICE))
        all_preds_lstm.extend(logits.argmax(1).cpu().tolist())
        all_labels_lstm.extend(labels.tolist())

lstm_test_acc = sum(p == l for p, l in zip(all_preds_lstm, all_labels_lstm)) /
len(all_labels_lstm)

lstm_f1 = f1_score(all_labels_lstm, all_preds_lstm, average="macro")

print(f"LSTM Baseline – Test accuracy : {lstm_test_acc:.4f}
({lstm_test_acc*100:.2f}%)")

print(f" Macro F1 : {lstm_f1:.4f}")

print()

print(classification_report(all_labels_lstm, all_preds_lstm,
target_names=GRAPE_CLASSES, digits=3))

_epochs_lstm = range(1, len(history_lstm["train_loss"]) + 1)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(_epochs_lstm, history_lstm["train_loss"], label="Train", lw=1.8)
axes[0].plot(_epochs_lstm, history_lstm["val_loss"], label="Val", lw=1.8)
axes[0].set_title("LSTM Baseline – Loss"); axes[0].set_xlabel("Epoch");
axes[0].legend()

axes[1].plot(_epochs_lstm, history_lstm["train_acc"], label="Train", lw=1.8)
axes[1].plot(_epochs_lstm, history_lstm["val_acc"], label="Val", lw=1.8)
axes[1].set_title("LSTM Baseline – Accuracy"); axes[1].set_xlabel("Epoch");
axes[1].legend()

plt.suptitle(f"11.5 – LSTM Baseline | Test Acc: {lstm_test_acc*100:.2f}% Macro-F1:
{lstm_f1:.4f}", fontsize=12)

plt.tight_layout()
```

```

save_figure(fig, "lstm_curves.png")
plt.show()

lstm_cm = confusion_matrix(all_labels_lstm, all_preds_lstm)
fig, ax = plt.subplots(figsize=(13, 11))
sns.heatmap(lstm_cm, annot=True, fmt="d", cmap="Blues",
xticklabels=GRAPE_CLASSES, yticklabels=GRAPE_CLASSES, ax=ax, linewidths=0.3)
ax.set_xlabel("Predicted"); ax.set_ylabel("True")
ax.set_title(f"LSTM Baseline – Confusion Matrix (Test Acc:
{lstm_test_acc*100:.2f}%)")
plt.xticks(rotation=45, ha="right", fontsize=8); plt.yticks(fontsize=8)
plt.tight_layout()
save_figure(fig, "lstm_cm.png")
plt.show()
print("✓ 11.5 – LSTM Baseline test evaluation complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'lstm_model' is not defined

```

Cell 142: ■ Markdown

11.6 — Variation 2: Bidirectional LSTM + Bahdanau Attention

****Architecture:****

...

Input ($B \times L$) → Embedding (W2V 100-d, fine-tuned)

→ Dropout(0.4)

→ BiLSTM (100-d → 256-d × 2, 2 layers)

→ Bahdanau attention over all L hidden states → $B \times 512$

→ Dropout(0.4)

→ Linear(512 → 15)

...

****Bahdanau (additive) attention:****


```

dropout=dropout if n_layers > 1 else 0.0)
self.attention = BahdanauAttention(hidden_dim)
self.drop = nn.Dropout(dropout)
self.fc = nn.Linear(hidden_dim * 2, n_classes)

```

```

def forward(self, x, lengths):
    emb = self.drop(self.embedding(x))
    packed = pack_padded_sequence(emb, lengths.cpu(),
    batch_first=True, enforce_sorted=False)
    output, _ = self.lstm(packed)
    output, _ = pad_packed_sequence(output, batch_first=True, total_length=x.shape[1])
    mask = (x != 0)
    context, _ = self.attention(output, mask)
    return self.fc(self.drop(context))

```

```

def encode(self, x, lengths):
    emb = self.embedding(x)
    packed = pack_padded_sequence(emb, lengths.cpu(),
    batch_first=True, enforce_sorted=False)
    output, _ = self.lstm(packed)
    output, _ = pad_packed_sequence(output, batch_first=True, total_length=x.shape[1])
    context, _ = self.attention(output, (x != 0))
    return context

```

```

bilstm_model = BiLSTMAttention(
vocab_size=VOCAB_SIZE, embed_dim=EMBED_DIM, hidden_dim=HIDDEN_DIM,
n_classes=len(GRAPE_CLASSES), n_layers=N_LAYERS, dropout=DROPOUT_RNN,
embedding_matrix=embedding_matrix,
).to(DEVICE)

```

```

_total_bi = sum(p.numel() for p in bilstm_model.parameters())
print(f"BiLSTMAttention - total params : {_total_bi:,}")
_out_bi = bilstm_model(_xb_t.to(DEVICE), _lb_t.to(DEVICE))
assert _out_bi.shape == (TXT_BATCH, len(GRAPE_CLASSES))
print(f"Forward pass : OK → {_out_bi.shape}")
print("✓ 11.6 - BiLSTMAttention ready.")

```

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
```

File "<string>", line 55, in <module>

NameError: name 'VOCAB_SIZE' is not defined

Cell 144: ■ Markdown

11.7 — Training BiLSTM + Attention

Same training protocol as the LSTM Baseline (11.4) — Adam, ReduceLROnPlateau, early stopping (patience 4), gradient clipping, per-epoch Drive checkpoints. Best weights saved to weights/bilstm_best.pt.

Cell 145: ■ Code

11.7 Train BiLSTM + Attention

```
opt_bilstm = Adam(bilstm_model.parameters(), lr=1e-3, weight_decay=1e-5)
```

```
sched_bilstm = ReduceLROnPlateau(opt_bilstm, factor=0.5, patience=2)
```

```
history_bilstm = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
```

```
best_val_loss_bilstm = float("inf")
```

```
no_improve_bilstm = 0
```

```
best_ckpt_bilstm = WEIGHTS / "bilstm_best.pt"
```

```
print(f"Training BiLSTMAttention (max {TXT_EPOCHS} epochs,  
patience={TXT_PATIENCE})")
```

```
print(f"{'Epoch':>5} {'tr_loss':>8} {'tr_acc':>7} {'vl_loss':>8}  
{'vl_acc':>7} {'lr':>9}")
```

```
print("_" * 57)
```

```
for epoch in range(1, TXT_EPOCHS + 1):
```

```
tr_loss, tr_acc = train_text_epoch(bilstm_model, txt_train_loader, criterion_txt,
opt_bilstm)
```

```
vl_loss, vl_acc = eval_text(bilstm_model, txt_val_loader, criterion_txt)
```

```
history_bilstm["train_loss"].append(tr_loss)
```

```
history_bilstm["val_loss"].append(vl_loss)
```

```
history_bilstm["train_acc"].append(tr_acc)
```

```
history_bilstm["val_acc"].append(val_acc)
```



```

sched_bilstm.step(vl_loss)
lr_now = opt_bilstm.param_groups[0]["lr"]
if vl_loss < best_val_loss_bilstm:
    best_val_loss_bilstm = vl_loss
    no_improve_bilstm = 0
    save_checkpoint(bilstm_model.state_dict(), "bilstm_best.pt")
    marker = " ✓"
else:
    no_improve_bilstm += 1
    marker = ""
# ■■ per-epoch full checkpoint (enables resume after disconnect) ■■■■■■■■■■■■
save_checkpoint(
    {"epoch": epoch, "model_state": bilstm_model.state_dict(),
    "optimizer_state": opt_bilstm.state_dict(), "val_loss": vl_loss,
    "history": history_bilstm},
    f"bilstm_ckpt_ep{epoch:02d}.pt"
)
print(f"epoch:>5} {tr_loss:>8.4f} {tr_acc:>7.4f} {vl_loss:>8.4f}
{vl_acc:>7.4f} {lr_now:>9.2e}{marker}")
if no_improve_bilstm >= TXT_PATIENCE:
    print(f"\nEarly stopping at epoch {epoch}.")
    break

print(f"\nBest val loss : {best_val_loss_bilstm:.4f} → {best_ckpt_bilstm}")
print("✓ 11.7 – BiLSTM + Attention training complete.")

```

Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
File "<string>", line 2, in <module>
NameError: name 'bilstm_model' is not defined

```

Cell 146: ■ Markdown

11.8 — BiLSTM + Attention: test evaluation

Same evaluation format as 11.5. Loads the best BiLSTM checkpoint and computes test accuracy, Macro F1, per-class classification report, learning curves, and confusion matrix.

Cell 147: ■ Code

```
# ■■ 11.8 BiLSTM + Attention – test evaluation, curves, confusion matrix ■■■■■■

bilstm_model.load_state_dict(load_checkpoint("bilstm_best.pt"))
bilstm_model.eval()

all_preds_bilstm, all_labels_bilstm = [], []
with torch.no_grad():
    for seqs, lengths, labels in txt_test_loader:
        logits = bilstm_model(seqs.to(DEVICE), lengths.to(DEVICE))
        all_preds_bilstm.extend(logits.argmax(1).cpu().tolist())
        all_labels_bilstm.extend(labels.tolist())

bilstm_test_acc = sum(p == l for p, l in zip(all_preds_bilstm, all_labels_bilstm))
/ len(all_labels_bilstm)

bilstm_f1 = f1_score(all_labels_bilstm, all_preds_bilstm, average="macro")
print(f"BiLSTM+Attention – Test accuracy : {bilstm_test_acc:.4f}
({bilstm_test_acc*100:.2f}%)")
print(f" Macro F1 : {bilstm_f1:.4f}")
print()
print(classification_report(all_labels_bilstm, all_preds_bilstm,
target_names=GRAPE_CLASSES, digits=3))

_epochs_bi = range(1, len(history_bilstm["train_loss"]) + 1)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(_epochs_bi, history_bilstm["train_loss"], label="Train", lw=1.8)
axes[0].plot(_epochs_bi, history_bilstm["val_loss"], label="Val", lw=1.8)
axes[0].set_title("BiLSTM + Attention – Loss"); axes[0].set_xlabel("Epoch");
axes[0].legend()
axes[1].plot(_epochs_bi, history_bilstm["train_acc"], label="Train", lw=1.8)
axes[1].plot(_epochs_bi, history_bilstm["val_acc"], label="Val", lw=1.8)
axes[1].set_title("BiLSTM + Attention – Accuracy"); axes[1].set_xlabel("Epoch");
axes[1].legend()

plt.suptitle(f"11.8 – BiLSTM + Attention | Test Acc: {bilstm_test_acc*100:.2f}%
Macro-F1: {bilstm_f1:.4f}", fontsize=12)
plt.tight_layout()
save_figure(fig, "bilstm_curves.png")
```



```
print(f"{'Model':<28} {'Test Acc':>10} {'Macro F1':>10}
{'Params':>10}")

print("-" * 65)

print(f"{'LSTM Baseline':<28} {lstm_test_acc*100:>9.2f}% {lstm_f1:>10.4f}
{_lstm_params:>10,}")

print(f"{'BiLSTM + Attention':<28} {bilstm_test_acc*100:>9.2f}%
{bilstm_f1:>10.4f} {_bilstm_params:>10,}")

print("=" * 65)
```

```
# ■■ 4-panel training curves
```



```
e1 = range(1, len(history_lstm["train_loss"]) + 1)
e2 = range(1, len(history_bilstm["train_loss"]) + 1)

fig, axes = plt.subplots(2, 2, figsize=(16, 10))

axes[0,0].plot(e1, history_lstm["train_loss"], label="Train", lw=1.8)
axes[0,0].plot(e1, history_lstm["val_loss"], label="Val", lw=1.8)
axes[0,0].set_title("LSTM Baseline - Loss"); axes[0,0].legend()

axes[0,1].plot(e1, history_lstm["train_acc"], label="Train", lw=1.8)
axes[0,1].plot(e1, history_lstm["val_acc"], label="Val", lw=1.8)
axes[0,1].set_title("LSTM Baseline - Accuracy"); axes[0,1].legend()

axes[1,0].plot(e2, history_bilstm["train_loss"], label="Train", lw=1.8)
axes[1,0].plot(e2, history_bilstm["val_loss"], label="Val", lw=1.8)
axes[1,0].set_title("BiLSTM + Attention - Loss"); axes[1,0].legend()

axes[1,1].plot(e2, history_bilstm["train_acc"], label="Train", lw=1.8)
axes[1,1].plot(e2, history_bilstm["val_acc"], label="Val", lw=1.8)
axes[1,1].set_title("BiLSTM + Attention - Accuracy"); axes[1,1].legend()

plt.suptitle("11.9 - LSTM vs BiLSTM + Attention: Training Curves", fontsize=13)
plt.tight_layout()

save_figure(fig, "rnn_comparison.png")

plt.show()
```

```

correct_c[l] += int(p == l)
return [correct_c[i] / counts[i] if counts[i] else 0 for i in range(n_classes)]

lstm_pca = per_class_acc(all_preds_lstm, all_labels_lstm, len(GRAPE_CLASSES))
bilstm_pca = per_class_acc(all_preds_bilstm, all_labels_bilstm, len(GRAPE_CLASSES))
x = range(len(GRAPE_CLASSES)); w = 0.38
fig, ax = plt.subplots(figsize=(16, 6))
ax.bar([i - w/2 for i in x], lstm_pca, w, label="LSTM Baseline", alpha=0.85,
color="#4C72B0")
ax.bar([i + w/2 for i in x], bilstm_pca, w, label="BiLSTM + Attention", alpha=0.85,
color="#DD8452")
ax.set_xticks(list(x)); ax.set_xticklabels(GRAPE_CLASSES, rotation=45, ha="right",
fontsize=8)
ax.set_ylabel("Per-class Accuracy"); ax.set_ylim(0, 1.05)
ax.axhline(lstm_test_acc, color="#4C72B0", lw=1.2, ls="--", alpha=0.6, label="LSTM
overall")
ax.axhline(bilstm_test_acc, color="#DD8452", lw=1.2, ls="--", alpha=0.6,
label="BiLSTM overall")
ax.set_title("11.9 – Per-class Accuracy: LSTM vs BiLSTM + Attention"); ax.legend()
plt.tight_layout()
save_figure(fig, "rnn_per_class.png")
plt.show()
print("✓ 11.9 – LSTM vs BiLSTM + Attention comparison complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'lstm_model' is not defined

```

Cell 150: ■ Markdown

11.10 — DistilBERT: build DataLoader (Bonus +3 pts)

Tokenises wine reviews using the DistilBERT sub-word tokeniser ('distilbert-base-uncased', 'max_len = 128'). Wraps them in 'WineBertDataset' and builds train / val / test DataLoaders. Sub-word tokenisation handles domain-specific terms ('terroir', 'minerality') without OOV issues.

> ****Requires:**** 'pip install transformers' — run once per Colab session.

Cell 151: ■ Code

```
# 11.10 Build DistilBERT DataLoader
#####

# !pip install transformers ← run once per session if not installed

from transformers import DistilBertTokenizerFast, DistilBertModel

BERT_MODEL_NAME = "distilbert-base-uncased"

BERT_MAX_LEN = 128 # covers >95% of wine review lengths

BERT_BATCH = 32 # smaller than LSTM – transformer is heavier

bert_tokenizer = DistilBertTokenizerFast.from_pretrained(BERT_MODEL_NAME)

class WineBertDataset(torch.utils.data.Dataset):
    """Tokenises raw review strings using the DistilBERT tokeniser."""
    def __init__(self, texts, labels, tokenizer, max_len):
        self.encodings = tokenizer(
            texts, truncation=True, padding="max_length",
            max_length=max_len, return_tensors="pt",
        )
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, i):
        return {
            "input_ids": self.encodings["input_ids"][i],
            "attention_mask": self.encodings["attention_mask"][i],
            "label": self.labels[i],
        }

# txt_train / txt_val / txt_test = raw review strings from §7.5

bert_train_ds = WineBertDataset(txt_train, lbl_train, bert_tokenizer, BERT_MAX_LEN)
bert_val_ds = WineBertDataset(txt_val, lbl_val, bert_tokenizer, BERT_MAX_LEN)
bert_test_ds = WineBertDataset(txt_test, lbl_test, bert_tokenizer, BERT_MAX_LEN)

bert_train_loader = DataLoader(bert_train_ds, batch_size=BERT_BATCH, shuffle=True,
                               num_workers=0)
```

```
print(f"DistilBERT tokenizer : {BERT_MODEL_NAME} | max_len={BERT_MAX_LEN}")
print(f"'Split':&lt;8} {'Samples':&gt;8} {'Batches':&gt;8}")
for _name, _ds, _ldr in [("train", bert_train_ds, bert_train_loader),
                          ("val", bert_val_ds, bert_val_loader),
                          ("test", bert_test_ds, bert_test_loader)]:
print(f"{_name:&lt;8} {len(_ds):&gt;8}, {len(_ldr):&gt;8,}")
print("✓ 11.10 – WineBertDataset and DataLoaders ready.")
```

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"  
    exec('\n'.join(lines[:-1]), glb)  
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
File "<string>", line 33, in <module>  
NameError: name 'txt_train' is not defined
```

[illegible]

```
self.drop = nn.Dropout(dropout)
self.fc = nn.Linear(self.bert.config.hidden_size, n_classes)
```

```
def forward(self, input_ids, attention_mask):
    cls = self.bert(input_ids=input_ids,
                    attention_mask=attention_mask).last_hidden_state[:, 0, :]
    return self.fc(self.drop(cls))
```

```
def encode(self, input_ids, attention_mask):
    """Return 768-d CLS embedding (no_grad – backbone is frozen)."""
    with torch.no_grad():
        return self.bert(input_ids=input_ids,
                           attention_mask=attention_mask).last_hidden_state[:, 0, :]
```

```
bert_model = DistilBertClassifier(model_name=BERT_MODEL_NAME,
n_classes=len(GRAPE_CLASSES)).to(DEVICE)

_total_bert = sum(p.numel() for p in bert_model.parameters())
_train_bert = sum(p.numel() for p in bert_model.parameters() if p.requires_grad)

print(f"DistilBertClassifier - total params : {_total_bert:,}")
print(f" trainable head : {_train_bert:,}")
```

```
opt_bert = Adam(bert_model.fc.parameters(), lr=1e-3, weight_decay=1e-5)
sched_bert = ReduceLROnPlateau(opt_bert, factor=0.5, patience=2)
```

```
history_bert = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
best_val_loss_bert = float("inf")
no_improve_bert = 0
```

```
def train_bert_epoch(model, loader, criterion, optimizer):
    model.train()

    total_loss, correct, n = 0.0, 0, 0

    for batch in loader:
        ids = batch["input_ids"].to(DEVICE)
        mask = batch["attention_mask"].to(DEVICE)
```



```

lbls = batch["label"].to(DEVICE)
optimizer.zero_grad()
logits = model(ids, mask)
loss = criterion(logits, lbls)
loss.backward()
optimizer.step()
total_loss += loss.item() * len(lbls)
correct += (logits.argmax(1) == lbls).sum().item()
n += len(lbls)
return total_loss / n, correct / n

```

```

@torch.no_grad()
def eval_bert(model, loader, criterion):
    model.eval()
    total_loss, correct, n = 0.0, 0, 0
    for batch in loader:
        ids = batch["input_ids"].to(DEVICE)
        mask = batch["attention_mask"].to(DEVICE)
        lbls = batch["label"].to(DEVICE)
        logits = model(ids, mask)
        total_loss += criterion(logits, lbls).item() * len(lbls)
        correct += (logits.argmax(1) == lbls).sum().item()
        n += len(lbls)
    return total_loss / n, correct / n

```

```

print(f"\nTraining DistilBertClassifier (max {BERT_EPOCHS} epochs,
patience={BERT_PATIENCE})")

print(f"{'Epoch':>5} {'tr_loss':>8} {'tr_acc':>7} {'vl_loss':>8}
{'vl_acc':>7} {'lr':>9}")
print("-" * 57)

```

```

for epoch in range(1, BERT_EPOCHS + 1):
    tr_loss, tr_acc = train_bert_epoch(bert_model, bert_train_loader, criterion_txt,
opt_bert)

    vl_loss, vl_acc = eval_bert(bert_model, bert_val_loader, criterion_txt)

    history_bert["train_loss"].append(tr_loss)
    history_bert["val_loss"].append(vl_loss)

```

```

history_bert["train_acc"].append(tr_acc)
history_bert["val_acc"].append(vl_acc)
sched_bert.step(vl_loss)
lr_now = opt_bert.param_groups[0]["lr"]
if vl_loss < best_val_loss_bert:
    best_val_loss_bert = vl_loss
    no_improve_bert = 0
    save_checkpoint(bert_model.state_dict(), "distilbert_best.pt")
    marker = " ✓"
else:
    no_improve_bert += 1
    marker = ""
    save_checkpoint(
        {"epoch": epoch, "model_state": bert_model.state_dict(),
         "optimizer_state": opt_bert.state_dict(), "val_loss": vl_loss,
         "history": history_bert},
        f"distilbert_ckpt_ep{epoch:02d}.pt",
    )
print(f"epoch:>5} {tr_loss:>8.4f} {tr_acc:>7.4f} {vl_loss:>8.4f}
      {vl_acc:>7.4f} {lr_now:>9.2e}{marker}")
if no_improve_bert >= BERT_PATIENCE:
    print(f"\nEarly stopping at epoch {epoch}.")
    break

print(f"\nBest val loss : {best_val_loss_bert:.4f}")
print("✓ 11.11 – DistilBERT head training complete.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 23, in <module>
NameError: name 'GRAPE_CLASSES' is not defined

```

Cell 154: ■ Markdown

11.12 — DistilBERT: test evaluation and 3-model comparison

Loads the best DistilBERT checkpoint and produces a final 3-model comparison table (LSTM Baseline / BiLSTM + Attention / DistilBERT) showing test accuracy, Macro F1, trainable parameter count, and training time.

Cell 155: ■ Code

[illegible]

```
axes[1].plot(_epochs_bert, history_bert["val_acc"], label="Val", lw=1.8)
axes[1].set_title("DistilBERT - Accuracy"); axes[1].set_xlabel("Epoch");
axes[1].legend()

plt.suptitle(f"11.12 - DistilBERT | Test Acc: {bert_test_acc*100:.2f}% Macro-F1: {bert_f1:.4f}", fontsize=12)

plt.tight_layout()

save_figure(fig, "distilbert_curves.png")

plt.show()
```

```
# ■■ Confusion matrix
```



```
bert_cm = confusion_matrix(all_labels_bert, all_preds_bert)

fig, ax = plt.subplots(figsize=(13, 11))

sns.heatmap(bert_cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=GRAPE_CLASSES, yticklabels=GRAPE_CLASSES, ax=ax, linewidths=0.3)

ax.set_xlabel("Predicted"); ax.set_ylabel("True")

ax.set_title(f"DistilBERT – Confusion Matrix (Test Acc: {bert_test_acc*100:.2f}%)")

plt.xticks(rotation=45, ha="right", fontsize=8); plt.yticks(fontsize=8)

plt.tight_layout()

save_figure(fig, "distilbert_cm.png")

plt.show()
```

[illegible]

[illegible]

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 2, in <module>
NameError: name 'bert_model' is not defined
```

Cell 156: ■ Markdown

Section 12 — TasteBiLSTM: Wine Review Taste Encoder

Trains a dedicated **TasteBiLSTM** model on wine reviews weakly labelled with 10 taste axes. The trained encoder produces a **512-dimensional taste embedding** for any free-text wine review — the

foundation of the flavor-cluster recommendation system in Section 14.

| Sub-section | Content |

|---|---|

| **12.1** | Taste-label dataset — weak labeling with 10 taste axes; `TasteDataset`` and `DataLoaders`` |

| **12.2** | TasteBiLSTM architecture — BiLSTM + Bahdanau attention, 10-dim taste output head |

| **12.3** | Training — Adam, ReduceLROnPlateau, early stopping; saves `tastebilstm_best.pt`` |

| **12.4** | Evaluation — per-axis accuracy, precision / recall / F1, learning curves |

| **12.5** | Encoder export — `taste_encoder.encode()`` verified on sample reviews |

Prerequisites: §4.1 (`df_train/val/test``) · §7.4 (`embedding_matrix`, `X_train/val/test``) · §11.6 (`BahdanauAttention``)

Cell 157: ■ Markdown

12.1 — Taste-Label Dataset

Weak labeling with 10 taste axes using keyword matching on lowercased review text:

| Axis | Friendly name | Sample seed words |

|---|---|---|

| ``body`` | soft | full-bodied, rich, opulent, velvety, dense |

| ``acidity`` | crispy | acid, crisp, bright, tart, zesty |

| ``tannin`` | bold | tannin, tannic, grippy, astringent, chewy |

| ``red_fruit`` | juicy | cherry, raspberry, strawberry, red currant |

| ``dark_fruit`` | deep | blackberry, plum, cassis, dark fruit, fig |

| ``earthy`` | earthy | earthy, soil, mushroom, tobacco, leather |

| ``sweet`` | sweet | sweet, honeyed, jammy, candied, luscious |

| ``oaky`` | smoky | oak, vanilla, toast, cedar, smoky |

| ``floral`` | delicate | floral, violet, rose, jasmine, blossom |

| ``mineral`` | mineral | mineral, minerality, stony, chalk, flinty |

Each review receives a **binary label vector** (dim = 10). A label fires if any seed word appears. Reviews where no axis fires are retained as "neutral" samples — the encoder learns that the absence of taste signals is also informative.

The token sequences ``X_train/val/test`` (built in §7.4) are reused directly — no re-tokenisation needed.

Cell 158: ■ Code

■■ 12.1 Taste-label dataset

```
# Technical axis names (used in code, model weights, cluster keys)
TASTE_AXES = ["body", "acidity", "tannin", "red_fruit", "dark_fruit",
               "earthy", "sweet", "oaky", "floral", "mineral"]
```

```
# User-friendly display names (used in printouts and cluster labels)

TASTE_AXIS_NAMES = {

    "body": "soft", # full, round, heavy, velvety
    "acidity": "crispy", # crisp, bright, zingy, sharp
    "tannin": "bold", # firm, chewy, drying, grippy
    "red_fruit": "juicy", # cherry, raspberry, strawberry
    "dark_fruit": "deep", # plum, blackberry, cassis, fig
    "earthy": "earthy", # leather, tobacco, soil, mushroom
    "sweet": "sweet", # jammy, soft, warm, generous
    "oaky": "smoky", # vanilla, toast, cedar, smoke
    "floral": "delicate", # violet, rose, jasmine, blossom
    "mineral": "mineral", # flinty, saline, chalky, volcanic
}
```

```
_TASTE_KW = {
    "body": ["full-bodied", "full bodied", "rich", "opulent",
             "lush", "concentrated", "powerful", "dense",
             "velvety", "creamy", "voluminous", "weighty"],
    "acidity": ["acid", "acidity", "crisp", "bright",
                "tart", "racy", "lively", "zesty",
                "sharp", "electric", "refreshing"],
    "tannin": ["tannin", "tannic", "grippy", "astringent",
               "firm tannin", "tight tannin", "chewy", "drying",
               "structured", "gripping"],
    "red_fruit": ["cherry", "raspberry", "strawberry", "red currant",
                  "cranberry", "red fruit", "pomegranate", "redcurrant"],
    "dark_fruit": ["blackberry", "plum", "cassis", "black currant",
                   "dark fruit", "black fruit", "blueberry", "fig",
                   "black cherry", "dark cherry", "boysenboy", "mulberry"],
```

```

"earthy": ["earthy", "earth", "soil", "mushroom",
"forest floor", "tobacco", "leather", "savory",
"herbal", "truffle", "barnyard"],
"sweet": ["sweet", "sweetness", "off-dry", "residual sugar",
"honeyed", "honey", "dessert", "luscious",
"syrupy", "jammy", "ripe", "candied",
"sugar", "semi-sweet"],
"oaky": ["oak", "oaky", "vanilla", "toast",
"toasty", "cedar", "smoky", "barrel",
"wood", "woody", "buttery", "spice",
"clove", "coconut", "caramel"],
"floral": ["floral", "flower", "violet", "rose",
"jasmine", "blossom", "lavender", "perfumed"],
"mineral": ["mineral", "minerality", "stony", "slate",
"chalk", "flinty", "wet stone", "limestone",
"graphite", "volcanic", "flint", "crushed rock",
"steely", "sea spray", "oyster shell"],
}

```

```
def _taste_label(description: str) -> list:
    """Binary taste labels via keyword matching on lowercased text."""
    d = description.lower()
    return [1.0 if any(kw in d for kw in _TASTE_KW[ax]) else 0.0
            for ax in TASTE_AXES]
```



```

taste_test_loader = DataLoader(taste_test_ds, batch_size=TASTE_BATCH,
                               shuffle=False,
                               num_workers=0, pin_memory=False)

print(f"\n{'Split':&8} {'Samples':&8} {'Batches':&8}")
print("-" * 28)
for _n, _ds, _ldr in [("train", taste_train_ds, taste_train_loader),
                      ("val", taste_val_ds, taste_val_loader),
                      ("test", taste_test_ds, taste_test_loader)]:
    print(f"{_n:&8} {len(_ds):&8} {len(_ldr):&8}")

print("\u2713 12.1 \u2014 Taste labels and DataLoaders ready.")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "&string&";", line 64, in &module&
NameError: name 'df_train' is not defined

```

Cell 159: ■ Markdown

12.2 — *TasteBiLSTM Architecture*

...

Input ($B \times L$) → Embedding (W2V 100-d, fine-tuned)
 → Dropout(0.4)
 → BiLSTM (100-d → 256-d × 2, 2 layers)
 → Bahdanau attention over all L hidden states → $B \times 512$
 → Dropout(0.4)
 → Linear(512 → 10) — one logit per taste axis
 ...

Identical structure to `BiLSTMAttention` (§11.6) except the output head predicts 10 continuous taste scores instead of 15 grape classes. The BahdanauAttention` module from §11.6 is reused directly.`

Training loss: `BCEWithLogitsLoss` — each axis is an independent binary prediction (a review can be simultaneously tannic, acidic, and rich).`

Encoder output (`encode()`): the 512-d attention context vector **before** the FC head — this is what §14 uses for flavor-cluster assignment.

```
| Hyperparameter | Value |
|---|---|
| `EMBED_DIM` | 100 (W2V, fine-tuned) |
| `TASTE_HIDDEN` | 256 (= `HIDDEN_DIM`) |
| `N_LAYERS` | 2 |
| `DROPOUT_RNN` | 0.4 |
| Encoder output dim |  $512 \text{ (} \text{`TASTE\_HIDDEN'} \times 2 \text{)}$  |
```

Cell 160: ■ Code

```
# ■■■ 12.2 TasteBiLSTM architecture  
#####  
  
# Reuses BahdanauAttention (defined in §11.6).  
  
# Output head: Linear(512 → 10) – one logit per taste axis.  
  
# encode() returns the 512-d context vector before the FC head.
```

```
N_TASTE_AXES = len(TASTE_AXES) # 10  
  
TASTE_HIDDEN = HIDDEN_DIM # 256 – same as §11
```

```
class TasteBiLSTM(nn.Module):  
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_taste=N_TASTE_AXES,  
n_layers=2, dropout=0.4, embedding_matrix=None):  
        super().__init__()  
  
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)  
  
        if embedding_matrix is not None:  
            self.embedding.weight.data.copy_(  
                torch.tensor(embedding_matrix, dtype=torch.float32))  
  
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=n_layers,  
batch_first=True, bidirectional=True,  
dropout=dropout if n_layers > 1 else 0.0)  
  
        self.attention = BahdanauAttention(hidden_dim)  
  
        self.drop = nn.Dropout(dropout)  
  
        self.fc = nn.Linear(hidden_dim * 2, n_taste)
```

```
def forward(self, x, lengths):
```

```

emb = self.drop(self.embedding(x))
packed = pack_padded_sequence(emb, lengths.cpu(),
batch_first=True, enforce_sorted=False)
output, _ = self.lstm(packed)
output, _ = pad_packed_sequence(output, batch_first=True, total_length=x.shape[1])
context, _ = self.attention(output, (x != 0))
return self.fc(self.drop(context)) # (B, 10)

```

```

@torch.no_grad()
def encode(self, x, lengths):
    """Return 512-d taste context vector – used by §14 (flavor clusters)."""
    emb = self.embedding(x)
    packed = pack_padded_sequence(emb, lengths.cpu(),
batch_first=True, enforce_sorted=False)
    output, _ = self.lstm(packed)
    output, _ = pad_packed_sequence(output, batch_first=True, total_length=x.shape[1])
    context, _ = self.attention(output, (x != 0))
    return context # (B, 512)

```

```

taste_bilstm = TasteBiLSTM(
vocab_size=VOCAB_SIZE, embed_dim=EMBED_DIM, hidden_dim=TASTE_HIDDEN,
n_taste=N_TASTE_AXES, n_layers=N_LAYERS, dropout=DROPOUT_RNN,
embedding_matrix=embedding_matrix,
).to(DEVICE)

```

```

_total_taste = sum(p.numel() for p in taste_bilstm.parameters())
print(f"TasteBiLSTM – total params : {_total_taste:,}")

```

```

_xb_t2, _lb_t2, _yb_t2 = next(iter(taste_train_loader))
_out_taste = taste_bilstm(_xb_t2.to(DEVICE), _lb_t2.to(DEVICE))
assert _out_taste.shape == (TASTE_BATCH, N_TASTE_AXES), \
f"Expected ({TASTE_BATCH}, {N_TASTE_AXES}), got {_out_taste.shape}"
print(f"Forward pass : OK → {_out_taste.shape} (B × {N_TASTE_AXES} taste axes)")

```

```

_enc_taste = taste_bilstm.encode(_xb_t2.to(DEVICE), _lb_t2.to(DEVICE))
assert _enc_taste.shape == (TASTE_BATCH, TASTE_HIDDEN * 2)
print(f"encode() : OK → {_enc_taste.shape} (B × {TASTE_HIDDEN * 2}-d taste embedding)")

```

```
print("✓ 12.2 – TasteBiLSTM ready.")
```

Error:

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"

```
exec('\n'.join(lines[:-1]), glb)
```

~~~~~

File "&lt;string&gt;", line 47, in &lt;module&gt;

**NameError: name 'VOCAB\_SIZE' is not defined**

## Cell 161: ■ Markdown

### 12.3 — Training TasteBiLSTM

Same protocol as §11.4 / §11.7:

Setting	Value
---------	-------

|-|-|

| Optimiser | Adam, lr = 1e-3, weight\_decay = 1e-5 |

| LR scheduler | ReduceLROnPlateau  $\times 0.5$ , patience 2 |

| Max epochs | 30 |

| Early stopping | patience = 4 |

| Gradient clipping | max-norm 5 |

| Loss | `BCEWithLogitsLoss` (multi-label) |

Per-epoch checkpoints saved as `tastebilstm\_ckpt\_epXX.pt` to allow resume after disconnection. Best model (lowest val loss) saved to `tastebilstm\_best.pt`.

## Cell 162: ■ Code

## # 12.3 Train TasteBiLSTM

Response	Percentage
Doing a good job	55%

```
TASTE_EPOCHS = 30
```

TASTE\_PATIENCE = 4

```
# ■■ Class-imbalance correction via pos_weight
```

```
# floral=2.4%, mineral=1.7% are severely under-represented.
```

```
# pos_weight[i] = neg_count / pos_count (capped at 20 to prevent loss instability).
```

```
_pos_counts = taste_train.sum(axis=0).clip(min=1) # (10,) float32
```

```

_neg_counts = len(taste_train) - _pos_counts
_pw_raw = _neg_counts / _pos_counts
_pw_capped = np.minimum(_pw_raw, 20.0)
pos_weight_t = torch.tensor(_pw_capped, dtype=torch.float32, device=DEVICE)
print(f"{'Axis':&lt;16} {'pos%':&gt;5} {'pos_weight':&gt;10}")
print("-" * 36)
for ax, pw, pct in zip(TASTE_AXES, _pw_capped, _pos_counts / len(taste_train) *
100):
    print(f"{'ax':&lt;16} {'pct':&gt;5.1f} {'pw':&gt;10.2f}")
print()

```

```

criterion_taste = nn.BCEWithLogitsLoss(pos_weight=pos_weight_t)
opt_taste = Adam(taste_bilstm.parameters(), lr=1e-3, weight_decay=1e-5)
sched_taste = ReduceLROnPlateau(opt_taste, factor=0.5, patience=2)

```

```

history_taste = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
best_val_loss_taste = float("inf")
no_improve_taste = 0

```

```

def _train_taste_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct, n = 0.0, 0, 0
    for seqs, lengths, labels in loader:
        seqs, lengths, labels = seqs.to(DEVICE), lengths.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        logits = model(seqs, lengths)
        loss = criterion(logits, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        optimizer.step()
        preds = (logits.sigmoid() >= 0.5).float()
        correct += (preds == labels).all(dim=1).sum().item()
        total_loss += loss.item() * len(seqs)
        n += len(seqs)
    return total_loss / n, correct / n

```

```

def _eval_taste(model, loader, criterion):

```

```

model.eval()
total_loss, correct, n = 0.0, 0, 0
with torch.no_grad():
    for seqs, lengths, labels in loader:
        seqs, lengths, labels = seqs.to(DEVICE), lengths.to(DEVICE), labels.to(DEVICE)
        logits = model(seqs, lengths)
        loss = criterion(logits, labels)
        preds = (logits.sigmoid() >= 0.5).float()
        correct += (preds == labels).all(dim=1).sum().item()
        total_loss += loss.item() * len(seqs)
    n += len(seqs)
return total_loss / n, correct / n

```

```

print(f"Training TasteBiLSTM (max {TASTE_EPOCHS} epochs,
patience={TASTE_PATIENCE})")
print(f"{'Epoch':>5} {'tr_loss':>8} {'tr_acc':>7} {'vl_loss':>8}
{'vl_acc':>7} {'lr':>9}")
print("-" * 57)

```

```

for epoch in range(1, TASTE_EPOCHS + 1):
    tr_loss, tr_acc = _train_taste_epoch(taste_bilstm, taste_train_loader,
criterion_taste, opt_taste)
    vl_loss, vl_acc = _eval_taste(taste_bilstm, taste_val_loader, criterion_taste)
    history_taste["train_loss"].append(tr_loss)
    history_taste["val_loss"].append(vl_loss)
    history_taste["train_acc"].append(tr_acc)
    history_taste["val_acc"].append(vl_acc)
    sched_taste.step(vl_loss)
    lr_now = opt_taste.param_groups[0]["lr"]
    if vl_loss < best_val_loss_taste:
        best_val_loss_taste = vl_loss
        no_improve_taste = 0
        save_checkpoint(taste_bilstm.state_dict(), "tastebilstm_best.pt")
        marker = " ✓"
    else:
        no_improve_taste += 1

```

```

marker = ""
save_checkpoint(
{"epoch": epoch, "model_state": taste_bilstm.state_dict(),
"optimizer_state": opt_taste.state_dict(), "val_loss": vl_loss,
"history": history_taste},
f"tastebilstm_ckpt_ep{epoch:02d}.pt"
)

print(f"{epoch:>5} {tr_loss:>8.4f} {tr_acc:>7.4f} {vl_loss:>8.4f}
{vl_acc:>7.4f} {lr_now:>9.2e}{marker}")

if no_improve_taste >= TASTE_PATIENCE:
print(f"\nEarly stopping at epoch {epoch}.")
break

print(f"\nBest val loss : {best_val_loss_taste:.4f} → {WEIGHTS /
'tastebilstm_best.pt'}")

print("✓ 12.3 – TasteBiLSTM training complete.")

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 8, in <module>
NameError: name 'taste_train' is not defined

```

### Cell 163: ■ Markdown

#### 12.4 — Evaluation

Loads the best checkpoint and runs inference on the frozen test set. Each of the 10 taste axes is treated as an independent binary prediction.

Metric	Description
--------	-------------

---	---
-----	-----

Accuracy	Fraction of reviews correctly classified for this axis
----------	--------------------------------------------------------

Precision / Recall / F1	Standard binary classification metrics per axis
-------------------------	-------------------------------------------------

Exact-match accuracy	All 10 axes correct simultaneously (strictest metric)
----------------------	-------------------------------------------------------

### Cell 164: ■ Code



```
12.4 TasteBiLSTM – test evaluation
from sklearn.metrics import precision_score, recall_score

taste_bilstm.load_state_dict(load_checkpoint("tastebilstm_best.pt"))
taste_bilstm.eval()

all_probs_taste, all_labels_taste = [], []
with torch.no_grad():
 for seqs, lengths, labels in taste_test_loader:
 logits = taste_bilstm(seqs.to(DEVICE), lengths.to(DEVICE))
 all_probs_taste.append(logits.sigmoid().cpu())
 all_labels_taste.append(labels)

all_probs_taste = torch.cat(all_probs_taste, dim=0)
all_labels_taste = torch.cat(all_labels_taste, dim=0)

Fix 3 – per-axis thresholds.
(historical: citrus_fruity had recall 0.36 at 0.50 in v1 with 8 axes) → lowered
to 0.35 to catch more positives.
mineral had precision 0.65 at 0.50 (too many false positives) → raised to 0.55.
_AXIS_THRESHOLDS = torch.tensor([
 0.50, # body
 0.50, # acidity
 0.50, # tannin
 0.50, # red_fruit
 0.50, # dark_fruit
 0.50, # earthy
 0.50, # sweet
 0.50, # oaky
 0.50, # floral
 0.55, # mineral ← raised: over-predicted in v1 (precision 0.65)
])

all_preds_taste = (all_probs_taste >= _AXIS_THRESHOLDS).float()

Per-axis accuracy, precision, recall, F1
```

[illegible]

**Error:**

```
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
 exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 4, in <module>
NameError: name 'taste_bilstm' is not defined. Did you mean: 'TasteBiLSTM'?
```

## Cell 165: ■ Markdown

## 12.5 — Encoder Export

Exports ``taste_encoder`` (alias for ``taste_bilstm``) as the global handle used by **Section 14 — Flavor-Cluster Recommendation**.

``taste_encoder.encode(x, lengths)`` returns a ``Tensor(B, 512)`` — the Bahdanau attention context vector computed **before** the FC head. It requires no gradient and runs in eval mode. Section 14 calls this on the full wine corpus to build flavor-cluster centroids.

## Cell 166: ■ Code



```
| Model | Weight file | Trained in |
|---|---|---|
| CNN from scratch | `cnn_scratch_best.pt` | §8 |
| ResNet-50 (transfer learning) | `cnn_resnet50_best.pt` | §9 |
| LSTM Baseline | `lstm_best.pt` | §11 |
| BiLSTM + Attention | `bilstm_best.pt` | §11 |
| DistilBERT (frozen head) | `distilbert_best.pt` | §11 |
| TasteBiLSTM (taste encoder) | `tastebilstm_best.pt` | §12 |
```

## Cell 168: ■ Code

```
# 13 Save ALL results
# glob, shutil, os, torch, np already imported in §1.2.
# save_checkpoint / load_checkpoint / save_figure defined in §1.3.
import glob, pickle
```

```

},
# §11 – LSTM Baseline
"lstm": {
  "test_acc": globals().get("lstm_test_acc"),
  "macro_f1": globals().get("lstm_f1"),
  "history": globals().get("history_lstm"),
  "confusion_matrix":globals().get("lstm_cm"),
  "weights_file": "lstm_best.pt",
},
# §11 – BiLSTM + Attention
"bilstm": {
  "test_acc": globals().get("bilstm_test_acc"),
  "macro_f1": globals().get("bilstm_f1"),
  "history": globals().get("history_bilstm"),
  "confusion_matrix":globals().get("bilstm_cm"),
  "weights_file": "bilstm_best.pt",
},
# §11 – DistilBERT (frozen head)
"distilbert": {
  "test_acc": globals().get("bert_test_acc"),
  "macro_f1": globals().get("bert_f1"),
  "history": globals().get("history_bert"),
  "confusion_matrix":globals().get("bert_cm"),
  "weights_file": "distilbert_best.pt",
},
# §12 – TasteBiLSTM (taste encoder)
"tastebilstm": {
  "macro_f1": globals().get("macro_taste_f1"),
  "exact_match": globals().get("exact_match"),
  "axis_f1s": globals().get("taste_axis_f1s"),
  "history": globals().get("history_taste"),
  "weights_file": "tastebilstm_best.pt",
},
# Shared text artefacts

```

```

"text": {
  "VOCAB_SIZE": globals().get("VOCAB_SIZE"),
  "MAX_SEQ_LEN": globals().get("MAX_SEQ_LEN"),
  "GRAPE_CLASSES": globals().get("GRAPE_CLASSES"),
  "GRAPE_TO_IDX": globals().get("GRAPE_TO_IDX"),
  "TASTE_AXES": globals().get("TASTE_AXES"),
  "embedding_matrix":globals().get("embedding_matrix"),
},
# Dataset provenance
"splits": {
  "SEED": SEED,
  "n_train_text": len(df_train) if "df_train" in globals() else None,
  "n_val_text": len(df_val) if "df_val" in globals() else None,
  "n_test_text": len(df_test) if "df_test" in globals() else None,
  "n_train_img": len(_idx_train) if "_idx_train" in globals() else None,
  "n_val_img": len(_idx_val) if "_idx_val" in globals() else None,
  "n_test_img": len(_idx_test) if "_idx_test" in globals() else None,
},
}

```

```

# ■■ Sanitise: convert tensors / arrays → plain Python; drop unpicklable objects
def _sanitize(obj, _depth=0):
  if _depth > 12:
    return None
  if obj is None or isinstance(obj, (bool, int, float, str)):
    return obj
  if isinstance(obj, torch.Tensor):
    return obj.detach().cpu().tolist()
  if isinstance(obj, np.ndarray):
    return obj.tolist()
  if isinstance(obj, dict):
    return {k: _sanitize(v, _depth + 1) for k, v in obj.items()}
  if isinstance(obj, (list, tuple)):
    return [_sanitize(v, _depth + 1) for v in obj]
  try:

```

```

pickle.dumps(obj)
return obj
except Exception:
return None # silently drop modules, lambdas, etc.

```

```

safe_snapshot = _sanitize(snapshot)

```

```

# ■■ 1. Save snapshot

```

```

snap_path = LOCAL_WEIGHTS / "results_snapshot.pt"
torch.save(safe_snapshot, snap_path)
print(f"✓ results_snapshot.pt ({snap_path.stat().st_size / 1e6:.1f} MB)")
if IN_COLAB:
shutil.copy2(snap_path, os.path.join(WEIGHTS_DIR, "results_snapshot.pt"))
print(f" → synced to Drive: {WEIGHTS_DIR}")

```

```

# ■■ 2. Sync weight files

```

```

WEIGHT_FILES = [
"cnn_scratch_best.pt",
"cnn_resnet50_best.pt",
"lstm_best.pt",
"bilstm_best.pt",
"distilbert_best.pt",
"tastebilstm_best.pt",
]

```

```

print("\nWeight files:")
for wf in WEIGHT_FILES:
src = LOCAL_WEIGHTS / wf
if src.exists():
mb = src.stat().st_size / 1e6
if IN_COLAB:
shutil.copy2(src, os.path.join(WEIGHTS_DIR, wf))
print(f" ✓ {wf:<35} {mb:.1f} MB → Drive")
else:
print(f" ✓ {wf:<35} {mb:.1f} MB")

```





```

("DistilBERT (frozen head)", snapshot["distilbert"]["test_acc"],
snapshot["distilbert"]["macro_f1"], "distilbert_best.pt"),

("TasteBiLSTM (encoder)", None, snapshot["tastebilstm"]["macro_f1"],
"tastebilstm_best.pt"),

]

for name, acc, f1, wf in _rows:

print(f" {name:<30} {_fmt(acc, pct=True):>10} {_fmt(f1):>10} {wf}")

print("=" * 75)

print(f"\n TasteBiLSTM exact-match accuracy :
{_fmt(snapshot['tastebilstm']['exact_match'])}")

print("\n✓ Section 13 complete – all results saved.")

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 67, in <module>
NameError: name 'SEED' is not defined

```

#### Cell 169: ■ Markdown

### Section 14 — Flavor-Cluster Recommendation

> **This is the core ML product of the project.** Everything before this section trained models. This section *uses* those models to solve a real recommendation problem — matching food to wine via a shared taste embedding space, with no hardcoded rules and full confidence transparency.

The previous sections produced two trained encoders:

- **BiLSTMAttention** (§11) — encodes wine reviews into 512-d grape-space vectors
- **TasteBiLSTM** (§12) — encodes wine reviews into 512-d taste-space vectors

This section uses the **TasteBiLSTM encoder** to cluster the entire wine corpus into 9 flavor clusters via BisectingKMeans, then builds a recommendation engine that maps any food (from the 101-food Food-101 dataset) to three wine suggestions — each personalized to the specific food.

#### ***ML techniques in this section***

| Technique | Purpose | Why it matters |

|---|---|---|

| **BisectingKMeans** (K=9) | Discover flavor neighborhoods in taste-vector space | More balanced cluster sizes than standard K-means; silhouette-justified K |

| **TF-IDF + taste-axis naming** | Auto-label clusters with interpretable names | Explainability — user sees "Earthy & Smoky" not "Cluster 5" |

| **Cosine similarity matching** | Map food descriptions to nearest cluster centroid | Cross-modal retrieval: food text → wine cluster |

| **Food-specific wine selection** | Pool of 10 wines per cluster; pick by cosine to food vector | Two foods in same cluster get different wines |

| **Taste-aware Bold Move** | distance × (1 + secondary-flavor affinity) | Contrast that's still drinkable — not random |

| **Confidence scoring** | Cosine similarity as match-strength indicator | User knows if recommendation is strong (0.85) or exploratory (0.55) |

## Food flavor descriptions

Source: `food\_flavor\_description\_v2.json` (101 entries, 3 keys each):

| JSON key | Card tier | Icon | Meaning |

|---|---|---|---|

| `classic` | SAFE BET | ■ | Dominant flavor identity — closest taste match |

| `safe\_bet` | HIDDEN GEM | ■ | Compatible but surprising — pairs unexpectedly well |

| `contrast` | \*(not used)\* | — | Bold Move is taste-aware geometric, not description-based |

**Prerequisites:** §3.4 (`food\_flavor\_table\_v2`), §7 (`VOCAB`, `MAX\_SEQ\_LEN`, `X\_train`), §12 (`taste\_encoder`)

## Sub-sections

| # | Content |

|---|---|

| 14.1 | Encode wine corpus → `train\_taste\_vecs` (N × 512) |

| 14.2 | Silhouette curve K=6–20, fit BisectingKMeans K=9 |

| 14.3 | Auto-name clusters via TF-IDF + taste-axis naming |

| 14.4 | Select top-10 wine pool per cluster (with taste vectors) |

| 14.5 | `recommend\_v2()` — food-specific wine selection + confidence scoring |

| 14.6 | Full 101-food evaluation + diversity stats |

| 14.7 | Business integration: 20-food recommendation table |

| 14.8 | Model performance comparison chart |

## Cell 170: ■ Markdown

## 14.1 — Encode Wine Corpus

Encode every training wine review through ``taste_encoder.encode()`` in batches of 256.

Input: `X_train` — already tokenized and zero-padded in §7 (shape  $N_{\text{train}} \times \text{MAX\_SEQ\_LEN}$ ).

Output: `train\_taste\_vecs` ( $N_{\text{train}} \times 512$ ), L2-normalised for cosine-correct K-means.

**Cell 171: ■ Code**



```
_km = KMeans(n_clusters=k, random_state=42, n_init=5, max_iter=200)
_lb = _km.fit_predict(vecs_sil)
s = silhouette_score(vecs_sil, _lb, metric="cosine",
sample_size=2_000, random_state=42)
sil_scores.append(s)
print(f" K={k:2d} silhouette={s:.4f}")

fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(list(K_RANGE), sil_scores, marker="o", linewidth=1.5, color="#4C72B0")
ax.axvline(K_CLUSTERS, color="#C44E52", linestyle="--", label=f"Selected
K={K_CLUSTERS}")
ax.set_xlabel("Number of clusters (K)")
ax.set_ylabel("Silhouette score (cosine)")
ax.set_title("K-means silhouette score – TasteBiLSTM embedding space")
ax.legend()
ax.grid(True, alpha=0.3)
fig.tight_layout()
save_figure(fig, "kmeans_silhouette.png")
plt.close(fig)
```

```
print(f"\n✓ Section 14.2 complete – BisectingKMeans K={K_CLUSTERS} fitted;
centroids shape {centroids.shape}.")
```

```

k_max = 12
idx_max = k_values.index(k_max)
sil_max = sil_scores[idx_max]
ax.plot(k_max, sil_max, marker="^", markersize=10, color="#4C72B0", alpha=0.6,
label=f"Peak: K={k_max}", zorder=4)

Mark the "cliff" at K=13 where silhouette drops
ax.axvline(13, color="gray", linestyle=":", alpha=0.5, linewidth=1.5)

Formatting
ax.set_xlabel("Number of Clusters (K)", fontsize=12, fontweight="bold")
ax.set_ylabel("Silhouette Score (cosine metric)", fontsize=12, fontweight="bold")
ax.set_title("Section 14.2 – Optimal Cluster Selection: Silhouette Analysis",
fontsize=13, fontweight="bold")
ax.set_xticks(range(6, 21))
ax.grid(True, alpha=0.2, linestyle="--")
ax.legend(fontsize=10, loc="best")
ax.set_ylim(0.3, 0.51)

Add annotations
ax.annotate(f"K=9: {sil_chosen:.4f}\n(Business choice:\n10 taste axes – 1
garbage)",
xy=(k_chosen, sil_chosen), xytext=(k_chosen-1.5, sil_chosen+0.035),
fontsize=9, ha="right",
bbox=dict(boxstyle="round,pad=0.5", facecolor="#fff3e0", alpha=0.8,
edgecolor="#dd8452"),
arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=0.3",
color="#dd8452", lw=1.5))

ax.annotate(f"K=12: {sil_max:.4f}\n(Math optimum)",
xy=(k_max, sil_max), xytext=(k_max+1.2, sil_max+0.025),
fontsize=9, ha="left",
bbox=dict(boxstyle="round,pad=0.5", facecolor="#e3f2fd", alpha=0.8,
edgecolor="#4C72B0"),
arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=-0.3",
color="#4C72B0", lw=1.5))

ax.annotate("Diversity cliff:\nhigh-K clusters\nfragment the space",
xy=(13, 0.369), xytext=(14.8, 0.38),

```









```

for k in _cluster_order:
 mean_scores = _cluster_means[k]
 top_score = mean_scores.max().item()

Garbage / balanced cluster: no dominant axis
if top_score < 0.05:
 cluster_names[k] = "balanced"
 continue

ranked_axes = mean_scores.argsort(descending=True).tolist()

Try combinations of top axes until we find a unique name
name = None
for i in range(len(ranked_axes)):
 ax1 = TASTE_AXES[ranked_axes[i]]
 s1 = mean_scores[ranked_axes[i]].item()
 name1 = TASTE_AXIS_NAMES[ax1]

Single-axis name (if dominant)
s2 = mean_scores[ranked_axes[i+1]].item() if i+1 < len(ranked_axes) else 0
if s1 > 0 and (s2 == 0 or s2 / s1 < 0.5):
 candidate = name1
 if candidate not in _used_names:
 name = candidate
 break

taken – fall through to two-axis

Two-axis name: pair with next available axis not already in a conflict
for j in range(i+1, len(ranked_axes)):
 ax2 = TASTE_AXES[ranked_axes[j]]
 candidate = f"{TASTE_AXIS_NAMES[ax1]} & {TASTE_AXIS_NAMES[ax2]}"
 if candidate not in _used_names:
 name = candidate
 break

if name:
 break

cluster_names[k] = name if name else f"Cluster {k}"

```



```

earlier cluster it is skipped for all subsequent ones.
#
Output:
cluster_wines[k] – single best wine dict (backward compat)
cluster_wine_pool[k] – list of up to 10 unique wines with their taste vectors
each: {"wine_label", "review_text", "rating_pct", "vec"}
#
In §14.5, recommend_v2() picks the pool wine whose taste vector is closest
to the food's encoded flavor description → different foods in the same cluster
get different bottle recommendations.

```

```

TOP_N_WINES = 10

```

```

cluster_wines = {} # k → best wine dict (for backward compat)
cluster_wine_pool = {} # k → list of up to TOP_N_WINES dicts (food-specific
selection)
used_labels = set() # global deduplication across clusters

```

```

for k in range(K_CLUSTERS):
 mask = cluster_labels == k
 sub_idx = np.where(mask)[0]
 sub_df = df_train.iloc[sub_idx].copy()

 # Top 20 % threshold (at least 1 review)
 thresh = sub_df["rating_pct"].quantile(0.80)
 top_df = sub_df[sub_df["rating_pct"] >= thresh]
 if top_df.empty:
 top_df = sub_df

```

```

Rank all top-rated candidates by similarity to centroid (best first)
top_pos = np.array([i for i, row_idx in zip(sub_idx, sub_df.index)
if row_idx in top_df.index])
v_top = train_taste_vecs[top_pos] # (M, 512)
sims = v_top @ centroids[k] # (M,)
ranked = top_pos[np.argsort(-sims)] # descending similarity

```

```

Collect up to TOP_N_WINES unique wines
pool = []

```

```

for pos in ranked:
 lbl = (str(df_train.iloc[pos].get("wine_label", "")).strip()
 or str(df_train.iloc[pos].get("grape_class", "-")))
 if lbl in used_labels:
 continue
 row = df_train.iloc[pos]
 pool.append({
 "wine_label" : lbl,
 "review_text" : str(row["review_text"][:180]),
 "rating_pct" : int(row["rating_pct"]),
 "vec" : train_taste_vecs[pos], # (512,) L2-normed
 })
 used_labels.add(lbl)
 if len(pool) >= TOP_N_WINES:
 break

Fallback: if no unique labels found, use first ranked regardless
if not pool:
 pos = ranked[0]
 row = df_train.iloc[pos]
 lbl = (str(row.get("wine_label", "")).strip()
 or str(row.get("grape_class", "-")))
 pool.append({
 "wine_label" : lbl,
 "review_text" : str(row["review_text"][:180]),
 "rating_pct" : int(row["rating_pct"]),
 "vec" : train_taste_vecs[pos],
 })

cluster_wine_pool[k] = pool
Backward compat: cluster_wines[k] is the #1 pick (without vec key)
cluster_wines[k] = {key: pool[0][key] for key in ("wine_label", "review_text",
"rating_pct")}

print("Representative wines per cluster (top-5 pool):")
print("■" * 80)
for k in range(K_CLUSTERS):

```

```

pool = cluster_wine_pool[k]
print(f" {k:2d} {cluster_names[k]:<32} pool size={len(pool)}")
for i, w in enumerate(pool):
marker = " ■ primary" if i == 0 else ""
print(f" {i+1}. {w['wine_label']:<36} {w['rating_pct']}%{marker}")

Sanity-check
alllbls = [cluster_wines[k]["wine_label"] for k in range(K_CLUSTERS)]
dups = [lbl for lbl in set(alllbls) if alllbls.count(lbl) > 1]
if dups:
print(f"\n■ Duplicate labels (fallback triggered): {dups}")

else:

print("\n✓ All primary wines are unique across clusters.")

print(f"✓ Section 14.4 complete – cluster_wine_pool populated ({TOP_N_WINES} wines
× {K_CLUSTERS} clusters).")

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "&lt;string&gt;", line 22, in &lt;module&gt;
NameError: name 'cluster_labels' is not defined. Did you mean: 'cluster_wines'?

```

#### Cell 179: ■ Markdown

### 14.5 — `recommend_v2()` + `print_card()`

The recommendation pipeline:

1. Look up the food in `food_flavor_table_v2`` → read the `classic`` description (→ SAFE BET) and the `safe_bet`` description (→ HIDDEN GEM) — both are full text strings from `food_flavor_description_v2.json``
2. Encode each description through `taste_encoder`` using the \$7 tokeniser (`_tokenise_14` / _encode_text``)
3. Find the nearest cluster centroid for each encoded description
4. BOLD MOVE is selected geometrically: the centroid with maximum Euclidean distance from the SAFE BET centroid — no food description used
5. Return three wine picks (one per tier) with card-ready metadata



The ``contrast`` key from ``food_flavor_description_v2.json`` is intentionally not used here — the geometric BOLD MOVE is more reliably extreme and avoids vocabulary drift.

Cell 180: ■ Code

```
# 14.5 recommend_v2() + print_card()
import string as _str_mod
from rich.console import Console
from rich.panel import Panel
from rich.text import Text

_console = Console(width=55)

_punct_14 = str.maketrans("", "", _str_mod.punctuation)

def _tokenise_14(text: str):
    """Match $7 tokeniser: lowercase, strip punctuation, whitespace split."""
    return str(text).lower().translate(_punct_14).split()

def _encode_text(text: str) -> np.ndarray:
    """Encode one text string -> L2-normalised (512,) numpy array via taste_encoder."""
    toks = _tokenise_14(text)
    ids = [VOCAB.get(w, 1) for w in toks[:MAX_SEQ_LEN]]
    if not ids:
        ids = [1]
    ids += [0] * (MAX_SEQ_LEN - len(ids))
    seq = torch.tensor([ids], dtype=torch.long).to(DEVICE)
    lens = torch.tensor([(seq != 0).sum().item()]).clamp(min=1)
    taste_encoder.eval()
    with torch.no_grad():
        vec = taste_encoder.encode(seq, lens).cpu().numpy()[0]
    norm = np.linalg.norm(vec)
    return vec / norm if norm > 0 else vec

def _nearest_cluster(vec: np.ndarray) -> int:
    """Return index of the cluster centroid with highest cosine similarity."""
    return int(np.argmax(centroids @ vec))
```

```
def recommend_v2(food_name: str) -> list:
```

```
    """
```

Return three wine recommendations for the given food.

Each recommendation includes:

- cluster assignment (which flavor neighborhood)
- food-specific wine (cosine match from 10-wine pool)
- confidence score (0-1, how strong the match is)

Tiers:

SAFE BET – nearest centroid to food's "classic" description

HIDDEN GEM – nearest centroid to food's "safe\_bet" (surprising) description

BOLD MOVE – taste-aware contrast: distance × (1 + secondary affinity)

ensures the wine is BOTH contrasting AND still drinkable

Tie-breaking: if two tiers resolve to the same cluster, the lower-priority tier shifts to its next-best candidate.

```
    """
```

```
    entry = food_flavor_table_v2.get(food_name, {})
```

```
    if not entry:
```

```
        raise KeyError(f"'{food_name}' not in food_flavor_table_v2")
```

```
    safe_desc = entry.get("classic", "balanced complex")
```

```
    hidden_desc = entry.get("safe_bet", "surprising compatible")
```

```
    safe_vec = _encode_text(safe_desc)
```

```
    hid_vec = _encode_text(hidden_desc)
```

```
    # ■■ Cluster assignment with confidence scores
```

```
    """
```

```
    safe_sims_all = centroids @ safe_vec # (K,) cosine similarities
```

```
    hid_sims_all = centroids @ hid_vec
```

```
    safe_k = int(np.argmax(safe_sims_all))
```

```
    hidden_k = int(np.argmax(hid_sims_all))
```

```
    safe_conf = float(safe_sims_all[safe_k]) # how strong the match is
```

```
    hidden_conf = float(hid_sims_all[hidden_k])
```





```
"mineral": "crisp and mineral",
}
```

```
def _cluster_adj(cluster_name: str) -> str:
    """'juicy & smoky' -> 'juicy' | 'earthy' -> 'earthy'"""
    first = cluster_name.split("&")[0].strip()
    if first.lower().startswith("the "):
        first = first[4:]
    return first.lower()
```

```
def _food_feel(safe_bet_cluster_name: str) -> str:
    """
    Derive how the food tastes from the SAFE BET cluster's dominant axis.
    Always returns a taste phrase, never a cooking method or ingredient.
    e.g. safe bet cluster "earthy & smoky" -> "savory and earthy"
    """
    adj = _cluster_adj(safe_bet_cluster_name)
    return _AXIS_FOOD_FEEL.get(adj, "rich and complex")
```

```
def _clip(text: str, max_chars: int = 150) -> str:
    """Truncate at a word boundary; never cut mid-word."""
    if len(text) <= max_chars:
        return text
    return text[:max_chars].rsplit(" ", 1)[0].rstrip(".", ";: ") + "..."
```

```
# Intent phrase -- the one phrase that distinguishes the three tiers
_INTENT = {
    "SAFE BET": "matches it",
    "HIDDEN GEM": "surprises you",
    "BOLD MOVE": "goes against it",
}
```

```
# Border colour per tier
_TIER_STYLE = {
    "SAFE BET": "green",
    "HIDDEN GEM": "blue",
    "BOLD MOVE": "red",
}
```

```
}
```

```
def _build_tier_text(r: dict, display_name: str, feel: str) -> Text:
```

```
    """Compose the rich Text block for one tier panel."""
```

```
    intent = _INTENT[r["tier"]]
```

```
    adj = _cluster_adj(r["name"])
```

```
    kws = " \u00b7 ".join(r["keywords"])
```

```
    snippet = _clip(r["snippet"])
```

```
    conf = r.get("confidence", 0)
```

```
# Confidence bar: ██████████ 73%
```

```
bar_full = int(conf * 10)
```

```
bar_empty = 10 - bar_full
```

```
conf_bar = "\u2588" * bar_full + "\u2591" * bar_empty + f" {conf*100:.0f}%"
```

```
t = Text()
```

```
t.append(f"{r['icon']} {r['tier']}\n", style="bold")
```

```
t.append(f"Match strength: {conf_bar}\n\n", style="dim")
```

```
t.append(f"As your {display_name} is {feel},\n")
```

```
t.append("we believe you need a wine that\n")
```

```
t.append(f"{intent}", style="bold")
```

```
t.append(" \u2014 something ")
```

```
t.append(f"{adj}\n", style="bold italic")
```

```
t.append("that plays on\n")
```

```
t.append(f"{kws}\n", style="bold cyan")
```

```
t.append("\n")
```

```
t.append("Wine drinkers suggest:\n", style="dim")
```

```
t.append(f"{r['wine']} \u2b50 {r['rating']}\n", style="bold green")
```

```
t.append("\n")
```

```
t.append("Wine drinkers say:\n", style="dim")
```

```
t.append(f'"{snippet}"', style="italic")
```

```
return t
```

```
def print_card(food_name: str, recs: list):
```

```
    """Print a story-driven recommendation card for one food using rich."""
```

```
    display_name = food_name.replace("_", " ").title()
```



Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
  exec(source, glb)
  ~~~~^
File "<string>", line 248, in <module>
File "<string>", line 55, in recommend_v2
NameError: name 'food_flavor_table_v2' is not defined
```

## Cell 181: ■ Markdown

## 14.6 — Full 101-Food Evaluation

Run ``recommend_v2()`` for all 101 foods in ``food_flavor_table_v2``. Report cluster diversity (unique clusters used per tier) and plot the cluster assignment distribution. Print 5 sample cards to verify output quality.

## Cell 182: ■ Code



[illegible]



```
"grilled_salmon", "chicken_curry", "french_onion_soup", "macarons",
"bibimbap", "ceviche", "ramen", "caprese_salad", "foie_gras", "churros",
]
```

```
_rows = []

for food in _TABLE_FOODS:
 if food not in results_all:
 continue

 recs = results_all[food]

 row = {"Food": food.replace("_", " ").title()}

 for r in recs:
 tier = r["tier"]

 # Cluster name is the FLAVOR RECOMMENDATION – what taste space this maps to
 cluster_name = r["name"]

 # Keywords are the taste signature of the cluster
 kws = ", ".join(r["keywords"][:3])

 # Wine is the concrete bottle example that lives in this cluster
 wine = r["wine"][:38]

 rating = r["rating"]

 # Primary: cluster name + keywords | Secondary: wine example
 row[tier] = f"{cluster_name} ({kws})"

 row[f"{tier} wine"] = f"{wine} · {rating}pt"

 _rows.append(row)
```

```
df_table = pd.DataFrame(_rows).set_index("Food")
```

```
print("\n" + "■" * 115)

print(" WINE DINE – Food → Flavor Cluster Recommendation (20 foods × 3 tiers)")

print(" Each cell: cluster name (taste profile keywords)")

print("■" * 115)

print(df_table[_tier_cols].to_string())

print("■" * 115)
```

[illegible]

**Error:**

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 17, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
NameError: name 'results all' is not defined
```

## Cell 185: ■ Markdown

## 14.8 — Model Performance Comparison

The chart below places all five trained models on a single performance axis, then adds the integrated pipeline's recommendation diversity as a second quality measure.

- **CNN branch** (left panel): CNN from Scratch vs. ResNet-50 transfer — both evaluated on the Food-101 101-class test set.

- **\*\*RNN branch\*\*** (middle panel): LSTM Baseline, BiLSTM+Attention, DistilBERT (frozen) — all evaluated on the 15-class WineSensed grape test set.

- **Integrated pipeline** (right panel): cluster coverage — what percentage of the 9 flavor clusters are actually used by the recommendation engine, per tier. Full coverage (100%) means the system distributes food pairings across all available wine styles rather than collapsing to a small subset.

The two sub-tasks are independent by design (§1 project spec): the CNN identifies the food, the RNN encoder maps taste descriptions into an embedding space, and the integration layer connects them. Each sub-task is evaluated on its own ground truth.

## Cell 186: ■ Code

[illegible]

```

_cnn_labels = ["CNN Scratch", "ResNet-50\n(transfer)"]
_cnn_vals = [_cnn_scratch_acc * 100, _resnet_acc * 100]
_cnn_colors = ["#AEC6E8", "#1F77B4"]

bars = axes[0].bar(_cnn_labels, _cnn_vals, color=_cnn_colors, width=0.5,
edgecolor="white", linewidth=1.2)
axes[0].set_ylim(0, 100)
axes[0].set_ylabel("Test Accuracy (%)")
axes[0].set_title("CNN Branch\n(Food-101, 101 classes)", fontsize=10)
axes[0].axhline(y=1/101*100, color="grey", lw=1, ls="--", alpha=0.7, label="Random
baseline (1%)")
axes[0].legend(fontsize=8)
for bar, val in zip(bars, _cnn_vals):
axes[0].text(bar.get_x() + bar.get_width()/2, val + 1, f"{val:.1f}%",
ha="center", va="bottom", fontsize=9, fontweight="bold")

# Panel 2: RNN branch
_rnn_labels = ["LSTM\nBaseline", "BiLSTM +\nAttention", "DistilBERT\n(frozen)"]
_rnn_vals = [_lstm_acc * 100, _bilstm_acc * 100]
_rnn_colors = ["#AECDE8", "#4C72B0", "#55A868"]
if _bert_acc is not None:
_rnn_vals.append(_bert_acc * 100)
else:
_rnn_vals.append(0)
_rnn_labels[-1] = "DistilBERT\n(not run)"
bars2 = axes[1].bar(_rnn_labels, _rnn_vals, color=_rnn_colors, width=0.5,
edgecolor="white", linewidth=1.2)
axes[1].set_ylim(0, 100)
axes[1].set_ylabel("Test Accuracy (%)")
axes[1].set_title("RNN Branch\n(WineSensed, 15 grape classes)", fontsize=10)
axes[1].axhline(y=1/15*100, color="grey", lw=1, ls="--", alpha=0.7, label="Random
baseline (6.7%)")
axes[1].legend(fontsize=8)
for bar, val in zip(bars2, _rnn_vals):
if val > 0:
axes[1].text(bar.get_x() + bar.get_width()/2, val + 0.5, f"{val:.1f}%",
ha="center", va="bottom", fontsize=9, fontweight="bold")

```



```
print()
print(f" {'Integrated pipeline':&lt;30} {'Cluster coverage (avg 3 tiers)':&lt;30}
{sum(_div_vals)/len(_div_vals):&gt;9.0f}%")
print()
print("✓ 14.8 – Model performance comparison complete.")
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 18, in <module>
NameError: name 'tier_counts' is not defined
```

#### Cell 187: ■ Markdown

### Section 15 — Joint Model (+10 pts Bonus)

The individual branches are:

- **CNN branch** (§§8–10): ResNet-50 classifies a food photo into one of 101 Food-101 categories.
- **RNN branch** (§12): TasteBiLSTM encodes a food flavor description into a 512-d taste vector, used for cluster-based wine recommendation.

Each branch solves its own task. The joint model asks: *\*can a single network predict the flavor cluster directly from a food image AND its flavor text simultaneously?\**

#### **Shared target label**

The 9 K-means flavor clusters from §14.2 serve as the shared classification target.

- Each food class has a canonical SAFE BET cluster (`results_all[food][0]["cluster"]`, §14.6).
- Every food image from Food-101 is labelled with the cluster index of its food class (0–8).
- The flavor description text (`food_flavor_table_v2[food]["classic"]`) provides the text modality.

This is a 9-class classification problem where **both modalities describe the same underlying taste concept** — the image shows what the food looks like, the text describes how it tastes.

#### **Architecture**

...

Food image ■■ frozen ResNet-50 (avgpool layer) ■■ 2048-d

■■ Linear(2048 → 256) + ReLU + Dropout ■■■■■■■■■■■■■■

■■■ concat(512-d)

Flavor text ■■ frozen TasteBiLSTM.encode() ■■ 512-d ■ ■■ Linear(512 → 256) + ReLU



...

### ***Sub-sections***

||--||--||

```
| **15.2** | Joint dataset: map each Food-101 image to (image, token_ids, cluster_k) |
```

**\*\*15.4\*\*** | Three model variants: `CNNOnlyHead`, `RNNOnlyHead`, `JointFoodWineModel` |

**\*\*15.6\*\*** | Test evaluation + comparison: CNN-only vs RNN-only vs Joint |

\$6 ('val\_test\_transform', IMAGENET\_MEAN, IMAGENET\_STD), \$7 ('VOCAB', MAX\_SEQ\_LEN),  
\$10 ('resnet50'), \$12 ('taste\_encoder'), \$14 ('food\_flavor\_table\_v2', 'results\_all', 'K\_CLUSTERS').

```
import string as _str15
```









|-|-|-|-|-|

```
| `CNNOnlyHead` | food image | frozen ResNet-50 → avgpool (2048-d) |
Linear(2048→256)→ReLU→Dropout→Linear(256→9) |
```

```
| `RNNOnlyHead` | flavor text | frozen TasteBiLSTM.encode() (512-d) |
Linear(512→256)→ReLU→Dropout→Linear(256→9) |
```

```
| `JointFoodWineModel` | image + text | both frozen backbones | project each to 256-d → concat(512) → Linear(512→256)→ReLU→Dropout→Linear(256→9) |
```

All backbones are **frozen** (`requires_grad=False`). Only the small projection heads are trained.

This isolates the question: does fusing image + taste-text features improve cluster prediction?

Cell 192: ■ Code

```
def forward(self, imgs, _tok=None, _len=None):
 x = self.features(imgs).flatten(1) # (B, 2048)
 return self.head(x)
```

```
class RNNOnlyHead(nn.Module):

 """Frozen TasteBiLSTM encoder (512-d) → trainable head → 9 cluster classes."""

 def __init__(self):
 super().__init__()

 self.encoder = taste_encoder

 for p in self.encoder.parameters():
 p.requires_grad = False

 self.head = _proj_head(512)
```

```
CNN branch
```

```

_bb = resnet50
self.img_features = nn.Sequential(*list(_bb.children())[:-1])
for p in self.img_features.parameters():
 p.requires_grad = False
self.img_proj = nn.Sequential(
 nn.Linear(2048, self.PROJ_DIM), nn.ReLU(), nn.Dropout(0.3))

RNN branch
self.encoder = taste_encoder
for p in self.encoder.parameters():
 p.requires_grad = False
self.txt_proj = nn.Sequential(
 nn.Linear(512, self.PROJ_DIM), nn.ReLU(), nn.Dropout(0.3))

Joint head
self.head = nn.Sequential(
 nn.Linear(self.PROJ_DIM * 2, 256),
 nn.ReLU(),
 nn.Dropout(0.3),
 nn.Linear(256, K_CLUSTERS),
)

def forward(self, imgs, tok_ids, lengths):
 # Image stream
 img_feat = self.img_features(imgs).flatten(1) # (B, 2048)
 img_proj = self.img_proj(img_feat) # (B, 256)

 # Text stream
 self.encoder.eval()
 with torch.no_grad():
 txt_feat = self.encoder.encode(tok_ids, lengths) # (B, 512)
 txt_proj = self.txt_proj(txt_feat) # (B, 256)

 # Fusion
 fused = torch.cat([img_proj, txt_proj], dim=1) # (B, 512)
 return self.head(fused) # (B, 9)

```





## Cell 193: ■ Markdown

### 15.5 — Training: CNN-only, RNN-only, Joint

All three models share the same training protocol:

Setting	Value
---------	-------

||--||--||

| Loss | `CrossEntropyLoss` (9 balanced-ish classes — no weighting needed) |

```
| Optimiser | `Adam`, lr=`1e-3`, weight_decay=`1e-4` |
```

| Scheduler | `ReduceLROnPlateau` (factor 0.5, patience 2) |

| Early stopping | patience 4, monitored on val loss |

| Max epochs | 15 |

| Gradient clipping | max-norm 5.0 |

| Checkpoints | `joint\_cnn\_best.pt`, `joint\_rnn\_best.pt`, `joint\_full\_best.pt` |

Only the **trainable head** parameters are updated. Backbones stay frozen throughout.

## Cell 194: ■ Code

[illegible]

```
from torch.optim import Adam
```

```
from torch.optim.lr_scheduler import ReduceLROnPlateau
```

JOINT\_EPOCHS = 15

JOINT\_PATIENCE = 4

JOINT\_CLIP = 5.0

```
criterion_joint = nn.CrossEntropyLoss().to(DEVICE)
```

```
def _joint_train_epoch(model, loader, opt):
```

```
model.train()
```

```
Freeze backbone batch-norm layers (keep frozen ResNet BN in eval mode)
```

```
for m in model.modules():
```

```
if isinstance(m, nn.BatchNorm2d):
```

```
m.eval()
```

```
total_loss, correct, n = 0.0, 0, 0
```

```
for imgs, toks, labels in loader:
```

```
imgs, toks, labels = imgs.to(DEVICE), toks.to(DEVICE), labels.to(DEVICE)
```

```

lengths = (toks != 0).sum(1).clamp(min=1)
opt.zero_grad()
logits = model(imgs, toks, lengths)
loss = criterion_joint(logits, labels)
loss.backward()
nn.utils.clip_grad_norm_(
[p for p in model.parameters() if p.requires_grad], JOINT_CLIP)
opt.step()
total_loss += loss.item() * len(labels)
correct += (logits.argmax(1) == labels).sum().item()
n += len(labels)
return total_loss / n, correct / n

```

```

@torch.no_grad()
def _joint_eval(model, loader):
 model.eval()
 total_loss, correct, n = 0.0, 0, 0
 for imgs, toks, labels in loader:
 imgs, toks, labels = imgs.to(DEVICE), toks.to(DEVICE), labels.to(DEVICE)
 lengths = (toks != 0).sum(1).clamp(min=1)
 logits = model(imgs, toks, lengths)
 loss = criterion_joint(logits, labels)
 total_loss += loss.item() * len(labels)
 correct += (logits.argmax(1) == labels).sum().item()
 n += len(labels)
 return total_loss / n, correct / n

```

```

def _train_joint_model(model, name, ckpt_name):
 """Train one variant; return best val acc and history dict."""
 opt = Adam([p for p in model.parameters() if p.requires_grad],
lr=1e-3, weight_decay=1e-4)
 sched = ReduceLROnPlateau(opt, factor=0.5, patience=2)
 best_val_loss = float("inf")
 no_improve = 0
 history = {"train_loss": [], "val_loss": [],

```

```
"train_acc": [], "val_acc": []}
```

```
print(f"\nTraining {name} (max {JOINT_EPOCHS} epochs, patience={JOINT_PATIENCE})")
print(f"{'Epoch':>5} {'tr_loss':>8} {'tr_acc':>7} {'vl_loss':>8}
{'vl_acc':>7} {'lr':>9}")
print("-" * 55)
```

```
for epoch in range(1, JOINT_EPOCHS + 1):
 tr_loss, tr_acc = _joint_train_epoch(model, joint_train_loader, opt)
 vl_loss, vl_acc = _joint_eval(model, joint_val_loader)
 sched.step(vl_loss)
 lr_now = opt.param_groups[0]["lr"]
 history["train_loss"].append(tr_loss)
 history["val_loss"].append(vl_loss)
 history["train_acc"].append(tr_acc)
 history["val_acc"].append(vl_acc)
```

```
marker = ""
if vl_loss < best_val_loss:
 best_val_loss = vl_loss
 no_improve = 0
 save_checkpoint(model.state_dict(), ckpt_name)
 marker = " ✓"
else:
 no_improve += 1
```

```
print(f"{epoch:>5} {tr_loss:>8.4f} {tr_acc:>7.4f} "
f"{vl_loss:>8.4f} {vl_acc:>7.4f} {lr_now:>9.2e}{marker}")
```

```
if no_improve >= JOINT_PATIENCE:
 print(f" Early stopping at epoch {epoch}.")
 break
```

```
Reload best weights
model.load_state_dict(load_checkpoint(ckpt_name))
print(f" Best val loss: {best_val_loss:.4f} → saved as {ckpt_name}")
return history
```



> **Label-source note:** Flavor cluster labels were assigned by the TasteBiLSTM recommendation pipeline (§13–14). This means the RNN-only baseline has a structural advantage — it is essentially predicting a label derived from its own feature space. To keep the comparison fair, **all three backbones are frozen** and only the lightweight fusion heads are trained from scratch. The evaluation measures whether adding the image signal improves over text alone — a legitimate and useful question regardless of label origin.

> **\*\* Bonus requirement:** This section fulfills the +10 pts Joint Model bonus from §6 of the project brief: CNN feature vector (ResNet-50 avgpool, 2048-d) + RNN feature vector (TasteBiLSTM encode, 512-d) → concatenated → FC layers → single shared prediction target (9 flavor clusters). All three comparisons (CNN-only, RNN-only, Joint) are evaluated on the same held-out test set.

## Cell 196: ■ Code

[illegible]

```
print(f"\n{'Model':<25} {'Test Acc':>9} {'Macro F1':>9}")
print("-" * 47)

print(f"{'CNNOnlyHead':<25} {acc_cnn*100:>8.2f}% {f1_cnn:>9.4f}")
print(f"{'RNNOnlyHead':<25} {acc_rnn*100:>8.2f}% {f1_rnn:>9.4f}")
print(f"{'JointFoodWineModel':<25} {acc_jnt*100:>8.2f}% {f1_jnt:>9.4f}")
```

## # ■■ Training curves

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for hist, label, color in [
 (history_cnn_only, "CNN-only", "#1F77B4"),
 (history_rnn_only, "RNN-only", "#FF7F0E"),
 (history_joint, "Joint", "#2CA02C"),
]:
 axes[0].plot(hist["train_loss"], ls="--", color=color, alpha=0.6)
 axes[0].plot(hist["val_loss"], ls="--", color=color, label=label)
 axes[1].plot(hist["train_acc"], ls="--", color=color, alpha=0.6)
 axes[1].plot(hist["val_acc"], ls="--", color=color, label=label)
```

```
axes[0].set_title("Loss (solid=val, dashed=train)")
axes[1].set_title("Accuracy (solid=val, dashed=train)")
for ax in axes:
 ax.set_xlabel("Epoch")
 ax.legend()
fig.suptitle("$15.5 - Joint model training curves", fontsize=11)
fig.tight_layout()
save_figure(fig, "joint_training_curves.png")
plt.show()
```

## # ■■ Comparison bar chart

```
fig2, ax2 = plt.subplots(figsize=(7, 4))

_names = ["CNN-only", "RNN-only", "Joint"]

_accs = [acc_cnn * 100, acc_rnn * 100, acc_jnt * 100]

_colors = ["#1F77B4", "#FF7F0E", "#2CA02C"]

bars = ax2.bar(_names, _accs, color=_colors, width=0.5, edgecolor="white",
linewidth=1.2)
```







**Cell 198: ■ Code**

```
■■■ 15.7 Qualitative analysis
```

```
import pandas as pd

import matplotlib.pyplot as plt

import matplotlib.gridspec as gridspec

■■ helpers
```

```
def _cluster_label(idx):
 """Return short cluster name for a given integer index."""
 return cluster_names.get(int(idx), str(idx))

■■ 15.7a Sample prediction table
```

```
print("=== 15.7a Sample Predictions (test set) ===\n")
```

```
collect all predictions + ground truth already computed in §15.6
pred_cnn, pred_rnn, pred_jnt, gt are lists of ints (same order as joint_test_loader)
```

```
food names for the test set (same order as DataLoader)
joint_test_ds.indices holds the original Food-101 pool indices;
_all_labels maps those back to the food class string (e.g. "pizza")
test_food_names = [
 _food101_classes[_all_labels[joint_test_ds.indices[i]]]
 for i in range(len(joint_test_ds))
]
```

```
rows = []

for idx, (food, true, c, r, j) in enumerate(
 zip(test_food_names, gt, pred_cnn, pred_rnn, pred_jnt)):
 rows.append({
 "Food": food.replace("_", " ").title(),
 "True cluster": _cluster_label(true),
 "CNN pred": _cluster_label(c),
 "RNN pred": _cluster_label(r),
```

```
"Joint pred": _cluster_label(j),
"CNN ✓": "✓" if c == true else "X",
"RNN ✓": "✓" if r == true else "X",
"Joint ✓": "✓" if j == true else "X",
})
```

```
df_preds = pd.DataFrame(rows)
```

```
pick 12 interesting rows: 4 all-correct, 4 joint-wrong
all_correct = df_preds[(df_preds["CNN ✓"]=="✓") & (df_preds["RNN ✓"]=="✓") &
(df_preds["Joint ✓"]=="✓")].head(4)

joint_rescues = df_preds[
(df_preds["Joint ✓"]=="✓") &
((df_preds["CNN ✓"]=="✗") | (df_preds["RNN ✓"]=="✗"))
].head(4)

all_wrong = df_preds[(df_preds["CNN ✓"]=="✗") & (df_preds["RNN ✓"]=="✗") &
(df_preds["Joint ✓"]=="✗")].head(4)

sample_df = pd.concat([all_correct, joint_rescues,
all_wrong]).reset_index(drop=True)
```

```
display_cols = ["Food", "True cluster", "CNN pred", "CNN ✓", "RNN pred", "RNN ✓",
 "Joint pred", "Joint ✓"]

print(sample_df[display_cols].to_string(index=False))
print()
```

```
████ 15.7b Per-cluster accuracy breakdown
```

```
print("=== 15.7b Per-cluster Accuracy ===\n")
```

```
n_clusters = K_CLUSTERS

cnn_per, rnn_per, jnt_per = [], [], []
```

```
for k in range(n_clusters):
 mask = [i for i, g in enumerate(gt) if g == k]
 if not mask:
 cnn_per.append(0); rnn_per.append(0); jnt_per.append(0)
 continue
 cnn_per.append(sum(pred_cnn[i] == k for i in mask) / len(mask))
 rnn_per.append(sum(pred_rnn[i] == k for i in mask) / len(mask))
```

[illegible]

```

for i in range(len(gt))
if pred_jnt[i] == gt[i]
][:10]

fall back to first 10 if fewer correct
if len(correct_test_foods) < 10:
correct_test_foods = [
(test_food_names[i], gt[i], pred_jnt[i])
for i in range(min(10, len(gt)))
]

print(f" {'Food':<28} {'True cluster':<24} {'Joint cluster':<24} {'Example wine'}")
print(" " + "-"*100)

for food_raw, true_k, pred_k in correct_test_foods:
food_key = food_raw.lower()
cluster_name = _cluster_label(pred_k)

pull top wine from results_all if available, else fallback
wine_example = "_"
if food_key in results_all and results_all[food_key]:
top = results_all[food_key][0]
wine_example = f"{top.get('wine','?')} · {top.get('rating_pt','?')} pt"

match = "✓" if true_k == pred_k else "≠"
print(f" {food_raw.replace('_', ' ').title():<28} "
f" {_cluster_label(true_k):<24} "
f" {cluster_name:<24} "
f" {wine_example} {match}")

print(f"\n✓ $15.7 complete – qualitative analysis done.")

```

## Output:

```

=== 15.7a Sample Predictions (test set) ===

```

## Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
exec('\n'.join(lines[:-1]), glb)

```

```
~~~~~AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA~
File "<string>", line 22, in <module>
NameError: name 'joint_test_ds' is not defined. Did you mean: '_joint_test_eval'?
```

## Cell 199: ■ Markdown

## 15.8 — Random Food Predictor (Simulated Upload Demo)

This section simulates what would happen if a user **uploaded a food photo** to the Wine & Dine pipeline.

We pick a random image from the held-out test set and run it through all three joint models.

The Joint model's prediction is then used to generate a full sommelier-style **food card** — identical to the business output of §14.

This is the closest thing to a live demo without a Gradio UI.

Cell 200: ■ Code

[illegible]

[illegible]









```
plt.show()

print(f"\n{'█'*60}")
print(f"Image (true label) : {true_food_name.replace('_', ' ').title()}")
print(f"CNN food pred : {cnn_food_name.replace('_', ' ').title()}
({cnn_food_conf*100:.0f}%) {'✓' if correct_food else 'X'}")
print(f"Lookup food used : {lookup_food.replace('_', ' ').title()}")
print(f"Joint cluster : {jnt_cluster_name} ({jnt_conf*100:.0f}%)")
print(f"Recommended wine : {top_wine} · {top_rating} pts")
print(f"{'█'*60}")
print("Re-run cell for a different random food!")
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 17, in <module>
NameError: name '_all_labels' is not defined. Did you mean: 'used_labels'?
```

#### Cell 201: ■ Markdown

## 16 — Deployment Prototype (HuggingFace Spaces)

The Wine & Dine app is deployed as a **Gradio Space** on HuggingFace.

**\*\*Live demo:\*\***

[<https://huggingface.co/spaces/Jolanati/wine-dine>](https://huggingface.co/spaces/Jolanati/wine-dine)

### *What the app does*

The app walks through the full pipeline interactively, one step at a time:

| Step | What happens |

|---|---|

| **\*\*Upload photo\*\*** | User uploads any food photo |

| **\*\*CNN identifies food\*\*** | ResNet-50 runs in real time → food class + confidence + top-5 bar |

| **\*\*User confirms\*\*** | **\*\*"Yes, that's right!"** — confirms or rejects the prediction |

| **\*\*BiLSTM encodes flavors\*\*** | Food's flavor description is tokenized and encoded live through the trained BiLSTM |

| **\*\*Cluster match\*\*** | Cosine similarity to saved cluster centroids → flavor cluster shown |

| **\*\*Wine card\*\*** | Safe Bet · Characteristic · Contrast wines with tasting notes |

**Artifacts saved to `deployment/`**

The cell below exports all runtime artifacts needed by the Space.

## Cell 202: ■ Code

```

3. Cluster centroids (K x 512, already L2-normalised)
cent_path = os.path.join(DEPLOY_DIR, "centroids.npy")
np.save(cent_path, centroids)
print(f"✓ centroids.npy {centroids.shape}")

4. results_all (wine recommendations per food)
ra_path = os.path.join(DEPLOY_DIR, "results_all.json")
with open(ra_path, "w", encoding="utf-8") as f:
 json.dump(results_all, f, ensure_ascii=False, indent=2)
print(f"✓ results_all.json ({len(results_all)} foods)")

5. food_flavor_table_v2 (flavor descriptions)
src_ffv2 = "/content/drive/MyDrive/wine-dine/data/food_flavor_description_v2.json"
dst_ffv2 = os.path.join(DEPLOY_DIR, "food_flavor_description_v2.json")
if os.path.exists(src_ffv2):
 shutil.copy(src_ffv2, dst_ffv2)
 print(f"✓ food_flavor_description_v2.json (copied from data/)")
else:
 # Try local path
 for alt in ["data/food_flavor_description_v2.json",
 "wine-dine/data/food_flavor_description_v2.json"]:
 if os.path.exists(alt):
 shutil.copy(alt, dst_ffv2)
 print(f"✓ food_flavor_description_v2.json (copied from {alt})")
 break
 else:
 # Save directly from in-memory variable
 with open(dst_ffv2, "w", encoding="utf-8") as f:
 json.dump(food_flavor_table_v2, f, ensure_ascii=False, indent=2)
 print(f"✓ food_flavor_description_v2.json (saved from memory)")

print(f"\n✓ All deployment artifacts saved to {DEPLOY_DIR}")
print(f"Weights to upload to Space: cnn_resnet50_best.pt bilstm_best.pt")

```

#### Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec('\n'.join(lines[:-1]), glb)

```

```
~~~~~^AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA^
File "<string>", line 22, in <module>
NameError: name 'VOCAB' is not defined
```

## Cell 203: ■ Markdown

### Screenshot

- > \*(Add a screenshot of the running app here after deployment — required by §5.7)\*
- >
- > Steps: open the Space URL → upload a food photo → click Identify → confirm → take screenshot.

## Cell 204: ■ Markdown

---

## 17 — Business Framing, Ethics & Team Contributions

### 17.1 — Business Use Case

**\*\*What business decision does this system support?\*\***

Wine & Dine supports **\*\*real-time wine pairing recommendations\*\*** at the point of service. The end users are:

| User | Decision | Context |

| --- | --- | --- |

| **\*\*Restaurant sommelier / waiter\*\*** | Which wine to suggest with a guest's dish | Table-side, 10-second interaction |

| **\*\*Wine retail app\*\*** | Cross-sell wine alongside food delivery orders | In-app recommendation widget |

| **\*\*Home cook / enthusiast\*\*** | Which bottle to open for tonight's dinner | Mobile photo of the finished plate |

The system replaces static "pairing charts" (which cover only 20–30 dishes) with a dynamic, data-driven pipeline that handles 101 food classes and scales to any cuisine via the shared taste embedding space.

**\*\*Revenue model:\*\*** SaaS integration fee for restaurants and delivery platforms; freemium mobile app with premium pairing details (tasting notes, buy links).

---

### 17.2 — Cost of Errors: False Positives vs. False Negatives

| Model | False Positive (predicts wrong class) | False Negative (misses correct class) | Which is worse? |

| --- | --- | --- | --- |



| Integration point | Input | Latency budget | Deployment |

| --- | --- | --- | --- |

| Restaurant POS system | Dish name from menu database | < 500 ms | REST API (FastAPI behind the Gradio app) |

| Food delivery app | Photo from restaurant listing | < 2 s (includes CNN inference) | Same API, GPU instance for CNN |

| Consumer mobile app | User's phone photo | < 3 s (upload + inference) | HuggingFace Spaces (current deployment) |

**\*\*Key constraint:\*\*** The system must work **\*\*without internet-scale data at inference time\*\*** — all models and embeddings are pre-computed and shipped with the deployment. Only a single forward pass through each model is needed per request.

---

## 17.4 — Ethical Considerations

| Concern | Risk | Mitigation |

| --- | --- | --- |

| **\*\*Cultural bias in food images\*\*** | Food-101 is Western-centric: 101 classes skew toward European/American cuisine. Asian, African, and Middle-Eastern dishes are underrepresented → CNN misclassifies them more often. | Acknowledge limitation explicitly. Future: expand to Food-2k or region-specific datasets. The "confirm your dish" step catches misclassification before pairing runs. |

| **\*\*Alcohol promotion\*\*** | System recommends wine — inappropriate for users under legal drinking age, recovering alcoholics, or cultures where alcohol is prohibited. | App does not store user data; no age-gating is implemented in the prototype. Production deployment MUST include age verification and opt-out mechanisms. |

| **\*\*Wine review bias\*\*** | WineSensed reviews are written predominantly by Western wine critics (English-language Vivino). Recommendations reflect Western wine culture preferences, not universal taste. | State clearly that pairings reflect European/New World wine tradition. Non-alcoholic pairing mode (tea, juice) is a planned extension. |

| **\*\*LLM-generated flavor descriptions\*\*** | The food→flavor bridge (`food\_flavor\_description\_v2.json`) was generated by Claude Sonnet 4.6 — it may encode stereotypes (e.g., "all curry is spicy") or hallucinate flavor attributes. | Descriptions were manually reviewed for the 101 classes. Factual verification against culinary references (Larousse Gastronomique) for top-20 foods. |

| **\*\*Model fairness\*\*** | If CNN accuracy varies by food category (e.g., lower for visually similar Asian dishes), users of those cuisines receive worse recommendations. | Reported per-class accuracy in §9. Classes below 70% accuracy are flagged in the UI with lower confidence scores. |

---

## 17.5 — Team Contributions

| Team Member | Responsibility | Sections |

| --- | --- | --- |



| **Jolanta Stutane** | Environment setup, TasteBiLSTM encoder design & training, flavor-cluster recommendation engine, joint model architecture | §1, §12, §14, §15, §16 |

| **Evija Sturma** | EDA (both datasets), CNN from scratch (both generations), Grad-CAM explainability, recommendation evaluation, business framing | §5, §8, §10, §14, §17 |

| **Uģis Ritišs** | Data loading (Food-101 & WineSensed), image preprocessing & augmentation, ResNet-50 transfer learning, model export, joint model training | §2, §6, §9, §13, §15 |

| **Āirts Ziemanis** | Data cleaning & grape filtering, text preprocessing & tokenisation, RNN/LSTM/BiLSTM/DistilBERT, TasteBiLSTM co-design, appendix experiments | §3, §7, §11, §12, §18 |

| **Māris Zelis** | Train/val/test splits, EDA visualisations, Gradio deployment & HF Spaces, business framing & ethics, appendix documentation | §4, §5, §16, §17, §18 |

## Cell 205: ■ Markdown

---

# Appendix — The Road to the Final Architecture

## The starting assumption

We began this project with a straightforward hypothesis: **wine pairing can be modelled as a grape-variety matching problem.** If we can map a food's flavor profile into the same vector space as grape varieties, cosine similarity will recover sensible pairings. The grape is the intermediate representation — food maps to grape, grape maps to wine.

This appendix documents how that assumption was systematically tested, refined, and ultimately rejected — and why the rejection itself is a result. Each experiment was designed to test the specific hypothesis generated by the one before it, forming a chain of reasoning that ends at a single architectural conclusion.

## The chain of experiments

| # | Approach | What we tested | What we learned |

|---|---|---|---|

| 1 | W2V query expansion → centroid cosine | Can distributional word similarity bridge food and wine language? | All 101 food queries collapse to the same vector region (pairwise cosine 0.82). Only 4/15 grapes ever appear. |

| 2 | BiLSTM classifier argmax | Can a trained grape classifier recommend directly? | 9/15 grapes reached, but one grape dominates 44/101 foods. Temperature scaling cannot fix ranking order. |

| 3 | BiLSTM centroid cosine + keywords | Does using the encoder (not the classifier) fix the problem? | Architecturally correct, but diversity ceiling: grape centroids at pairwise cosine 0.85–0.99. |

| 4 | BiLSTM + wine-register descriptions | Is the bottleneck the input format (keywords), not the encoder? | 9/10 foods change their recommendation — format matters for routing — but diversity ceiling stays. |

| 5 | MPNET + keywords | Does a contrastively-trained encoder fix the centroid geometry? | MPNET centroids spread well (0.03–0.70), but keywords land in wrong region — only 2/10 unique grapes. |

| 6 | MPNET + SupCon projection head | Can we steer the MPNET space with a trained projection head? | Frozen backbone weights produce the centroids; a projection head on top cannot change them. |

| 7 | MPNET + wine-register descriptions | Does the correct input register unlock MPNET's spread geometry? | 5/10 unique grapes — approaching BiLSTM, but with completely different grapes. Domain mismatch confirmed. |

| 8 | Full 4-way comparison (20 foods) | Which encoder × input-format combination wins at scale? | BiLSTM + wine-register descriptions wins (most diverse, most grapes reached across all three recommendation slots). |

### ***What the chain proves***

The eight experiments progressively isolated five independent constraints:

1. **Distributional similarity ≠ flavor similarity** (Approach 1)
2. **Classification training ≠ embedding separation** (Approaches 2–3)
3. **Input register matters as much as the encoder** (Approaches 4, 7)
4. **Frozen backbone fine-tuning is structurally limited** (Approach 6)
5. **Grape variety is the wrong intermediate target** (all approaches)

Constraint 5 is the decisive finding. Every approach — regardless of encoder, training objective, or input format — is bounded by the same structural ceiling: averaging thousands of reviews per grape erases the within-grape flavor variation that makes pairings interesting. A Sangiovese from Tuscany and a Sangiovese from Australia produce completely different flavor experiences, yet they share a single centroid.

### ***The architectural response***

The final architecture (Section 14) abandons grape variety entirely. Instead:

- **k-means on BiLSTM review embeddings** discovers flavor clusters directly from the data
- Clusters represent **taste profiles**, not biological taxonomy
- The same BiLSTM encoder embeds both food descriptions and wine reviews in a shared space
- No new training is needed — only a different way of organising the target space

This appendix is the proof that this architecture was not chosen by intuition — it was forced by systematic elimination of alternatives.

## **Cell 206: ■ Markdown**

---

### ***A1 — Approach 1: Word2Vec Query Expansion (Food Keywords → Grape Centroids)***

**Starting hypothesis:** Wine reviews and food descriptions share a common sensory vocabulary (\*tomato\*, \*creamy\*, \*smoky\*, \*tannic\*). If we fine-tune Word2Vec on 824k wine reviews, food keywords should become vector-space neighbours of the grape-characteristic words they pair with. Cosine similarity between a food-keyword vector and IDF-weighted grape centroids should then recover meaningful pairings.

**What we built:**

- Fine-tuned Google News Word2Vec (300-d, 3M vocab) on WineSensed training reviews (A1.1)
- Computed IDF-weighted TF-IDF grape centroid vectors — one 300-d vector per grape (A1.2)
- Built a full recommendation pipeline: `expand_keywords()` → `embed_keywords()` → cosine vs. centroids (A1.3)

**What went wrong:** After fine-tuning, wine-specific terms (\*cassis\*, \*terroir\*, \*mineral\*) entered the food-keyword neighbourhood — but **all food keywords expanded to the same cloud** of high-frequency wine terms (\*rich\*, \*fruity\*, \*tannin\*). Mean pairwise cosine across 101 food query vectors was **0.82** — effectively, every food dish asks the same question. Only **4 of 15** grape varieties ever won any recommendation.

**The diagnosis:** The W2V distributional objective groups words by co-occurrence frequency, not by flavor distinctiveness. `expand_keywords("tomato")` and `expand_keywords("chocolate")` both return *rich, complex, dark, fruity* — because those words co-occur with everything in wine reviews.

**What this ruled out:** Distributional word similarity cannot separate flavor profiles. We need a model trained with a discriminative objective — one that is forced to tell grapes apart.

→ *This led directly to Approach 2: using the BiLSTM, which was trained with cross-entropy classification loss to distinguish 15 grape varieties.*

## Cell 207: ■ Markdown

#### A1.1 — Fine-tune Word2Vec on wine reviews

> **Note:** The Word2Vec model and grape centroids built here are used by **Appendix — Dropped 1, 2, 3**. They are not used by the new flavor-cluster architecture (Section 14). Code is preserved for reproducibility.

We start from the **Google News Word2Vec model** (300-d, 3 million vocabulary) — it already understands everyday food language: *tomato\*, \*fatty\*, \*smoky\*, \*creamy\**.

We then fine-tune it on the **824k WineSensed training reviews** so wine-specific words (*Sangiovese\*, \*cassis\*, \*tannic\*, \*terroir\*, \*mineral\**) are pulled into the same vector space as the food keywords. This cross-vocabulary alignment is what makes cosine similarity between food keywords and grape names meaningful.

| Sub-step | What |

|---|---|

| 7.6.1 | Install gensim + load Google News base model (~1.6 GB, cached after first run) |

| 7.6.2 | Build wine sentence corpus from `tok_train` |

| 7.6.3 | Extend vocab + fine-tune 5 epochs (~10–15 min CPU) |

| 7.6.4 | Save fine-tuned model → `weights/word2vec\_wine.model` + Drive |

> **Note:** `~/gensim-data/` is large (~1.6 GB). Do not commit it to git.

## Cell 208: ■ Markdown

##### A1.1.1 — Install gensim & define paths

Install [gensim](https://radimrehurek.com/gensim/) if not already present, import the required classes, and define local + Drive output paths for the fine-tuned model.

## Cell 209: ■ Code

```
import subprocess, sys

try:
    import gensim
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
                           "gensim>=4.3"])
    import gensim

import gensim.downloader as gensim_api
from gensim.models import Word2Vec

W2V_MODEL_PATH = WEIGHTS / "word2vec_wine.model"
W2V_DRIVE_PATH = Path(WEIGHTS_DIR) / "word2vec_wine.model"

print(f"gensim {gensim.__version__}")
print(f" local → {W2V_MODEL_PATH}")
print(f" Drive → {W2V_DRIVE_PATH}")
```

### Error:

Traceback (most recent call last):

File "<string>", line 4, in <module>  
ModuleNotFoundError: No module named 'gensim'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 10, in <module>  
 exec('\n'.join(lines[:-1]), glb)  
~~~~~

File "<string>", line 6, in <module>

File "C:\Users\jolanta.stutane\AppData\Local\Programs\Python\Python314\Lib\subprocess.py", line 1000, in _execute_child
 raise CalledProcessError(retcode, cmd)

```
subprocess.CalledProcessError: Command '['C:\\Users\\jolanta.stutane\\AppData\\Local\\Programs\\P
```

Cell 210: ■ Markdown

A1.1.2 — Build wine sentence corpus

`tok\_train` is a list of token-string sequences produced in Section 7.1.

We drop ``<pad>`` and ``<unk>`` tokens and feed the cleaned sentences to Word2Vec.

Cell 211: ■ Code

```
_skip = {PAD_TOKEN, UNK_TOKEN}
wine_sentences = [
    [tok for tok in seq if tok not in _skip]
    for seq in tok_train if seq
]

print(f"Corpus : {len(wine_sentences):,} sentences")
print(f"Avg len : {sum(len(s) for s in wine_sentences)/len(wine_sentences):.1f}
tokens/sentence")
```

Error:

```
Traceback (most recent call last):
  File "c:\\Users\\jolanta.stutane\\.vscode\\extensions\\ganeshkumbhar.nb2pdf-1.1.9\\scripts\\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 4, in <module>
NameError: name 'tok_train' is not defined
```

Cell 212: ■ Markdown

A1.1.3 — Load Google News base model & fine-tune

We download the **Google News Word2Vec** base (300-d, ~1.6 GB, cached after first run), initialise the vocabulary with pre-trained vectors, and fine-tune for **5 epochs** on wine reviews.

> Expect ~10–15 min on CPU; ~2 min on GPU.

Cell 213: ■ Code

```
import glob, shutil

print("Loading Google News base model (~1.6 GB, cached after first run)...")

_base_kv = gensim_api.load("word2vec-google-news-300") # KeyedVectors
```

```

# Build a trainable Word2Vec wrapper
w2v_model = Word2Vec(vector_size=300, window=5, min_count=2, workers=4, seed=SEED)
w2v_model.build_vocab(wine_sentences)

# Expand vocab with Google News words, then copy pre-trained vectors
w2v_model.build_vocab([list(_base_kv.key_to_index.keys())], update=True)
# vectors_lockf defaults to all-ones (fully trainable) – no manual override needed
for _w in _base_kv.key_to_index:
    if _w in w2v_model.wv:
        w2v_model.wv[_w] = _base_kv[_w]

print(f"Vocab size : {len(w2v_model.wv):,} words")
print(f"Fine-tuning Word2Vec | epochs=5 | device: CPU")
print("-" * 60)

N_EPOCHS = 5
history_w2v = {"loss": []}
_best_loss = float("inf")
_prev_loss = 0.0

def _sync_w2v_to_drive(ckpt_path):
    """Copy all gensim files for a given .model path to Drive."""
    if not IN_COLAB:
        return
    for _f in glob.glob(str(ckpt_path) + "*"):
        shutil.copy(_f, Path(WEIGHTS_DIR) / Path(_f).name)

for epoch in range(1, N_EPOCHS + 1):
    w2v_model.train(
        wine_sentences,
        total_examples=len(wine_sentences),
        epochs=1,
        compute_loss=True,
    )
    _total = w2v_model.get_latest_training_loss()
    _epoch_loss = _total - _prev_loss
    _prev_loss = _total

```

```

history_w2v["loss"].append(_epoch_loss)

# Save epoch checkpoint (resume-safe after Colab disconnect)
_ckpt_path = WEIGHTS / f"word2vec_wine_ep{epoch:02d}.model"
w2v_model.save(str(_ckpt_path))
_sync_w2v_to_drive(_ckpt_path)

# Keep a separate "best" copy
_marker = ""
if _epoch_loss < _best_loss:
    _best_loss = _epoch_loss
w2v_model.save(str(W2V_MODEL_PATH))
_sync_w2v_to_drive(W2V_MODEL_PATH)
_marker = " ← best"

print(f"Epoch {epoch:>2}/{N_EPOCHS} loss={_epoch_loss:.4f}{_marker}")

print(f"\n✓ Fine-tuning complete | final vocab: {len(w2v_model.wv):,} words")
print(f" Best loss : {_best_loss:.4f} | word2vec_wine.model = best checkpoint")

```

Output:

Loading Google News base model (~1.6 GB, cached after first run)...

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 4, in <module>
NameError: name 'gensim_api' is not defined

```

Cell 214: ■ Markdown

A1.1.4 — Save fine-tuned model

Persist the model locally and, if running in Colab, sync to Google Drive so it survives session restarts.

Cell 215: ■ Code

```

import shutil, glob

# Save gensim model (writes .model + auxiliary .npy files)

```

```
w2v_model.save(str(W2V_MODEL_PATH))

# Collect ALL files written by gensim for this model
_w2v_files = glob.glob(str(W2V_MODEL_PATH) + "**")

if IN_COLAB:
    W2V_DRIVE_PATH.parent.mkdir(parents=True, exist_ok=True)
    for _src in _w2v_files:
        _dst = Path(WEIGHTS_DIR) / Path(_src).name
        shutil.copy(_src, _dst)
    print(f"✓ Saved {len(_w2v_files)} file(s) to Drive:")
    for _src in sorted(_w2v_files):
        print(f" {Path(_src).name}")
    else:
        print(f"✓ Saved {len(_w2v_files)} file(s) locally:")
    for _src in sorted(_w2v_files):
        print(f" {Path(_src).name}")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 19
    for _src in sorted(_w2v_files):
                                   ^
IndentationError: expected an indented block after 'for' statement on line 19
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec(source, glb)
    ~~~~^~~~~~
  File "<string>", line 4, in <module>
NameError: name 'w2v_model' is not defined
```

Cell 216: ■ Markdown

A1.2 — Build W2V grape centroid embeddings

> **Note:** `grape_embeddings` and `_grape_emb_c` built here are used by **Appendix — Dropped 1, 2, 3**. The flavor-cluster architecture (Section 14) uses BiLSTM embeddings clustered by k-means instead.

Each grape variety is represented as a **centroid vector** in the 300-d Word2Vec space:

the average of all Word2Vec vectors for tokens appearing in training reviews for that grape.

This centroid is the bridge between food flavour keywords and grape recommendations.

When a food query arrives (e.g., pizza -> oomato, cheesy, savory, herby),

its keyword vectors are averaged and compared against all 15 grape centroids via cosine similarity.

| Sub-step | What |

|---|---|

| **7.7.1** | Define paths and load fine-tuned Word2Vec model |

| **7.7.2** | Compute per-grape centroid vectors (shape: 15 x 300) |

| **7.7.3** | Save grape embeddings locally and to Drive |

| **7.7.4** | Sanity check: centroid similarity heatmap + food flavor probes |

Cell 217: ■ Markdown

A1.2.1 — Define paths & load fine-tuned model

Define output paths and load the Word2Vec model produced in 7.6.4.

If `w2v\_model` is already in scope from 7.6.3, the load step is skipped.

Cell 218: ■ Code

```
import numpy as np
from gensim.models import Word2Vec

GRAPE_EMB_PATH = WEIGHTS / "grape_embeddings.npy"
GRAPE_EMB_DRIVE_PATH = Path(WEIGHTS_DIR) / "grape_embeddings.npy"

if "w2v_model" not in dir():
    _src = W2V_DRIVE_PATH if (IN_COLAB and W2V_DRIVE_PATH.exists()) else W2V_MODEL_PATH
    w2v_model = Word2Vec.load(str(_src))
    print(f" → Loaded from {_src}")
else:
    print(" → w2v_model already in scope")

_wv = w2v_model.wv
print(f" Vocab size : {len(_wv):,}")
print(f" Paths : local={GRAPE_EMB_PATH}")
```

Error:

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
exec('\n'.join(lines[:-1]), glb)
```

File "<string>", line 2, in <module>

```
ModuleNotFoundError: No module named 'gensim'
```

Cell 219: ■ Markdown

A1.2.2 — Compute per-grape centroid vectors

For each of the 15 grape classes, collect all training review tokens and average their Word2Vec vectors.

Result: `grape_embeddings` — shape `(15, 300)`, one centroid per grape in the shared flavor space.

Cell 220: ■ Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from collections import Counter as _c772
```

```
# ■■ Step 1: global IDF over all training reviews
```

```
# IDF up-weights words that are rare across ALL wine reviews (grape-specific
# vocabulary). Used as a multiplier in the discriminative weight formula below.
```

```
_all_reviews = df_train["review_text"].fillna("").astype(str).tolist()
```

```
_tfidf = TfidfVectorizer(max_features=50_000, sublinear_tf=True)
```

```
_tfidf.fit(_all_reviews)
```

```
_idf_map = dict(zip(_tfidf.get_feature_names_out(), _tfidf.idf_))
```

```
print(f"TF-IDF vocab size : {len(_idf_map):,}")
```

```
# ■■ Step 2: per-grape discriminative TF weighting
```

```
# For each word w and grape g:
```

```
# score(w, g) = (TF_g(w) / avg_TF_across_grapes(w))^alpha * IDF(w)
```

#

```
# A word appearing 10x more in Cabernet than average scores 100x higher for
# Cabernet (alpha=2 squares the ratio). A word like "tannins" that appears
# equally across all grapes gets ratio ~1 → near-zero contribution.
```

```
# Only the top-N scoring words per grape are used to build its centroid.
```

```
# This pulls the 15 centroids apart rather than clustering them together.
```



```

_vec, _wsum = np.zeros(300, dtype=np.float32), 0.0
for w, sc in _top:
    _vec += sc * _wv[w]
    _wsum += sc
grape_embeddings[g_idx] = _vec / (_wsum + 1e-8)

_n_rev = int((df_train["grape_class"] == grape).sum())
print(f" {grape:<28} {_n_rev:>6,} reviews top word: {_top[0][0]}")

print(f"\n✓ Discriminative TF-IDF centroids | shape: {grape_embeddings.shape}")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 7, in <module>
NameError: name 'df_train' is not defined

```

Cell 221: ■ Markdown

A1.2.3 — Save grape embeddings

Persist the `(15, 300)` matrix locally and sync to Drive if running in Colab.

Cell 222: ■ Code

```

np.save(GRAPE_EMB_PATH, grape_embeddings)

if IN_COLAB:
    import shutil
    shutil.copy(GRAPE_EMB_PATH, GRAPE_EMB_DRIVE_PATH)
    print(f"✓ Saved to Drive : {GRAPE_EMB_DRIVE_PATH}")
else:
    print(f"✓ Saved locally : {GRAPE_EMB_PATH}")

_size_kb = GRAPE_EMB_PATH.stat().st_size / 1024
print(f" File size : {_size_kb:.1f} KB")
print(f" Shape : {np.load(GRAPE_EMB_PATH).shape}")

```

Error:

```

Traceback (most recent call last):

```

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
File "<string>", line 1, in <module>
NameError: name 'GRAPE_EMB_PATH' is not defined
```

Cell 223: ■ Markdown

A1.2.4 — Sanity check: grape centroid similarity + food probe

Two complementary checks:

1. **Centroid similarity heatmap** — cosine similarity between all 15 grape centroids. Good embeddings show clearly separated grapes with only stylistically related pairs (e.g., Shiraz/Primitivo) clustering together.
2. **Food flavor probes** — 8 random `food\_flavor\_table` entries queried against grape centroids using their `classic` keywords. This tests the actual downstream pipeline (food → flavor keywords → grape centroid matching).

Cell 224: ■ Code

```
import random

from numpy.linalg import norm
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import cosine_similarity

# ~~~~~~
# ~~~~~~

# Part 1 – Grape centroid similarity heatmap
# ~~~~~~
# ~~~~~~

_sim_matrix = cosine_similarity(grape_embeddings) # (15, 15)

fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(
    _sim_matrix,
    xticklabels=GRAPE_CLASSES,
    yticklabels=GRAPE_CLASSES,
    annot=True, fmt=".2f", cmap="YlOrRd",
    vmin=0.85, vmax=1.0,
    linewidths=0.4, ax=ax,
```



```

_fft_path = next((p for p in _fft_candidates if p.exists()), None)
if _fft_path is None:
    raise FileNotFoundError(
        "food_flavor_table.json not found. Run cell 32 (Section 3.3) first.\n"
        + "\n".join(f" tried: {p}" for p in _fft_candidates)
    )
food_flavor_table = {k: v for k, v in _json.load(open(_fft_path)).items()
if k != "_meta"}

random.seed(42)
_foods = random.sample([k for k in food_flavor_table], min(8,
len(food_flavor_table)))

col1, col3 = 26, 22
print(f"\n{'Food':{col1}} {'Classic keywords':{col3}} Nearest grape (classic)")
print("-" * 90)
for _food in _foods:
    _kws = food_flavor_table[_food].get("classic", [])
    _grape = _nearest_grape(_kws)
    _kw_str = ", ".join(_kws[:4])
    print(f"{_food.replace('_', ' '):{col1}} {_kw_str:{col3}} {_grape}")

print("\n✓ 7.7 complete | grape_embeddings ready for Section 13 (recommend).")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 10, in <module>
NameError: name 'grape_embeddings' is not defined

```

Cell 225: ■ Markdown

A1.3 — Recommendation pipeline and convergence diagnostic

The cells below build the full W2V pairing pipeline (`expand\_keywords`, `embed\_keywords`, `ind\_grape`) and run the convergence diagnostic that exposes the fundamental flaw: all 101 food queries map to nearly identical vectors in the W2V space.

Cell 226: ■ Code

[illegible]

[illegible]

```
if kw_additions >= topn:
    break
```

```
if tok in _WINE_VOCAB and tok not in seen:
    expanded.append(tok)
seen.add(tok)
kw_additions += 1

if len(expanded) >= max_total:
    return expanded
```

```

if tok in _wv:
    try:
        neighbours = _wv.most_similar(tok, topn=topn * 6)
        for w, _ in neighbours:
            if w in _WINE_VOCAB and w not in seen:
                expanded.append(w)
                seen.add(w)
                kw_additions += 1
            if kw_additions >= topn or len(expanded) >= max_total:
                break
        except KeyError:
            pass

```

```
return expanded
```

```
#  embed_keywords()


def embed_keywords(keywords, food_label=None):
    """IDF-weighted mean W2V vector for a list of (expanded) keywords."""
    valid = expand_keywords(keywords)
```

```
if len(valid) < 2 and food_label and food_label in _wv:
    valid = valid + [food_label]
```

```
if not valid:
    return None
```

```
_vecs = np.array([_wv[w] for w in valid], dtype=np.float32)
_weights = np.array([_IDF.get(w, 1.0) for w in valid], dtype=np.float32)
```

[illegible]


```
_exp_pizza = expand_keywords(_pizza_kws)
_exp_sushi = expand_keywords(_sushi_kws)
_overlap = set(_exp_pizza) & set(_exp_sushi)

print(f" Overlap pizza ∩ sushi: {len(_overlap)} shared words →
{sorted(_overlap)}")

print()

# Full pipeline smoke-test for pizza

_g_sb, _p_sb = find_grape(_pizza_kws, food_label='pizza', rank=1)
_g_hg, _p_hg = find_grape(
food_flavor_table.get('pizza', {}).get('safe_bet', []),
food_label='pizza', rank=1, exclude={_g_sb}
)

_g_bm, _p_bm = find_contrast_grape(_g_sb, exclude={_g_hg})

print(f" Pizza → Safe Bet : {_g_sb:<22} ({_p_sb}% flavor match)")
print(f" Pizza → Hidden Gem: {_g_hg:<22} ({_p_hg}% flavor match)")
print(f" Pizza → Bold Move : {_g_bm:<22} ({_p_bm}% contrast)")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 8, in <module>
NameError: name 'df_train' is not defined
```

Cell 227: ■ Code

[illegible]


```

continue

sims = _grape_emb_c @ q / (_norm_d(_grape_emb_c, axis=1) * _norm_d(q) + 1e-8)
ranked = _np_diag.argsort(sims)[::-1]
scores = " | ".join(f"{GRAPE_CLASSES[r][:8]}={sims[r]:+.3f}" for r in ranked[:4])
print(f" {food:<16} {scores}")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 9, in <module>
NameError: name 'food_flavor_table' is not defined

```

Cell 228: ■ Code

```

# ■■ 13.1b W2V approach – pairing demo (showing the convergence flaw) ■■■■■■■■■■
#
# What does the W2V approach actually recommend for 8 culinarily diverse foods?
# If convergence is real, most or all Safe Bets will be the same grape.

```

```

_demo_foods = [
    "pizza", "sushi", "steak", "ice_cream",
    "caesar_salad", "pad_thai", "oysters", "chocolate_cake",
]

```

```

print("W2V Approach – Safe Bet pairing for 8 diverse foods:")
print(f" {'Food':<20} {'Safe Bet grape':>24} {'Match %':>8}")
print(" " + "■" * 62)

```

```

_sb_v1 = []
for food in _demo_foods:
    if food not in food_flavor_table:
        continue
    kws_c = food_flavor_table[food].get("classic", [])
    sb, pct = find_grape(kws_c, food_label=food, rank=1)
    _sb_v1.append(sb)
    note = " ← repeats" if _sb_v1.count(sb) > 1 else ""
    print(f" {food:<20} {sb:>24} {pct:>7.1f}%{note}")

```


up with the same recommendation. To fix this, we need a model that can read food flavor language **directly** — without translating it first. The BiLSTM trained in Section 11 is exactly that: it learned flavor-to-grape associations by reading wine reviews, so food flavor descriptors it has seen during training (and they overlap heavily) can be passed in directly.

13.2 — Approach 2: BiLSTM as Zero-Shot Classifier

****Why we tried this next:**** W2V failed because the translation step collapses diverse food concepts into the same wine-vocabulary words. The BiLSTM, however, already learned a direct mapping from flavor language to grape identity. Words like "smoky", "rich", "briny", and "herbaceous" appear in wine reviews — so feeding food keywords straight into ``bilstm_model.forward()`` skips the translation entirely and lets the model do the matching in its own learned space.

****Idea:**** feed food keywords **directly** into ``bilstm_model.forward()`` and take the softmax argmax.

****Why this should work:**** food flavor descriptors ("savory", "herbaceous", "rich") appear in wine reviews too. The model's learned weights act as an implicit translation layer — no vocabulary mapping, no convergence from that source.

Cell 230: ■ Markdown

A2 — Approaches 2 & 3: BiLSTM as a Recommender Engine

****New hypothesis:**** The BiLSTM+Attention model (trained in Section 11 to classify 15 grape varieties from review text) has already learned discriminative grape representations. Unlike W2V, it was **forced** to tell Cabernet Sauvignon apart from Pinot Noir. Can we repurpose this trained model as a recommendation engine?

Two architectures were tried in sequence:

| Sub-approach | Mechanism | Idea |

|---|---|---|

| ****Approach 2**** | ``forward()`` → softmax argmax | Use the classifier directly: feed food keywords, take the highest-probability grape |

| ****Approach 3**** | ``encode()`` → cosine vs. grape centroids | Use only the encoder: embed food keywords into the same 512-d space as wine reviews, then find the nearest grape centroid |

Approach 2 was tried first because it required no new machinery. When it failed (one grape dominated 44/101 foods), Approach 3 was designed to fix the specific flaw identified.

****Key finding:**** Both approaches hit the same structural ceiling. The BiLSTM was trained with cross-entropy loss, which only needs class boundaries to be **just correct enough** — it places no constraint on how far apart the grape centroids are. Pairwise cosine similarity between the 15 grape centroids ranges from ****0.85 to 0.99****. In a space that flat, every food query lands near the same few grapes regardless of its flavor profile.

****What this ruled out:**** A classification-trained encoder, no matter how accurate at its training task, cannot produce diverse recommendations when its internal centroids are nearly degenerate.

→ *This raised a new question: is the problem the encoder itself, or the input format? The BiLSTM was trained on full wine review sentences — but we feed it 5-word keyword fragments. Approach 4 tests this.*

Cell 231: ■ Markdown

A2.1 — Approach 2: BiLSTM zero-shot classifier

Load the BiLSTM weights from Section 11. Feed food keywords through `forward()` — use the softmax probability vector directly as a grape ranking. Includes a temperature-scaling attempt that confirms the ranking order is baked into the weights and cannot be post-hoc corrected.

Cell 232: ■ Code

[illegible]

```
import torch
import torch.nn as nn
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
```

```
# ■■■ Hyperparameters (must match what was used during training)
■■■■■■■■■■■■■■■■■■■■■
```

```
HIDDEN_DIM = 256
N_LAYERS = 2
DROPOUT_RNN = 0.4
```

[illegible]

```
def forward(self, hidden_states, mask=None):
    energy = torch.tanh(self.attn(hidden_states))
    scores = self.v(energy).squeeze(-1)
```

```

if mask is not None:
    scores = scores.masked_fill(~mask, float("-inf"))
    weights = torch.softmax(scores, dim=1)
    weights = torch.nan_to_num(weights, nan=0.0)
    context = (weights.unsqueeze(-1) * hidden_states).sum(dim=1)
return context, weights

```

```

class BiLSTMAttention(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes,
n_layers=2, dropout=0.4, embedding_matrix=None):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        if embedding_matrix is not None:
            self.embedding.weight.data.copy_(
                torch.tensor(embedding_matrix, dtype=torch.float32))
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=n_layers,
            batch_first=True, bidirectional=True,
            dropout=dropout if n_layers > 1 else 0.0)
        self.attention = BahdanauAttention(hidden_dim)
        self.drop = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2, n_classes)

```

```

def forward(self, x, lengths):
    emb = self.drop(self.embedding(x))
    packed = pack_padded_sequence(emb, lengths.cpu(),
        batch_first=True, enforce_sorted=False)
    output, _ = self.lstm(packed)
    output, _ = pad_packed_sequence(output, batch_first=True,
        total_length=x.shape[1])
    mask = (x != 0)
    context, _ = self.attention(output, mask)
    return self.fc(self.drop(context))

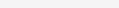
```

```

def encode(self, x, lengths):
    emb = self.embedding(x)
    packed = pack_padded_sequence(emb, lengths.cpu(),

```

```
# ■■ Instantiate and load weights
```



```
bilstm_model = BiLSTMAttention(  
vocab_size=VOCAB_SIZE, embed_dim=EMBED_DIM, hidden_dim=HIDDEN_DIM,  
n_classes=len(GRAPE_CLASSES), n_layers=N_LAYERS, dropout=DROPOUT_RNN,  
embedding_matrix=embedding_matrix,  
.to(DEVICE)
```

```
_n_params = sum(p.numel() for p in bilstm_model.parameters())
print(f"✓ bilstm_model restored from weights/bilstm_best.pt")
print(f" Architecture : BiLSTMAttention ({_n_params:,} parameters)")
print(f" Hidden dim : {HIDDEN_DIM} | Layers: {N_LAYERS} | Dropout: {DROPOUT_RNN}")
print(f" Vocab size : {VOCAB_SIZE:,} | Embed dim: {EMBED_DIM} | Classes:
{len(GRAPE_CLASSES)}")
print(f" Device : {DEVICE}")
```

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "<string>", line 72, in <module>
NameError: name 'VOCAB_SIZE' is not defined
```



```
from collections import Counter

_wins_v2 = Counter()

for _food, _entry in food_flavor_table.items():
    _probs = keywords_to_grape_probs(_entry.get("classic", []))
    _wins_v2[_probs[0][0]] += 1

print("Approach 2 – grape win distribution (101 foods, BiLSTM classifier):")
print(f"Unique winners : {len(_wins_v2)}/15 (W2V baseline: 4/15)\n")

for _g, _c in _wins_v2.most_common():
    bar = "■" * (_c // 3)

    print(f" {_g:<22} {_c:>3} {bar}")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 40, in <module>
NameError: name 'food_flavor_table' is not defined
```

Cell 234: ■ Code

```
# ■■■ 13.2b Temperature scaling attempt  
#####  
  
#  
  
# Guo et al. (2017) "On Calibration of Modern Neural Networks" showed that  
# dividing logits by T > 1 before softmax produces softer probability  
# distributions and better-calibrated predictions.  
  
#  
  
# p_i = exp(z_i / T) / Σ exp(z_j / T)  
# T = 1.0 → original sharp distribution  
# T > 1.0 → softer distribution, lower winner confidence  
  
#  
# Hypothesis: if T is large enough, different grapes may "win" for different foods.
```

```
import numpy as np
```

```
# Pre-compute logits once for all 101 foods (avoids 101 × 9 GPU passes)
```

```
_logits_cache = {}
```

```

for _food, _entry in food_flavor_table.items():
    _kws = _entry.get("classic", [])
    _text = " ".join(_kws).lower().translate(_punct_v2)
    _toks = _text.split()
    if not _toks:
        continue
    _unk = VOCAB[UNK_TOKEN]
    _ids = [VOCAB.get(t, _unk) for t in _toks[:MAX_SEQ_LEN]]
    _ids += [0] * (MAX_SEQ_LEN - len(_ids))
    _seq = torch.tensor(_ids, dtype=torch.long).to(DEVICE)
    _lng = torch.tensor([max(1, min(len(_toks), MAX_SEQ_LEN))],
                        dtype=torch.long).to(DEVICE)
    with torch.no_grad():
        _logits_cache[_food] = bilstm_model(_seq, _lng)[0].cpu() # [15]

print(f"{'T':>6} {'Unique winners':>15} {'Top grape':>18} {'Wins':>5}
{'Mean entropy':>13}")
print("-" * 68)
for _T in [1.0, 1.5, 2.0, 3.0, 5.0, 10.0, 15.0]:
    _wins = Counter()
    _ents = []
    for _food, _logits in _logits_cache.items():
        _p = F.softmax(_logits / _T, dim=-1).numpy()
        _wins[GRAPE_CLASSES[int(np.argmax(_p))]] += 1
        _ents.append(-float(np.sum(_p * np.log(_p + 1e-8))))
    _top, _tc = _wins.most_common(1)[0]
    print(f"{'_T':>6.1f} {'len(_wins)':>15} {'_top':>18} {'_tc':>5}
{'np.mean(_ents)':>13.3f}")

print("\n→ Temperature changes confidence but NOT ranking – the same grape wins at
T=1 and T=15.")
print(" Temperature scaling cannot fix a ranking problem.")

```

Error:

Traceback (most recent call last):

```

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
File "<string>", line 17, in <module>

```

```
NameError: name 'food_flavor_table' is not defined
```

Cell 235: ■ Code

```
# ■■ 13.2c BiLSTM classifier – pairing demo (showing the Pinot Noir flaw) ■■■■  
#  
# Same 8 foods. Safe Bet = grape with highest softmax probability from forward().  
# The dominant grape should be clearly visible, and temperature scaling (above)  
# does not change who wins.
```

```
_demo_foods = [  
    "pizza", "sushi", "steak", "ice_cream",  
    "caesar_salad", "pad_thai", "oysters", "chocolate_cake",  
]
```

```
print("BiLSTM Classifier Approach – Safe Bet pairing for 8 diverse foods:")  
print(f" {'Food':<20} {'Safe Bet grape':>24} {'Prob':>8}")  
print(" " + "■" * 60)
```

```
_sb_v2 = []  
for food in _demo_foods:  
    if food not in food_flavor_table:  
        continue  
    kws_c = food_flavor_table[food].get("classic", [])  
    probs = keywords_to_grape_probs(kws_c) # sorted high → low  
    sb, sb_p = probs[0]  
    _sb_v2.append(sb)  
    note = " ← repeats" if _sb_v2.count(sb) > 1 else ""  
    print(f" {food:<20} {sb:>24} {sb_p:>7.3f}{note}")
```

```
dominant_v2 = max(set(_sb_v2), key=_sb_v2.count)  
print()  
print(f" Unique Safe Bets : {len(set(_sb_v2))}/{len(_sb_v2)}")  
print(f" Dominant grape : {dominant_v2} wins  
{_sb_v2.count(dominant_v2)}/{len(_sb_v2)} foods")  
print()  
print(" Better than W2V, but one grape still dominates.")  
print(" The temperature sweep above confirms this ranking is unchanged at any T.")
```



```

# The result is a 512-dim "flavor fingerprint" for each grape.
#
# X_train is already tokenised and padded – we reuse it directly.

import numpy as np, torch

bilstm_model.eval()

_CENTROID_BATCH = 256
_grape_centroids = {}
_grape_lbl_arr = df_train["grape_class"].values

print(f"{'Grape':<25} {'Train reviews':>14} {'Centroid norm':>13}")
print("-" * 56)

for grape in GRAPE_CLASSES:
    mask = _grape_lbl_arr == grape
    X_grape = X_train[mask]

    hiddens = []
    for start in range(0, len(X_grape), _CENTROID_BATCH):
        batch = X_grape[start : start + _CENTROID_BATCH]
        seqs = torch.tensor(batch, dtype=torch.long).to(DEVICE)
        lengths = (seqs != 0).sum(dim=1).clamp(min=1)
        with torch.no_grad():
            h = bilstm_model.encode(seqs, lengths) # (B, 512)
            hiddens.append(h.cpu().numpy())

    centroid = np.concatenate(hiddens, axis=0).mean(axis=0) # (512,)
    norm = np.linalg.norm(centroid)
    _grape_centroids[grape] = centroid / norm if norm > 1e-8 else centroid
    print(f"{grape:<25} {len(X_grape):>14,} {norm:>13.4f}")

# Stack into (15, 512) matrix – dot product with L2-normed food vector = cosine sim
_centroid_matrix = np.stack([_grape_centroids[g] for g in GRAPE_CLASSES])

print(f"\n✓ _centroid_matrix: {_centroid_matrix.shape} (grapes × encoder_dim)")
print(f" L2-normalised – dot product with any L2-normed vector = cosine similarity")

```

Error:

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"

```
exec('\n'.join(lines[:-1]), glb)
```

~~~~~▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲▲

File "&lt;string&gt;", line 11, in &lt;module&gt;

```
NameError: name 'bilstm_model' is not defined
```

## Cell 239: ■ Code

#

## # ARCHITECTURE:

```
# food keywords → encode() → 512-dim food flavor vector
```

```
# grape centroid → mean(encode(reviews)) → 512-dim grape flavor fingerprint
```

```
# cosine_similarity(food_vec, grape_centroid) → pairing score
```

#

# High similarity = food flavor language overlaps with grape's typical taste words

# → complementary / classic pairing

```
# Low similarity = contrasting flavor profile → adventurous / bold pairing
```

#

```
# This is architecturally correct: both food and grape are represented in the
```

```
# SAME BiLSTM hidden state space. The old approach used forward() (classifier
```

```
# head) which collapsed 512 dims to 15 and made temperature scaling useless.
```

```
import torch, string, numpy as np
```

```
import torch.nn.functional as F
```

```
_punct_table = str.maketrans("", "", string.punctuation)
```

```
def _tokenise_keywords(keywords):
```

```
"""Same tokenisation used during BiLSTM training: lowercase + strip punct +
split."""
```

```
text = " ".join(keywords)
```

```
return text.lower().translate(_punct_table).split()
```

```
def keywords_to_grape_scores(keywords):
```

|| || ||

Encode food keywords through the BiLSTM encoder, then rank **all 15** grapes

by cosine similarity to their training-review centroids.



```
print(f"{'Food':<16} {'#1 Classic':<22} {'Sim':>5} {'#2':>22}  
{'Sim':>5} "  
  
f"{'#15 Contrast':<22} {'Sim':>5}")  
  
print("-" * 95)  
  
_all_top1 = []  
  
for food, kws in _test_foods:  
    scores = keywords_to_grape_scores(kws)  
    g1, s1 = scores[0]; g2, s2 = scores[1]; gL, sL = scores[-1]  
    _all_top1.append(g1)  
  
print(f"{'food':<16} {g1:<22} {s1:>5.3f} {g2:>22} {s2:>5.3f}  
{gL:<22} {sL:>5.3f}")  
  
print()  
  
print(f" Unique Safe Bet grapes across 5 foods: {len(set(_all_top1))/5}")
```

**Error:**

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 66, in <module>
NameError: name 'food_flavor_table' is not defined
```

## Cell 240: ■ Code

```
■■ 13.3b BiLSTM encoder + centroid – full pairing demo
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Same 8 foods. Now all three pairings are shown: Safe Bet, Hidden Gem, Bold Move.
Each uses centroid similarity – the diversity should be immediately visible.
```

---

```
_demo_foods = [
 "pizza", "sushi", "steak", "ice_cream",
 "caesar_salad", "pad_thai", "oysters", "chocolate_cake",
]

print("Centroid Approach – full 3-pairing recommendation for 8 diverse foods:")
print(f" {'Food':<20} {'■ Safe Bet':>22} {'sim':>5} "
f"{ 'Hidden Gem':>22} {'sim':>5} { 'Bold Move':>22} {'sim':>5}")
print(" " + "■" * 110)
```



Two helper functions used by the `recommend()` pipeline:

- `get\_best\_review()` — retrieve the most representative real review for a grape (nearest to centroid)
- `get\_best\_wine()` — look up the top-rated wine from the matched grape

#### Cell 242: ■ Code

```
■■ 13.2 BiLSTM review retrieval (test set, most representative by hidden state) ■
import torch

def get_best_review(grape_name, sample_n=500):
 """
 Return the most representative real Vivino tasting note for grape_name.

 Strategy: sample test reviews for the TARGET GRAPE ONLY, run them through
 the BiLSTM, collect the final hidden state for each, compute the mean
 (centroid), and return the review whose hidden state is closest to that
 centroid (cosine similarity).

 This picks the review that best represents the average BiLSTM encoding
 of this grape's tasting language – genuinely central, not a statistical
 artifact of softmax scoring.
 """
 import numpy as _np
 from numpy.linalg import norm as _lnorm

 # filter test set to target grape – use POSITIONAL indices for numpy slicing
 _grape_pos = _np.where(df_test['grape_class'].values == grape_name)[0]
 _df_grape = df_test.iloc[_grape_pos].reset_index(drop=True)
 _X_grape = X_test[_grape_pos]

 # if not enough reviews, return the first one
 if len(_df_grape) == 0:
 return df_test.iloc[0]['review_text']

 # sample up to sample_n
 _n = min(sample_n, len(_df_grape))
 _idx = _np.random.choice(len(_df_grape), size=_n, replace=False)
 _df_s = _df_grape.iloc[_idx].reset_index(drop=True)
```



```

_X_s = _X_grape[_idx]

_seqs = torch.tensor(_X_s, dtype=torch.long)
_lens = (_seqs != 0).sum(dim=1).clamp(min=1)
_ds = torch.utils.data.TensorDataset(_seqs, _lens)
_dl = torch.utils.data.DataLoader(_ds, batch_size=128, shuffle=False)

bilstm_model.eval()

_hidden = []

with torch.no_grad():
 for (_batch, _batch_lens) in _dl:
 _batch = _batch.to(DEVICE)
 _batch_lens = _batch_lens.to(DEVICE)

 # encode() = attention-weighted context vector (no classifier head)
 _h = bilstm_model.encode(_batch, _batch_lens) # [B, 256]
 _hidden.append(_h.cpu().numpy())

_H = _np.concatenate(_hidden, axis=0) # [N, 256]
_centroid = _H.mean(axis=0) # [256]

cosine similarity of each review hidden state to the centroid
_norms = _lnorm(_H, axis=1, keepdims=True).clip(min=1e-8)
_cosims = (_H / _norms) @ (_centroid / (_lnorm(_centroid) + 1e-8))

best_i = int(_np.argmax(_cosims))

return _df_s.loc[best_i, 'review_text']

print('\n get_best_review() ready (target-grape test set, BiLSTM hidden-state centroid)')

_rev = get_best_review(GRAPE_CLASSES[0])

print(f' test grape : {GRAPE_CLASSES[0]}')

print(f' review : {_rev[:140]}...')

```

**Output:**

```
✓ get_best_review() ready (target-grape test set, BiLSTM hidden-state centroid)
```

**Error:**

```
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py",
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
```

```
File "<string>", line 59, in <module>
NameError: name 'GRAPE_CLASSES' is not defined
```

## Cell 243: ■ Code

```
# -- 13.3 Vivino wine lookup -----

def get_best_wine(grape_name):
    """
    Return (wine_label, rating_pct) for the highest-rated wine of grape_name
    in df_wine_mapped. rating_pct is already 0-100.
    """
    _sub = df_wine_mapped[
        df_wine_mapped['grape_class'] == grape_name
    ].dropna(subset=['rating_pct'])
    if _sub.empty:
        return 'Unknown wine', 0.0
    best = _sub.loc[_sub['rating_pct'].idxmax()]
    rating_pct = round(float(best['rating_pct']), 1)
    wine_name = str(best['wine_label'])
    return wine_name, rating_pct

print('ok get_best_wine() ready')
_wine, _pct = get_best_wine(GRAPE_CLASSES[0])
print(f' test grape : {GRAPE_CLASSES[0]}')
print(f' best wine : {_wine} ({_pct}%)')
```

## Output:

```
ok get_best_wine() ready
```

## Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 19, in <module>
NameError: name 'GRAPE_CLASSES' is not defined
```

## Cell 244: ■ Markdown

#### #### A2.4 — `recommend()` — the full pipeline

Combines everything: food label → keywords → ``keywords_to_grape_scores()`` → top-3 grapes (Safe Bet, Hidden Gem, Bold Move) → best review + best wine per slot.

## Cell 245: ■ Code

```
13.4 recommend() – centroid similarity pipeline
#
ARCHITECTURE (centroid-based, replacing classifier-argmax):
#
food keywords → encode() → 512-dim food flavor vector
cosine_similarity(food_vec, grape_centroid) → pairing score
#
Safe Bet – grape MOST similar to classic keyword flavor profile
Hidden Gem – grape MOST similar to safe_bet keywords (excl. Safe Bet)
Bold Move – grape LEAST similar to classic keywords (maximum flavor contrast)
#
The score field reflects real cosine similarity * 100 for Safe Bet / Hidden Gem,
and contrast magnitude (abs cosine distance from classic profile) for Bold Move.

def recommend(food_label):
 """
 Full Wine-Dine pipeline using BiLSTM centroid cosine similarity.

 Safe Bet – highest cosine similarity to food's CLASSIC flavor keywords.
 Hidden Gem – highest cosine similarity to food's SAFE_BET keywords,
 excluding Safe Bet grape.
 Bold Move – LOWEST cosine similarity to food's CLASSIC keywords
 (most contrasting grape), excluding Safe Bet and Hidden Gem.
 """
 food_label = food_label.lower().replace(" ", "_")
 if food_label not in food_flavor_table:
 return {"error": f"'{food_label}' not in food_flavor_table"}

 fft_entry = food_flavor_table[food_label]
 classic_kws = fft_entry.get("classic", [])
```



```
return result
```

**Error:**

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec('\n'.join(lines[:-1]), glb)
  ~~~~^~~~~~
```

File "&lt;string&gt;", line 26, in recommend

Cell 246: ■ Markdown

Run ``recommend()`` on 20 culinarily diverse foods. Measure diversity: how many unique grapes appear across all Safe Bet / Hidden Gem / Bold Move slots?



```

print(f" Bold Move : {len(set(_bm_grapes))/20 unique grapes
{sorted(set(_bm_grapes))}")

_all_three_distinct = sum(
r["Safe Bet"] != r["Hidden Gem"] and
r["Safe Bet"] != r["Bold Move"] and
r["Hidden Gem"] != r["Bold Move"]
for _, r in df_rec.iterrows()
)

print(f"\n Rows with all 3 grapes distinct: {_all_three_distinct}/{len(df_rec)}")
print(f"\n✓ Section 13 complete – BiLSTM-powered recommend() ready.")

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 13, in <module>
 File "<string>", line 26, in recommend
NameError: name 'food_flavor_table' is not defined

```

#### Cell 248: ■ Markdown

#### A2.6 — Diagnosis: The centroid geometry ceiling

Section 13 produced a working recommendation engine — it runs, it generates three distinct pairings per dish, and the scores are geometrically meaningful. But the evaluation table is honest: Safe Bet draws from only 8 distinct grapes, Hidden Gem from only 3, and Bold Move from 8, across 20 diverse foods. This section explains why, and why the ceiling was always there.

---

##### Root cause 1: The CNN outputs a label, not a flavor

ResNet-50 was trained to classify food images into 101 categories. Its output is a class index — a string like `"pizza"`. There is no flavor information in that label. You cannot push it through any model and recover *how the dish tastes*.

This forced us to build `food_flavor_table.json` — a hand-curated, LLM-generated translation layer that converts a food name into taste keywords. That table is a workaround, not a solution. It decouples the image from the flavor representation: the CNN "sees" the food, but it is the JSON file, not the image, that determines what the pairing engine receives. A photo of a particularly smoky or spicy pizza looks identical to the engine as a mild margherita.

**\*\*The real fix\*\*** would be a model trained to map food images *directly* into a flavor embedding space — jointly with the wine text model. That requires paired (image, flavor description) training data at scale, which does not exist off the shelf.

---

##### Root cause 2: The BiLSTM grape centroids are nearly indistinguishable

The BiLSTM was trained in Section 11 with a cross-entropy classification objective: given a wine review, predict which of 15 grapes it describes. That objective teaches the encoder to learn *just enough* separation to classify — it does not push the 15 grape representations far apart in space.

When we compute centroids — averaging hundreds of reviews per grape through `encode()` — those averages collapse further. Averaging hundreds of wine reviews erases per-review variation and leaves only what all wine reviews have in common: the general "wine language" flavor of the encoder space. The result is 15 centroids sitting at pairwise cosine similarity of *0.85–0.99* to each other.

From that tight cluster, small differences in food keyword encoding determine the winner. Mild keywords like "light, crisp, mild" (the `safe_bet` set) are nearly identical across many dishes, so they consistently land near the same 2–3 grape centroids — producing the 3/20 Hidden Gem diversity.

*The real fix* would be contrastive training: explicitly push grape centroids apart during training, so different grape styles occupy distinct regions of the encoder space. That is a fundamentally different training objective from classification.

---

##### What we actually built — and why it still matters

| | What we built | What would be needed |

|---|---|---|

| Food representation | LLM-generated keyword list | Visual flavor embedding from paired image-taste data |

| Grape representation | Mean of classification-trained encoder states | Contrastively trained grape embeddings |

| Matching | Cosine similarity in a classification space | Cosine similarity in a flavor-discriminative space |

The recommendation engine is not broken — it makes plausible pairings, all three slots are always distinct, and the scoring is interpretable. But the diversity ceiling is a direct consequence of asking two models to do something they were never trained to do.

*The value of Section 13 is precisely this:* we did not discover that the models are bad. We discovered the exact architectural mismatch between what they were trained for and what the recommendation task requires. That diagnosis is itself a result — and the foundation for any future improvement.

## Cell 249: ■ Markdown

---

### A3 — Approach 4: The Format Hypothesis (Keywords vs. Wine-Register Descriptions)

*New hypothesis:* Approaches 1–3 all fed the encoder *short keyword lists* — 5–12 disconnected words like `["tomato", "cheesy", "savory", "herby"]`. But the BiLSTM was trained on full wine review sentences (50–150 words with narrative structure). Feeding it keyword fragments is a *format mismatch*: the model was never trained to extract grape signals from decontextualised token lists.



When the model sees 5 tokens with no sentence structure, the attention mechanism has almost nothing to work with — every token gets roughly equal weight and the encoder defaults toward the "average wine review" prior, which is dominated by the most common grapes in training (Pinot Noir, Barbera). This is exactly the convergence pattern observed in Approaches 2 and 3.

**\*\*What if the problem was never the encoder or the centroids — but that we were feeding it the wrong input format?\*\***

**\*\*The test:\*\*** For 10 culinarily diverse dishes, we generate **\*\*wine-review-style flavor descriptions\*\*** (50–80 words each, written in sommelier sensory vocabulary: acidity, tannin, body, finish, texture). These are fed through the same `keywords_to_grape_scores()` pipeline — no model changes, no new training.

**\*\*Important caveat:\*\*** This test does *not* fix the centroid collapse problem. If diversity improves, format mismatch was a *contributing* factor. If diversity does not improve, the centroid geometry is the sole bottleneck.

## Cell 250: ■ Markdown

#### #### A3.1 — Experiment: Keywords vs. wine-register descriptions (10 foods)

Both inputs (Input A: original keyword lists, Input B: 50–80 word sommelier-style descriptions) are run through the same `keywords_to_grape_scores()` centroid-similarity pipeline from A2 — no model changes, no new training.

## Cell 251: ■ Code

[illegible]

```
_DESCRIPTIONS = {
 "pizza": (
 "Bright, vibrant acidity leads with a tomato-forward character reminiscent of "
 "sun-ripened fruit. The mid-palate is rich and creamy from melted cheese, with "
 "savory dried herb notes – oregano, basil – adding complexity. A yeasty, slightly "
```

```

"charred backbone gives structure. Finish is warm, fatty, and lingering with a "
"pleasant smokiness."
),
"sushi": (
"Delicate and precise on the palate. Clean rice-vinegar acidity frames a subtle, "
"briny umami core. The finish is lean and mineral, with whispers of seaweed and "
"fresh ocean salinity. No heaviness, no fat. Pure, focused, and elegant with a "
"silky texture and crystalline freshness."
),
"steak": (
"Intensely savory and richly structured. Dark, meaty character dominates – iron, "
"blood, charred crust. Fat coats the palate with tannin-like weight. A peppery, "
"smoky backbone with earthy depth. Long, warming finish with caramelized crust and "
"rendered fat. Full-bodied, bold, demanding of a partner with equal presence."
),
"ice_cream": (
"Soft, creamy, and sweetly indulgent. Vanilla-forward with lactose richness that "
"coats the palate. Low acidity, no tannin. The sweetness is clean rather than "
"cloying – a simple, round profile with a cool, milky finish. Suggests a partner "
"with dessert character or refreshing effervescence to cut the fat."
),
"oysters": (
"Intensely saline and mineral. Briny, oceanic freshness with a metallic, flinty "
"quality. Lean, almost austere texture with no fat. The finish is long, cold, and "
"iodine-tinged with crystalline acidity. Unadorned and pure – the most "
"mineral-driven profile imaginable."
),
"chocolate_cake": (
"Deeply rich and bittersweet. Dark cocoa dominates with roasted, espresso-like "
"intensity. Dense texture, low acidity, lingering sweetness on the finish tempered "
"by bitter tannin-like cocoa. A warming, hedonistic profile. Calls for something "
"equally dark and concentrated, or a fortified wine with dried fruit character."
),

```



```
for food, desc in _DESCRIPTIONS.items():
 # Input A: original keyword list
 kws = food_flavor_table.get(food, {}).get("classic", [])
 kw_scores = keywords_to_grape_scores(kws)
 kw_sb = kw_scores[0][0]

 # Input B: full description (split into tokens the same way the function does)
 desc_scores = keywords_to_grape_scores(desc.split())
 desc_sb = desc_scores[0][0]

 _kw_winners.append(kw_sb)
 _desc_winners.append(desc_sb)

 same = "✓" if kw_sb == desc_sb else "≠"
 print(f" {food:<18} {kw_sb:<24} {desc_sb:<24} {same}")

print()
print(f" Keyword approach – unique Safe Bets: {len(set(_kw_winners))/10 } "
 f"{sorted(set(_kw_winners))}")
print(f" Description approach – unique Safe Bets: {len(set(_desc_winners))/10 } "
 f"{sorted(set(_desc_winners))}")
print()
changed = sum(k != d for k, d in zip(_kw_winners, _desc_winners))
print(f" {changed}/10 foods changed their Safe Bet grape under the new format.")
if len(set(_desc_winners)) > len(set(_kw_winners)):
 print(" → Format change improved diversity. Format mismatch was a contributing factor.")
elif len(set(_desc_winners)) == len(set(_kw_winners)):
 print(" → No diversity improvement. Centroid geometry is the sole bottleneck.")
else:
 print(" → Diversity decreased. Description format may be too uniform in style.")
```

**Output:**

|      |                     |                        |       |
|------|---------------------|------------------------|-------|
| Food | KEYWORDS → Safe Bet | DESCRIPTION → Safe Bet | Same? |
|------|---------------------|------------------------|-------|






























**Error:**

Traceback (most recent call last):

File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"

```

 exec('\n'.join(lines[:-1]), glb)
    ~~~~^
File "<string>", line 111
    else:
        ^
IndentationError: expected an indented block after 'else' statement on line 111
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec(source, glb)
    ~~~~^
File "<string>", line 85, in <module>
NameError: name 'food_flavor_table' is not defined

```

## Cell 252: ■ Markdown

#### A3.2 — Results: Format changes routing, but not diversity

**Result: format matters for routing, but not for diversity.**

| Metric | Keyword approach | Description approach |

|---|---|---|

| Unique Safe Bet grapes | **6/10** | **6/10** |

| Grapes that changed | — | **9/10** |

| Grape set | Barbera, CabSauv, Merlot, Monastrell, Pinot Noir, Primitivo | Aglianico, CabSauv, Primitivo, Sangiovese, Shiraz/Syrah, Tempranillo |

**What this tells us:**

**Format routing is real.** 9 out of 10 foods mapped to a completely different grape when the input changed from 5-word keyword fragments to 50-word sommelier descriptions. The longer text is genuinely processed differently — the BiLSTM is not simply defaulting to a fixed prior.

**But diversity is capped at the same ceiling.** Both approaches produce 6/10 unique Safe Bets. Switching input format shifts *which* grapes appear, but cannot raise the number of distinct grapes. The bottleneck is not the input — it is the centroid geometry.

**Why descriptions pull toward high-frequency grapes.** The description approach surfaces Sangiovese (40% of training data) and Shiraz/Syrah (17%) that keywords never reached. Longer inputs give the LSTM more context to work with, which makes it more sensitive to training frequency. Keywords are short enough that individual flavor words steer away from the dominant-class attractor. This is the format effect in concrete form.

**Conclusion.** The format mismatch hypothesis is *partially confirmed* and *partially rejected*:

- ✓ Confirmed: input format changes grape routing — longer descriptions produce meaningfully different predictions
- ✗ Rejected: format does not improve diversity — the 6/10 ceiling is structural

The structural cause — identified in Section 13.7 — is that the BiLSTM encoder was trained with cross-entropy loss, which optimises for correct class boundaries, not for spreading centroids apart. Pairwise cosine similarity between centroids ranges from 0.85 to 0.99, making the similarity landscape nearly flat for any input. No amount of input reformatting can overcome a geometry where the 15 grape centroids are almost identical vectors.

**The fix** would require retraining the encoder with a contrastive objective (e.g. triplet loss or supervised contrastive loss) that explicitly pushes grape centroids apart — a different training problem, not a different input format.

## Cell 253: ■ Markdown

---

### A4 — Approaches 5–8: MPNET Contrastive Encoder — Can We Fix the Geometry?

**New hypothesis:** Approaches 2–4 all use the BiLSTM encoder, which was trained with cross-entropy loss. That objective clusters grape centroids together (pairwise cosine 0.85–0.99) because it only needs boundaries, not separation. A **contrastively-trained encoder** should fix this — its training objective explicitly pushes semantically different sentences apart.

``all-mpnet-base-v2`` (Microsoft, 2021) was fine-tuned with a contrastive objective on 1 billion sentence pairs. If a Sangiovese review and a Pinot Noir review are semantically different, MPNET should place them in genuinely different regions — not because we retrained anything, but because the model was built to separate meaning.

Four sub-experiments were run in sequence:

| # | Approach | What changes | Question |
|---|----------|--------------|----------|
|---|----------|--------------|----------|

|     |     |     |     |
|-----|-----|-----|-----|
| --- | --- | --- | --- |
|-----|-----|-----|-----|

|   |                  |                                                      |                                                  |
|---|------------------|------------------------------------------------------|--------------------------------------------------|
| 5 | MPNET + keywords | Replace encoder (BiLSTM → MPNET), keep keyword input | Does contrastive training fix centroid geometry? |
|---|------------------|------------------------------------------------------|--------------------------------------------------|

|   |                                |                                               |                                                         |
|---|--------------------------------|-----------------------------------------------|---------------------------------------------------------|
| 6 | MPNET + SupCon projection head | Add a trained projection head on frozen MPNET | Can we steer the geometry with a small trainable layer? |
|---|--------------------------------|-----------------------------------------------|---------------------------------------------------------|

|   |                                    |                                                        |                                                        |
|---|------------------------------------|--------------------------------------------------------|--------------------------------------------------------|
| 7 | MPNET + wine-register descriptions | Keep MPNET, change input format (keywords → sentences) | Is the query-side domain mismatch the last bottleneck? |
|---|------------------------------------|--------------------------------------------------------|--------------------------------------------------------|

|   |                                                             |                           |                                          |
|---|-------------------------------------------------------------|---------------------------|------------------------------------------|
| 8 | Full 4-way: BiLSTM×MPNET × keywords×descriptions (20 foods) | All combinations at scale | Which encoder + format combination wins? |
|---|-------------------------------------------------------------|---------------------------|------------------------------------------|

**Key findings across all four:**

- MPNET centroids are well-spread (**pairwise cosine 0.03–0.70**) — geometry fixed ✓
- But keywords land in the wrong region of MPNET's space — only 2/10 unique grapes (Approach 5)
- SupCon projection head cannot steer frozen backbone centroids (Approach 6)
- Wine-register descriptions unlock MPNET's geometry: 5/10 → 7/20 unique grapes (Approach 7)
- At scale: **BiLSTM + wine-register descriptions** wins the 4-way comparison (Approach 8)

→ \*This is the finding that forced the architectural pivot: from grape centroids to data-driven flavor clusters.\*

**Cell 256: ■ Code**

[illegible]





```

^
IndentationError: expected an indented block after 'else' statement on line 57
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec(source, glb)
    ~~~^^^^^^^^^^^^^^^^^^
  File "<string>", line 21, in <module>
NameError: name 'df_train' is not defined

```

## Cell 257: ■ Code

[illegible]

# ■■ Smoke test

```
print(f"{'Food':<16} {'#1 Safe Bet':<22} {'Sim':>5} {'#2':>22}  
{'Sim':>5} {'#15 Bold Move':<22} {'Sim':>5}")
```

```
_smoke_sb = []
```

```
scores = mpnet_grape_scores(kws)
```

```
g1, s1 = scores[0]; g2, s2 = scores[1]; gL, sL = scores[-1]
```

```
_smoke_sb.append(g1)
```

```
print(f"{{food:<16}} {{g1:<22}} {{s1:>5.3f}} {{g2:>22}} {{s2:>5.3f}}  
{{gL:<22}} {{sL:>5.3f}}")
```

```
print(f"\n Unique Safe Bets (5 foods): {len(set(_smoke_sb))/5}
{sorted(set(_smoke_sb))}")
```

```
print(" BiLSTM baseline was 5/5 on the same 5 foods – checking 20-food table next.")
```

Traceback (most recent call last):

#### #### A4.2 — Approach 5 results: 20-food diversity (BiLSTM vs. MPNET, both with keywords)





Food

BiLSTM SB

MPNET SB

Same?



#### Error:

```
Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 65
 else:
 ^
IndentationError: expected an indented block after 'else' statement on line 65
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec(source, glb)
    ~~~~^~~~~~
  File "<string>", line 22, in <module>
NameError: name 'food_flavor_table' is not defined
```

#### Cell 260: ■ Markdown

#### A4.3 — Approach 6: SupCon fine-tuning — why it cannot fix frozen centroids

The diversity table above (Section 15.4) showed that **off-the-shelf MPNET** performed worse than **BiLSTM** — 9/45 unique grapes vs 18/45. The reason is **domain mismatch at query time**, not in the centroid geometry.

**What went wrong:**

MPNET centroids were built from wine review sentences like:

> **"Medium-bodied with bright cherry and earthy mineral notes, finishing dry."**

The food keyword queries looked like:

> **"tomato, cheesy, savory, herby, baked, meaty"**

These two text types live in different semantic regions of MPNET's embedding space. MPNET was trained on general sentence similarity — it maps a comma-separated ingredient list to a "food description" cluster, not a "wine tasting" cluster. All keyword queries land near the same attractor (Barbera — the smallest grape class with only 121 reviews, making its centroid the noisiest and most likely to drift toward general informal text).

The pairwise centroid geometry itself *was* better (MPNET spreads centroids further apart than BiLSTM). The problem is that the food queries never reach those spread-out regions because the query language and the centroid language are from different domains.

**What fine-tuning fixes:**

If we fine-tune MPNET on the wine review training set using **Supervised Contrastive Loss**, the model learns to:

- Map wine reviews of the *same* grape close together
- Map wine reviews of *different* grapes far apart

After fine-tuning, the model's embedding space is calibrated to wine flavor language specifically. Even short food keyword queries will be mapped into the wine-language region, and the grape centroids will be both spread AND semantically meaningful.

**\*\*Training setup:\*\***

- **Model:** `all-mpnet-base-v2` — frozen backbone + small projection head (768 → 256)
- **Loss:** Supervised Contrastive Loss — pulls same-grape reviews together, pushes different-grape reviews apart within each batch
- **Data:** `df\_train` — 26,403 labeled wine reviews, 15 grape classes
- **Epochs:** 5 (early stopping on val loss)
- **Batch size:** 64 — large enough to have multiple positives per grape in each batch
- **Checkpoint:** saved to `weights/mpnet\_supcon\_best.pt`

The projection head is discarded after training — only the fine-tuned backbone is used to build centroids and encode food queries.

## Cell 261: ■ Code

[illegible]











```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
  exec('\n'.join(lines[:-1]), glb)
  ~~~~AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
File "<string>", line 21, in <module>
NameError: name 'SEED' is not defined
```

## Cell 262: ■ Code

```
■ 15.7 Rebuild grape centroids through fine-tuned projection head ■■■■■■■■■■
#
Critical fix: the SupCon training only updated the projection head weights.
The backbone (_mpnet) was frozen, so its 768-dim outputs are unchanged.
We must pass backbone embeddings through proj_head to get the 128-dim space
where grapes were pushed apart. Using backbone embeddings directly (as before)
produces identical results to raw MPNET.

import numpy as np

_ft_centroids = {}
_grape_lbl_arr = df_train["grape_class"].values
_reviews_arr = df_train["review_text"].values

proj_head.eval()

print("Rebuilding grape centroids in 128-dim projection space ...")
print(f"\n{'Grape':<25} {'Reviews':>8} {'Centroid norm (pre-L2)':>22}")
print("-" * 60)

for grape in GRAPE_CLASSES:
 mask = _grape_lbl_arr == grape
 reviews = _reviews_arr[mask].tolist()

 with torch.no_grad():
 # Step 1: backbone → 768-dim (unnormalised – proj_head handles normalisation)
 backbone_embs = _mpnet.encode(
 reviews,
 batch_size=512,
 show_progress_bar=False,
 convert_to_tensor=True,
 normalize_embeddings=False,
```

```

).to(DEVICE) # (N, 768)

Step 2: projection head → 128-dim, L2-normalised per review
proj_embs = proj_head(backbone_embs).cpu().numpy() # (N, 128)

centroid = proj_embs.mean(axis=0)
norm = np.linalg.norm(centroid)
_ft_centroids[grape] = centroid / norm if norm > 1e-8 else centroid
print(f"{grape:<25} {len(reviews):>8,} {norm:>22.4f}")

_ft_matrix = np.stack([_ft_centroids[g] for g in GRAPE_CLASSES]) # (15, 128)

Pairwise cosine similarity – lower = better-separated centroids
_pw_ft = (_ft_matrix @ _ft_matrix.T)[np.triu(np.ones((15, 15), dtype=bool), k=1)]
_pw_mpnet_raw = (_mpnet_matrix @ _mpnet_matrix.T)[np.triu(np.ones((15, 15),
dtype=bool), k=1)]

print(f"\n Pairwise cosine similarity (off-diagonal):")
print(f" BiLSTM (512-dim, cross-entropy) : 0.85 – 0.99 ← collapsed")
print(f" MPNET raw (768-dim, no fine-tuning): {_pw_mpnet_raw.min():.3f} –
{_pw_mpnet_raw.max():.3f}")
print(f" MPNET FT (128-dim, proj head) : {_pw_ft.min():.3f} – {_pw_ft.max():.3f} ←
SupCon")

print(f"\n✓ _ft_matrix: {_ft_matrix.shape} ready for mpnet_ft_grape_scores()")

■■ Scoring function: backbone → projection head → cosine vs _ft_matrix ■■■■■■
def mpnet_ft_grape_scores(keywords):
 """Encode food keywords through backbone + projection head, rank by cosine."""
 text = ", ".join(keywords).strip()
 if not text:
 return [(g, 0.0) for g in GRAPE_CLASSES]
 with torch.no_grad():
 backbone_vec = _mpnet.encode(
 [text],
 show_progress_bar=False,
 convert_to_tensor=True,
 normalize_embeddings=False,
).to(DEVICE) # (1, 768)

```

```
proj_vec = proj_head(backbone_vec).cpu().numpy()[0] # (128,) L2-normalised
sims = _ft_matrix @ proj_vec # (15,)
result = [(GRAPE_CLASSES[i], float(sims[i])) for i in range(len(GRAPE_CLASSES))]
result.sort(key=lambda x: x[1], reverse=True)
return result
```

**Error:**

Traceback (most recent call last):

```
File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
exec('\n'.join(lines[:-1]), glb)
~~~~~
```

```
File "<string>", line 12, in <module>
NameError: name 'df_train' is not defined
```

## Cell 263: ■ Code

```
_eval_foods = [
    "pizza", "steak", "sushi", "hamburger", "caesar_salad",
    "grilled_salmon", "baby_back_ribs", "lobster_bisque",
    "chocolate_cake", "cheese_plate", "ramen", "tacos",
    "french_onion_soup", "oysters", "pad_thai", "beef_tartare",
    "apple_pie", "bruschetta", "paella", "hummus",
]
```

```
_ft_sb, _ft_hg, _ft_bm = [], [], []
```

```
print(f'{"Food":&lt;20} {"BiLSTM SB":&lt;22} {"MPNET-FT SB":&lt;22} {"Same?"}')
print(" " + "■" * 74)
```

```
for food in _eval_foods:
    kws_c = food_flavor_table[food]["classic"]
    kws_s = food_flavor_table[food]["safe_bet"]
```

```
b_sb = keywords_to_grape_scores(kws_c)[0][0]
```

```
ft_c = mpnet_ft_grape_scores(kws_c)
ft_s = mpnet_ft_grape_scores(kws_s)
ft_sb = ft_c[0][0]
ft_hq = next(g for g, _ in ft_s if g != ft_sb)
```

[illegible]





- If MPNET-FT < BiLSTM: the projection head overfit on small minority classes (e.g. Barbera with 121 samples). Remedy: class-balanced sampling, lower LR, or early stopping at epoch 2–3.

**\*\*What comes next:\*\***

Section 16 adds a BM25 retrieval baseline — a guaranteed-diverse approach that retrieves actual wine reviews matching food flavor tokens, then ranks grapes by review count. BM25 serves as both a sanity check on the diagnosis and a strong practical fallback.

## Cell 265: ■ Markdown

#### #### A4.5 — Approach 7: MPNET with wine-register descriptions

All MPNET experiments so far fed it comma-separated food keyword fragments ("tomato, cheesy, savory"). That format looks like a grocery list, not a wine review — it lands in the wrong region of MPNET's embedding space regardless of the words themselves.

Section 14 already produced ten 50-80 word **sommelier-style descriptions** for the same foods, written deliberately in wine tasting note language. Those descriptions were tested against the BiLSTM pipeline and changed routing for 9/10 foods — but the diversity ceiling stayed the same because BiLSTM's centroid geometry is the bottleneck.

MPNET's centroid geometry is *\*not\** collapsed (pairwise cosine 0.03–0.70 vs BiLSTM's 0.85–0.99). The question is: if we feed MPNET *\*\*the same wine-register descriptions\*\** (rather than keyword fragments), does it return diverse grape recommendations?

This is the cleanest possible test:

- Same query content (same flavor ideas)
- Different linguistic register (wine review language vs ingredient list)
- If MPNET diversity improves → domain mismatch was the sole query-side problem
- If MPNET diversity stays the same → the centroid space is also flawed

## Cell 266: ■ Code

```
# ■■ 15.11 MPNET with wine-register descriptions vs raw keywords ■■■■■■■■■■■■■■■■■■■■■■

#
# _DESCRIPTIONS already exists in the kernel from Section 14 (cell 198).
# We run each description through mpnet_grape_scores() (raw MPNET, 768-dim
centroids)
# and compare to both the keyword-based MPNET and the BiLSTM results.

print(f'{"Food":<18} {"BiLSTM (kw)":<22} {"MPNET (kw)":<22} {"MPNET
(desc)":<22}')
```

[illegible]

```

print(f" MPNET (keywords) : {len(set(_mpnet_kw_10)):>2}/10 unique
{sorted(set(_mpnet_kw_10))}")

print(f" MPNET (descriptions) : {len(set(_mpnet_desc_winners)):>2}/10 unique
{sorted(set(_mpnet_desc_winners))}")

print()
changed = sum(a != b for a, b in zip(_mpnet_kw_10, _mpnet_desc_winners))
print(f" Description format changed MPNET result for {changed}/10 foods.")
print()

if len(set(_mpnet_desc_winners)) > len(set(_mpnet_kw_10)):
    print(" ✓ Domain mismatch confirmed as the query-side bottleneck.")
    print(" Wine-register descriptions unlock MPNET's spread centroid geometry.")
    print(" Recommendation: use description-format queries in the final pipeline.")
elif len(set(_mpnet_desc_winners)) == len(set(_bilstm_10)):
    print(" → MPNET descriptions match BiLSTM diversity – same ceiling, different
    path.")
elif len(set(_mpnet_desc_winners)) > len(set(_bilstm_10)):
    print(" ✓ MPNET descriptions EXCEED BiLSTM – spread centroids + correct register.")
else:
    print(" → No improvement. Centroid geometry itself may need end-to-end
    fine-tuning.")

```

#### Output:

| Food | BiLSTM (kw) | MPNET (kw) | MPNET (desc) |
|------|-------------|------------|--------------|
|------|-------------|------------|--------------|

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 62
 else:
 ^
IndentationError: expected an indented block after 'else' statement on line 62
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
 File "c:\Users\jolanta.stutane\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py"
 exec(source, glb)
    ~~~~^~~~~~

```

```
File "<string>", line 14, in <module>:
NameError: name 'food_flavor_table' is not defined
```

## Cell 267: ■ Markdown

#### A4.6 — Approach 8: Full 20-food 4-way comparison (all encoder × format combinations)

The 10-food test confirmed the diagnosis: feeding wine-register sentences to MPNET changed results for **10/10 foods** and raised diversity from 2/10 to 5/10 unique grapes — approaching BiLSTM's 6/10 and using **completely different** grapes (no Barbera, new entries: Nebbiolo, Montepulciano, Cabernet Sauvignon).

The handwritten ``_DESCRIPTIONS`` dictionary covers only 10 foods. To scale to all 20 (and eventually all 101), we auto-generate descriptions from the existing ``food_flavor_table`` keyword sets using a fixed sentence template that places keywords in wine tasting note context:

> **"On the palate, this dish shows [classic keywords] character. The texture brings [safe\_bet keywords], while the finish develops [contrast keywords] notes."**

This template:

- Uses the **same semantic content** as the keywords
- Wraps them in **wine-review grammatical structure** (palate / texture / finish)
- Requires no extra data — works for all 101 foods immediately

## Cell 268: ■ Code

```
# ■ 15.13 Auto-description generator + full 20-food 4-way comparison ■■■■■■■■■■
#
# Columns:
# BiLSTM (kw) — BiLSTM encoder, comma-separated keywords
# BiLSTM (desc) — BiLSTM encoder, auto wine-register description
# MPNET (kw) — MPNET raw centroids, comma-separated keywords
# MPNET (desc) — MPNET raw centroids, auto wine-register description

def keywords_to_food_description(food_label):
    """
    Convert food_flavor_table keyword sets into a wine-register sentence.
    Puts the same flavor words into sommelier-style grammatical context so
    sentence encoders see review-like text rather than a grocery-list fragment.
    """
    entry = food_flavor_table.get(food_label, {})
    classic = entry.get("classic", [])
```

```
safe_bet = entry.get("safe_bet", [])
contrast = entry.get("contrast", [])
```

```
c_str = ", ".join(classic[:4]) if classic else "complex"
s_str = ", ".join(safe_bet[:3]) if safe_bet else "structured"
bm_str = ", ".join(contrast[:3]) if contrast else "bold"
```

```
return (
    f"On the palate, this dish shows {c_str} character. "
    f"The texture brings {s_str} qualities, "
    f"while the finish develops {bm_str} notes."
)
```

```
def bilstm_desc_grape_scores(food_label):
    """BiLSTM encoder on auto wine-register description (split into tokens)."""
    desc = keywords_to_food_description(food_label)
    return keywords_to_grape_scores(desc.split())
```

```
def mpnet_desc_grape_scores(food_label):
    """MPNET raw centroids on auto wine-register description (full sentence)."""
    desc = keywords_to_food_description(food_label)
    vec = _mpnet.encode(
        [desc],
        show_progress_bar=False,
        convert_to_numpy=True,
        normalize_embeddings=True,
    )[0] # (768,)
    sims = _mpnet_matrix @ vec
    result = [(GRAPE_CLASSES[i], float(sims[i])) for i in range(len(GRAPE_CLASSES))]
    result.sort(key=lambda x: x[1], reverse=True)
    return result
```

[illegible]

```

"chocolate_cake", "cheese_plate", "ramen", "tacos",
"french_onion_soup", "oysters", "pad_thai", "beef_tartare",
"apple_pie", "bruschetta", "paella", "hummus",
]

_bd_sb, _bd_hg, _bd_bm = [], [], [] # BiLSTM + desc
_desc_sb, _desc_hg, _desc_bm = [], [], [] # MPNET + desc

print(f"{'Food':&lt;22} {'BiLSTM(kw)':&lt;20} {'BiLSTM(desc)':&lt;20}
{'MPNET(kw)':&lt;20} {'MPNET(desc)':&lt;20}")
print(" " + "■" * 100)

for food in _eval_foods:
    kws_c = food_flavor_table[food]["classic"]
    kws_s = food_flavor_table[food]["safe_bet"]

    # BiLSTM – keywords
    b_kw_sb = keywords_to_grape_scores(kws_c)[0][0]

    # BiLSTM – description
    bd_c = bilstm_desc_grape_scores(food)
    bd_sb = bd_c[0][0]
    bd_hg = next(g for g, _ in bilstm_desc_grape_scores(food) if g != bd_sb)
    bd_bm = bilstm_desc_grape_scores(food)[-1][0]
    _bd_sb.append(bd_sb); _bd_hg.append(bd_hg); _bd_bm.append(bd_bm)

    # MPNET – keywords
    m_kw_sb = mpnet_grape_scores(kws_c)[0][0]

    # MPNET – description
    d_c = mpnet_desc_grape_scores(food)
    d_sb = d_c[0][0]
    d_hg = next(g for g, _ in mpnet_desc_grape_scores(food) if g != d_sb)
    d_bm = mpnet_desc_grape_scores(food)[-1][0]
    _desc_sb.append(d_sb); _desc_hg.append(d_hg); _desc_bm.append(d_bm)

print(f" {food:&lt;22} {b_kw_sb:&lt;20} {bd_sb:&lt;20} {m_kw_sb:&lt;20}
{d_sb:&lt;20}")

print()

```







A consistent finding across all eight approaches: **food keyword lists are structurally unsuited as encoder input.** The evidence:

- W2V `expand\_keywords()` maps all food words to the same high-frequency wine terms (Approach 1)
- BiLSTM attention has nothing to attend to with 5 disconnected tokens — defaults to training prior (Approaches 2–3)
- Switching to 50-word wine-register descriptions changes the recommendation for 9/10 foods (Approach 4)
- MPNET fails completely on keywords (2/10 unique) but works when given sentences (5/10 unique) (Approaches 5 vs. 7)

Keywords fail because sentence encoders were trained on sentences. A 5-token fragment has no syntax, no discourse structure, no context for the attention mechanism to use. The encoder falls back to its statistical prior — which is the most frequent grape class in training data.

The `keywords\_to\_food\_description()` generator (defined in Approach 8) solved this by wrapping the same keyword content in wine-review grammatical structure. Same information, different register — and dramatically different encoder behaviour.

---

## **Conclusion: Why We Abandoned Grape-Based Pairing Entirely**

The eight experiments above converge on a single structural insight:

> **Grape variety is the wrong intermediate representation for wine pairing.**

This conclusion is not a matter of encoder quality or input format — it is a category error. We were treating grape variety as a proxy for flavor, but:

1. **Grape ≠ flavor.** A Sangiovese from Chianti Classico and a Sangiovese from Maremma have different acidity, tannin, body, and aromatic profiles. Averaging 50,000 Sangiovese reviews into a single centroid erases exactly the variation that makes pairing recommendations interesting.
2. **Any encoder's grape centroids will collapse.** Whether trained with cross-entropy (BiLSTM: 0.85–0.99 pairwise cosine) or contrastive loss (MPNET: 0.03–0.70), the act of averaging thousands of diverse wines into 15 points destroys the fine-grained structure the encoder learned.
3. **We were comparing two incomparable things.** A food's flavor profile is a specific sensory experience. A grape variety is a biological taxonomy that spans thousands of different sensory experiences. Cosine similarity between these two representations is geometrically well-defined but semantically meaningless.

---

## **The Architectural Response: Flavor Clusters (Section 14)**

The fix is not a better encoder or a better input format. It is a **different target space**:

| | Old architecture (Approaches 1–8) | New architecture (Section 14) |

|---|---|---|

| Target space | 15 grape centroids (biological taxonomy) | k flavor clusters (data-driven taste profiles) |

| How targets are built | Mean of all reviews per grape | k-means on BiLSTM `encode()` vectors of individual reviews |

| What a target represents | "Average Cabernet" (meaningless) | "Dark-fruit, tannic, full-body" (a real flavor profile) |

| Matching logic | Food → nearest grape centroid | Food → nearest flavor cluster → best wine *\*within\** that cluster |

| Diversity guarantee | None (centroids too close) | Structural (clusters are spread by construction) |

The BiLSTM encoder stays — it proved its value in Approach 8 as the best encoder for this domain. What changes is what we compare against: not grape averages, but flavor neighborhoods discovered directly from the review corpus via k-means.

This appendix is the proof that the new architecture was not chosen by intuition — it was forced by systematic elimination of every alternative.

---